



中华人民共和国国家标准化指导性技术文件

GB/Z 34052.2—2017

统计数据与元数据交换(SDMX) 第2部分:信息模型 统一建模语言 (UML)概念设计

Statistical data and metadata exchange(SDMX)—
Part 2: Information model—UML conceptual design

2017-07-31 发布

2018-02-01 实施

中华人民共和国国家质量监督检验检疫总局
中国国家标准化管理委员会 发布

目 次

前言	III
引言	IV
1 范围	1
2 规范性引用文件	1
3 术语和定义	1
4 信息模型概述	1
4.1 技术建模和图形注释	1
4.2 信息模型的功能	2
5 参与者和用例	4
5.1 概述	4
5.2 用例图	4
6 SDMX 基础包	9
6.1 综述	9
6.2 Base 结构中的标识、版本控制和维护	9
6.3 基本继承关系	13
6.4 数据类型	14
6.5 项方案模式	15
6.6 结构模式	18
7 具体项方案	25
7.1 简介	25
7.2 继承视图	25
7.3 代码表	25
7.4 概念方案和概念	27
7.5 分类方案	31
7.6 组织方案	33
7.7 报告分类	36
8 数据结构定义和数据集	39
8.1 简介	39
8.2 继承视图	39
8.3 数据结构定义的关系视图	42
8.4 数据集的关系视图	50
9 数据立方体	57
9.1 语境	57

9.2 支持信息模型中的数据立方体	57
10 元数据结构定义和元数据集	58
10.1 语境	58
10.2 继承	58
10.3 元数据结构定义	60
10.4 元数据集	64
11 层级代码表	70
11.1 范围	70
11.2 继承	71
11.3 关系	71
12 结构集和映射	74
12.1 范围	74
12.2 结构集	75
12.3 结构映射	77
12.4 项方案映射	80
12.5 混合代码表映射	83
13 约束	85
13.1 范围	85
13.2 继承	85
13.3 约束	86
14 数据供应	94
14.1 类图	94
14.2 类图说明	95
15 过程	98
15.1 简介	98
15.2 模型-继承和关系视图	99
16 转换和表达式	100
16.1 范围	100
16.2 模型-继承视图	101

前 言

《统计数据与元数据交换(SDMX)》目前包括 7 个部分,以后随着技术发展可能增加新的部分:

- 第 1 部分:框架;
- 第 2 部分:信息模型 统一建模语言(UML)概念设计;
- 第 3 部分:SDMX-ML 模式和文档(XML Schema);
- 第 4 部分:SDMX-EDI 语法和文档(EDI);
- 第 5 部分:注册规范 逻辑功能和逻辑接口;
- 第 6 部分:技术说明;
- 第 7 部分:Web 服务使用指南。

本部分为《统计数据与元数据交换(SDMX)》的第 2 部分。

本部分按照 GB/T 1.1—2009 给出的规则起草。

本部分由全国电子业务标准化技术委员会(SAC/TC 83)提出并归口。

本部分起草单位:中国标准化研究院、北京海泰方圆科技股份有限公司、杭州市标准化研究院。

本部分起草人:章建方、邢立强、李秀娣、蒋红宇、安晓江、柳增寿、柳晶。

引 言

统计数据与元数据交换(SDMX)是联合国统计署(UNSD)、国际结算银行(BIS)、国际货币基金(IMF)、经合发组织(OECD)、世界银行(WB)、欧洲中央银行(ECB)、欧盟统计局(EUROSTAT)等国际组织共同研制的标准。本部分参考《SDMX Standards; Section 2 information model; UML Conceptual Design version 2.1》[《统计数据与元数据交换(SDMX) 第2部分:信息模型 统一建模语言(UML)概念设计》2.1版本]编制。



统计数据与元数据交换(SDMX)

第 2 部分:信息模型 统一建模语言 (UML)概念设计

1 范围

《统计数据与元数据交换(SDMX)》的本部分给出了 SDMX 信息模型的概念元模型的详细视图,主要包括信息模型的结构、参与者和用例、SDMX 基础包、具体项方案、数据结构定义和数据集、数据立方体、元数据结构定义和元数据集、层级代码表、结构集和映射、约束、数据供应、过程、转换和表达式等。

本部分适用于统计数据与元数据的交换和共享。

2 规范性引用文件

下列文件对于本文件的应用是必不可少的。凡是注日期的引用文件,仅注日期的版本适用于本文件。凡是不注日期的引用文件,其最新版本(包括所有的修改单)适用于本文件。

GB/T 18391—2009 信息技术 元数据注册系统(MDR)

GB/T 28174—2011 统一建模语言(UML)

3 术语和定义

下列术语和定义适用于本文件。

3.1

供应定义 provisioning definition

维护机构参与者的一类角色,以支持报告和发布统计数据和元数据的过程,即何方在何时向他方报告了什么内容。

3.2

结构定义 structural definition

维护机构参与者的一类角色,以界定报告和发布统计数据和元数据的格式和内容。

4 信息模型概述

4.1 技术建模和图形注释

SDMX 信息模型(SDMX-IM)采用 GB/T 28174—2011 规定的建模技术。

UML 类图包含了属性和操作(有时也称为方法),但在图式化时允许省略其中的一个或两个分栏。本部分因为类没有包含操作,所以没有显示操作分栏,隐藏操作的类如图 1 所示。

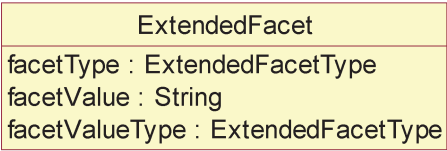


图 1 隐藏操作的类

在某些类图中,即使类可能存在一些属性,但还是可以将属性分栏取消,其目的是使类图更加清晰。属性分栏出现与否情况如下:

- 如果对类、其属性以及关联进行定义,则属性分栏应在类图中出现;
- 在诸如继承图的其他图中,为了获得清晰效果,则可以取消类的属性分栏,属性和操作都隐藏的类如图 2 所示。

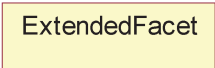


图 2 属性和操作都被隐藏的类

注:在任何情况下,从父类继承而来的属性都不应在子类中显示。

表 1 显示了定义类、属性和关联的结构。

表 1 定义类、属性和关联的结构

类	特征/属性	描述
ClassName(类名称)		
	attributeName(属性名称)	
	associationName(关联名称)	
	+ roleName(角色名称)	

“特征”列中的内容包含或解释了类的下列结构特征:

- 以斜体 *Courier* 字体表示的抽象类;
- 所继承的超类(如果有);
- 所包含的任意子类(如果有);
- 属性(以 *Courier* 字体的 attributeName 来表示);
- 关联(以 *Courier* 字体的 associationName 来表示);如果关联来源于超类之间的关联,则其格式为 /associationName;
- 角色(以 *Courier* 字体的 +roleName 来表示)。

“描述”分栏给出了类或特征的简短定义或解释。在描述中如使用 UML 类名,则类的英文名称采用正常字体且单词之间存在空格。例如,ConceptScheme(概念方案)类在描述中应表示为 Concept Scheme。

4.2 信息模型的功能

4.2.1 信息模型包概述

SDMX 信息模型(SDMX-IM)是一种概念元模型,它是具体实现中特定语法的基础。该模型是由一组功能包构成,这些功能包有助于模型的理解、复用和维护。为便于理解,这些功能包可以视作以下

3 种概念层中的某一层：

- SDMX 基础层(SDMX Base layer):包括结构化定义层、报告和发布层所使用的基础构建块；
- 结构化定义层(Structural Definitions layer):包括可支持数据与元数据报告和发布的结构化产物的定义；
- 报告和发布层(Reporting and Dissemination layer):包括报告和发布数据与元数据容器的定义。

实际运用中,这些层并没有隐含或明确的结构功能,因为任意一个包都可以使用其他包中的结构。

4.2.2 SDMX 信息模型包结构

与 2.0 版本相比,本版本的信息模型主要在元数据方面进行了扩展,但还是以多种方式对结构进行了定义,从而通过诸如 OLAP 等具有立方体类型结构知识的系统,以支持数据分析。与 2.0 版本相比,本信息模型增加的主要结构如下：

- 元数据结构定义；
- 元数据集；
- 层级代码表；
- 数据和元数据供应；
- 过程；
- 映射；
- 约束；
- 支持注册的结构。

此外,还用“数据结构定义”这个术语替代了术语“键族”:这两个术语都在不同群体使用,同时它们也是同义词。本模型采用了“数据结构定义”这个术语。SDMX 信息模型包结构如图 3 所示。

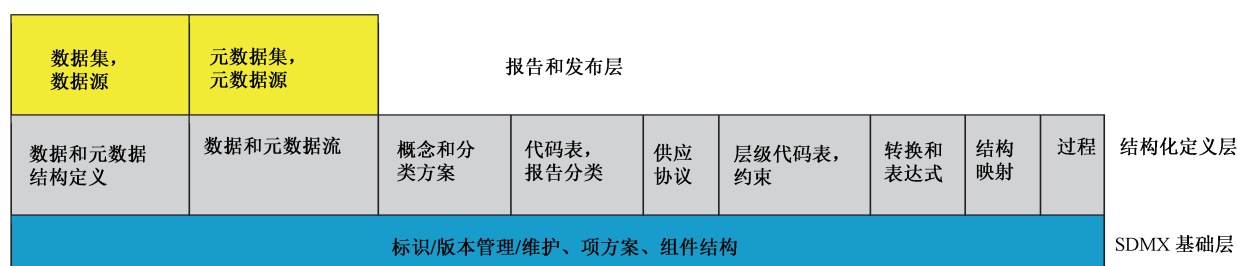


图 3 SDMX 信息模型包结构

基于应用场景的注册系统的其他结构在注册接口规范中进行了规定。注册接口规范中的结构信息包括：

- 订阅和通知；
- 注册；
- 发现。

注：注册系统除了使用注册功能所需的数据和元数据外,还可以利用本信息模型中的其他包。

包含注册库的 SDMX 信息模型包结构见图 4。

数据集，数据源	元数据集，元数据源	订阅和通知	数据和引用元数据注册		数据和引用元数据发现	报告和发布层			结构化定义层
数据和元数据结构定义	数据和元数据流	概念和分类方案	代码表，报告分类	供应协议	层级代码表，约束	转换和表达式	结构映射	过程	
标识/版本控制/维护、项方案、组件结构									SDMX 基础层

图 4 包含注册库的 SDMX 信息模型包结构

5 参与者和用例

5.1 概述

为了开发数据模型，应该理解基于需求定义的功能。这些功能在用例模型中进行了定义。用例模型由参与者和用例所组成。

● 参与者

参与者定义了一组角色，当系统与其发生交互作用时，这些角色可以是系统用户。参与者的实例可以是个人或外部系统。

● 用例

用例定义了一组用例实例，而每个实例都是由系统所执行的动作序列，并对特定角色产生可观察到的值的结果。

本模型的总体目标是在聚合统计数据和相关元数据领域中对数据和元数据进行报告、发布和交换。为了达到这一目标，模型应支持该过程的 3 个方面：

- 结构定义和供应定义的维护；
- 数据和引用元数据发布（报告）和使用；
- 对数据、引用元数据以及结构定义和供应定义的访问。

本部分涵盖了前两个方面，而本标准的注册逻辑模型部分则涵盖第 3 个方面。

5.2 用例图

5.2.1 结构化和供应定义的维护

5.2.1.1 用例

维护数据和元数据结构以及供应定义的用例如图 5 所示。

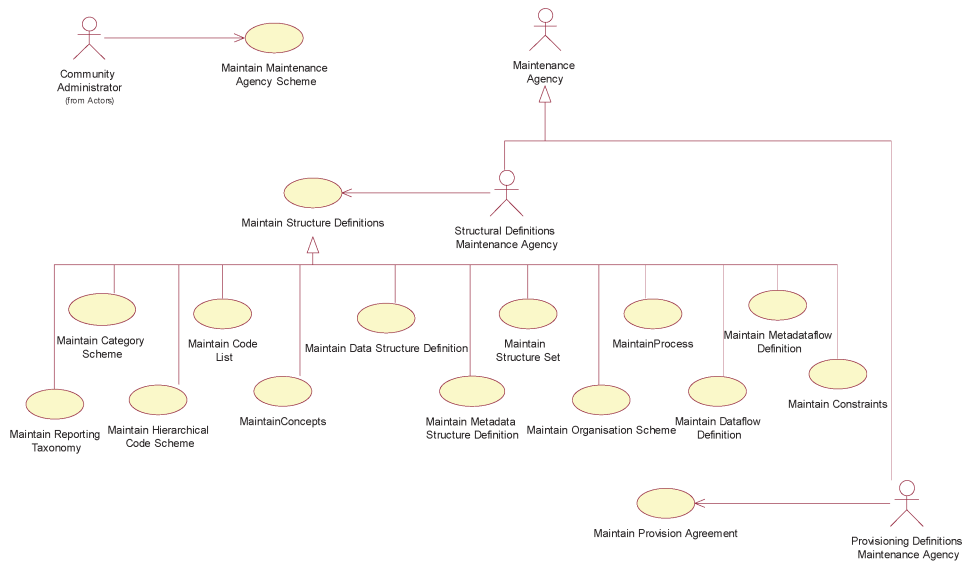


图 5 维护数据和元数据结构以及供应定义的用例

5.2.1.2 用例图的说明

为了使应用程序能够发布和使用数据和引用元数据,则应定义数据和引用元数据的结构和内容,并且能够被应用程序以及发布和使用过程的定义所使用。这项工作由维护机构负责。

维护机构参与者可以分为以下两类参与者角色:

- 维护结构定义;
- 维护供应定义。

虽然这两类角色的职责可由同一人来承担,或至少由同一维护机构来承担,但这些定义的目标不同,所以角色也就有所差异:结构定义对数据和引用元数据在报告或发布时的格式和内容进行界定,而供应定义则支持报告和发布过程(何方在何时向他方报告了什么内容)。

在基于组织的、至少结构定义可以共享的应用场景中,责任机构[此处称为组织管理员(Community Administrator)]对维护机构方案的维护很重要,同样地,维护机构(Maintenance Agency)Id的唯一性也很重要。

5.2.1.3 参与者和用例说明

表 2 给出了维护结构和供应定义的参与者和用例。

表 2 维护结构和供应定义的参与者和用例表

参与者	用例	说明
 组织管理员		对整个组织通用的结构定义进行管理的责任机构
	 管理维护机构方案	对组织(Community)维护机构顶层方案的创建和维护
 维护机构		负责维护结构化产物(如代码表、概念方案、DSD 的结构化定义、元数据结构定义)、数据和元数据供应产物(如供应协议)以及子维护机构。 子角色包括: 1) 结构定义维护机构(Structural Definitions Maintenance Agency) 2) 供应定义维护机构(Provisioning Definitions Maintenance Agency)
 结构定义维护机构		负责维护结构定义

表 2（续）

参与者	用例	说明
	 维护结构定义	对结构定义的维护。该用例对所维护的结构化产物都拥有子类用例
	 维护代码表  维护概念  维护分类方案  维护数据结构定义	创建和维护数据结构定义(DSD)、元数据结构定义(MSD)以及它们所使用的支撑物,比如代码表和概念
	 维护元数据结构定义  维护层级代码方案  维护报告分类  维护组织方案  维护过程  维护数据流定义  维护元数据流定义	包括了机构(Agency)、数据提供者(DataProvider)、数据使用者(Data Consumer)以及组织单元方案(Organisation Unit Scheme)

表 2（续）

参与者	用例	说明
 供应定义维护机构		负责维护数据和元数据供应定义
	 维护供应协议	供应定义的维护

5.2.2 数据和引用元数据的发布与使用

5.2.2.1 用例

数据和元数据发布和消费的参与者和用例如图 6 所示。

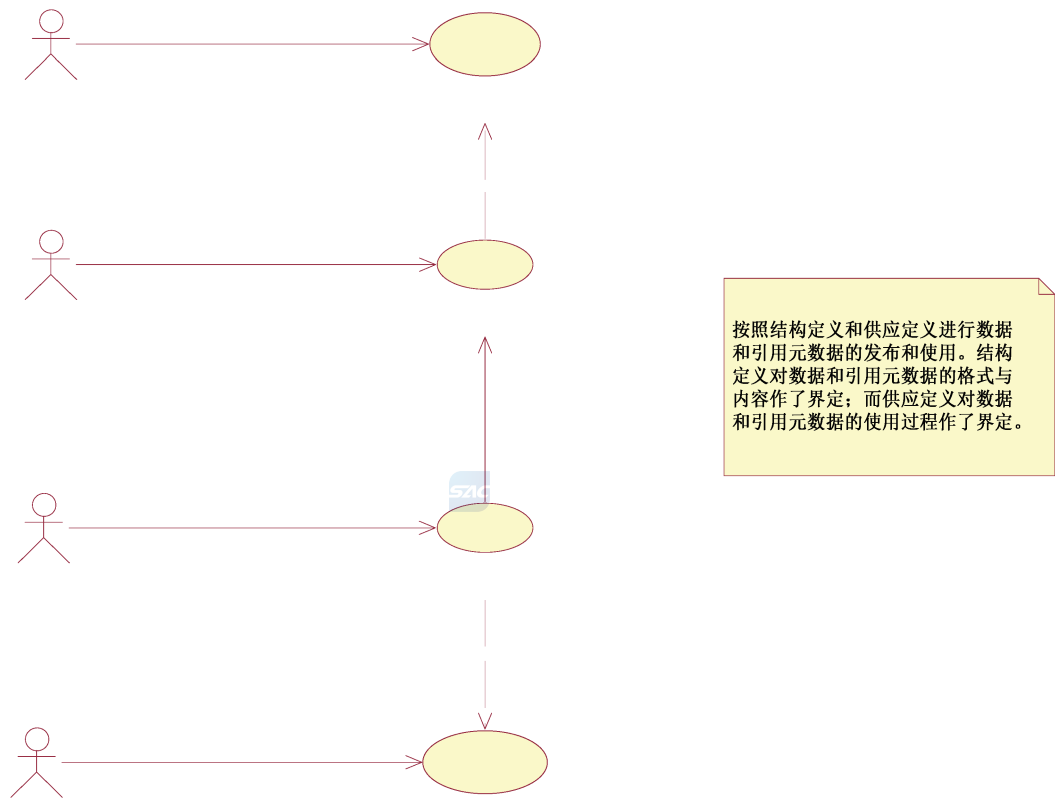


图 6 数据和元数据发布和消费的参与者和用例

5.2.2.2 用例图的说明

图 6 中,“发布”数据和引用元数据等同于“报告”数据和引用元数据。在某些情况下,数据的可用性可以起到上述两方面的作用。为了 Data Publisher(数据发布者)可以发布聚合数据,同时也为了应用程序可以处理数据和引用元数据,聚合数据的结构应是已知的。此外,应用程序还可能要求访问引用元数据,以便数据使用者更好地理解这些数据。正如数据一样,引用元数据也应按照可维护的结构进行格式化。除非数据或引用元数据已经“发布”,否则 Data Consumer(数据使用者)和 MetadataConsumer(元数据使用者)就无法使用数据或引用元数据,所以在“使用”用例与“发布”用例之间存在“数据源”或“元数据源”的依赖关系。

在数据和引用元数据的发布与使用时,发布和使用的应用程序都应访问 Provisioning Definitions(供应定义)。这些定义有可能简单,例如,何方在何时向他方提供了数据和引用元数据;这些定义也可能很复杂,例如,特定的发布者对发布的数据和元数据进行了约束;同时在数据共享场景中,数据和元数据来源于数据源,且数据源很详尽。

5.2.2.3 参与者和用例说明

数据和元数据发布和消费的参与者和用例如表 3 所示。

表 3 数据和元数据发布和消费的参与者和用例表

参与者	用例	说明
 数据发布者		根据 Data Structure Definition(数据结构定义)的定义以及相关供应定义来发布数据
	 发布数据	发布数据集。数据集可以是物理数据集,也可以是对数据源(如可以处理查询的数据库)进行访问而获得数据的方式
 数据使用者		数据的使用者。它可以是通过用户接口进行访问的具体的人,也可以是统计生产系统之类的应用程序
	 使用数据	对按照结构定义格式化并按照供应定义获得的数据进行使用。数据经常与元数据相链接,这样元数据可以驻留在不同地点并且可以独立发布和维护
 元数据发布者		按照规定的元数据结构定义以及相关的供应定义来发布引用元数据

表 3（续）

参与者	用例	说明
	 发布引用元数据	发布引用元数据集。 引用元数据集可以是物理元数据集，也可以是对元数据源（如可以处理查询的元数据仓库）进行访问而获得引用元数据的方式
 元数据使用者		引用元数据的使用者。它可以是通过用户接口进行访问的具体的人，也可以是统计生产系统或发布系统之类的应用程序
	 使用引用元数据	对按照结构化定义格式化并按照供应定义获得的引用元数据进行使用

6 SDMX 基础包

6.1 综述

SDMX 基础包中的结构包含了基础组件块，这些组件块支持模型中的许多其他结构。因此，该包中的许多类都是抽象类（也就是说，只有衍生子类可以存在于实现中）。

建立 SDMX 基础模型包的作用如下：

- 识别模型中的基础原型是一种公认的“最佳实践”；
- 将常见结构或“模式”进行标识更便于理解；
- 模式的标识有利于重用。

本章从不同角度给出了 SDMX 基础包中的各个类的视图，还给出了特定模式的详细视图，并概要给出了类之间的继承以及其他关系。

6.2 Base 结构中的标识、版本控制和维护

6.2.1 类图

SDMX 的标识、维护和版本控制如图 7 所示。

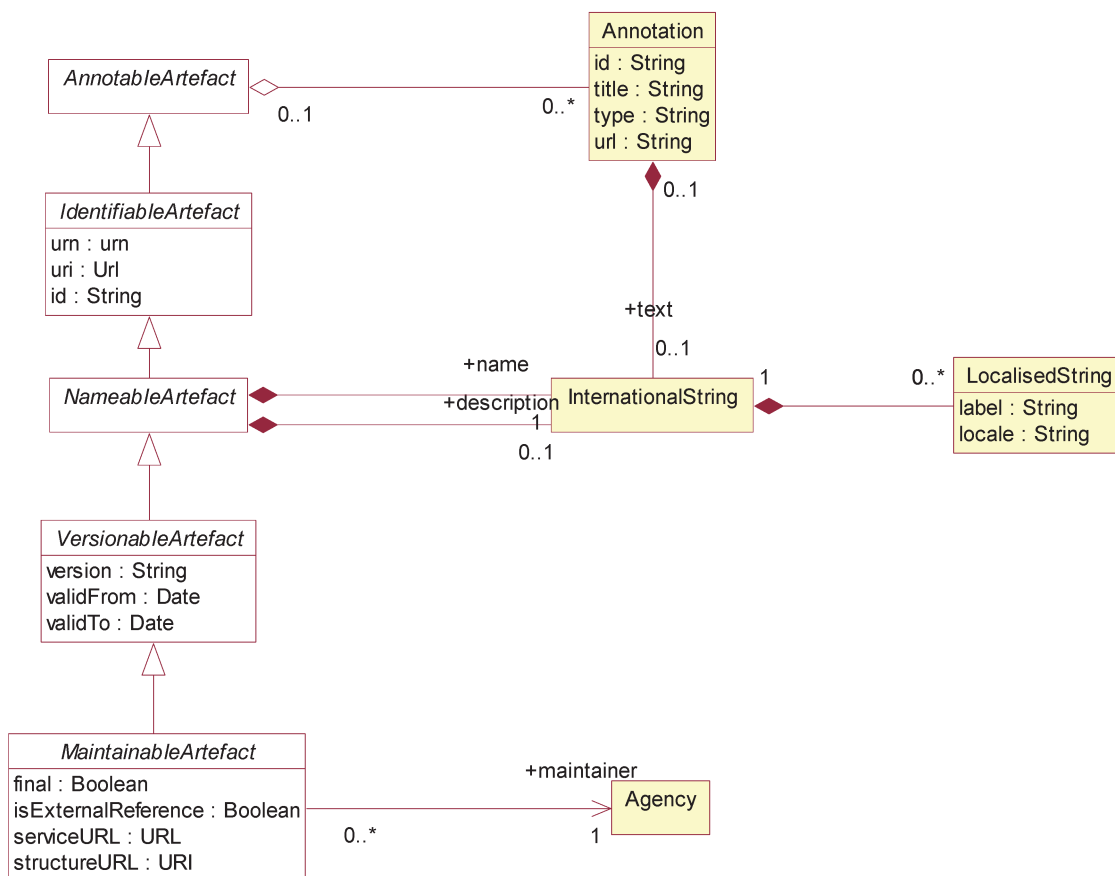


图7 SDMX 的标识、维护和版本控制类图

6.2.2 类图说明

6.2.2.1 描述

这组类是 SDMX 对象的管理核心。这组类给出的特性可供衍生类重用,以便支持诸如标识、版本控制等功能。

所有从抽象类 *AnnotableArtefact*(可注释产物)衍生的类都可以有注释(或备注):这样就满足了所有 SDMX-ML 元素增加备注的需求。*Annotation*(注释)用于表达描述 SDMX 结构的附加信息。这些信息可以是 URL 引用和/或多语种文本[由 *InternationalString*(国际字符串)的关联来表示]。

IdentifiableArtefact(可标识产物)是一个抽象类,它包含了标识所需的基本属性。基于 *IdentifiableArtefact*(可标识产物)的具体类全部都继承了可唯一标识的能力。

NameableArtefact(可命名产物)是继承于 *IdentifiableArtefact*(可标识产物)的一个抽象类,而 *+description* 和 *+name* 角色则支持基于 *NameableArtefact*(可命名产物)的所有对象的多语种描述和名称。*InternationalString*(国际字符串)支持以多地区语种描述的表示法(地区语种类似于语言,但是包含了地理上的差异,诸如加拿大法语、美式英语等)。*LocalisedString*(本地字符串)支持以某一种地区语言描述的表示法。

VersionableArtefact(可版本化产物)是一个抽象类,它继承于 *NameableArtefact*(可命名产物),并为所有从 *VersionableArtefact* 所衍生的类增加版本控制能力。

MaintainableArtefact(可维护产物)通过与 *Agency*(机构)的关联,使得衍生类得以维护,并增加

了位置信息(即在此位置该对象可以被检索到)。*MaintainableArtefact* 还可以用 *final*(是否最终的)属性来定义该产物是草案还是终稿。

从 *AnnotableArtefact*(可注释产物)至 *MaintainableArtefact*(可维护产物)的继承链允许 SDMX 类继承他们所需要的特征,包括简单的注释、标识、命名以及版本控制和维护。

6.2.2.2 标识、维护和版本控制类及其属性说明

SDMX 的标识、维护和版本控制类及其属性见表 4。

表 4 SDMX 的标识、维护和版本控制类及其属性

类	特征/属性	说明
<i>AnnotableArtefact</i>	Base 继承子类为: <i>IdentifiableArtefact</i>	该类所派生的类可以附加注释
<i>Annotation</i>		附加到一个对象的辅助描述信息
	id	<i>Annotation</i> (注释)的标识符。在同一个注释对象拥有多个 <i>Annotation</i> 时,它可以用来区别各个 <i>Annotation</i>
	title	用以标识注释的标题
	type	规定将如何处理注释
	url	与外部描述文本的链接
	+text	通过该角色, <i>International String</i> (国际字符串)给出了注释的多语种文本内容
<i>IdentifiableArtefact</i>	超类为 <i>AnnotableArtefact</i> Base 继承子类为: <i>NameableArtefact</i>	为所有衍生类提供标识。由于它是 <i>Annotable Artefact</i> (可注释产物)的子类,所以它还衍生类提供了注释
	id	对象的唯一标识符
	uri	可解析或不可解析的统一资源标识符
	urn	统一资源名称—在注册库中使用:所有注册的对象都应拥有 urn
<i>NameableArtefact</i>	超类为 <i>IdentifiableArtefact</i> Base 继承子类为: <i>VersionableArtefact</i>	在标识和注释之外为所有衍生类提供名称和说明
	+description	通过 <i>International String</i> (国际字符串)类由该角色提供多语种说明

表 4 (续)

类	特征/属性	说明
	+name	通过 International String(国际字符串)类由该角色提供多语种名称
InternationalString		International String(国际字符串)是 Localised String(本地字符串)的集合,并且支持多地区文本表示法
LocalisedString		Localised String(本地字符串)支持一个地区的文本表示法(地区类似于语言,但包含了地理差异,诸如加拿大法语、美式英语等)
	label	字符串的标签
	locale	字符串的当地地区语种,例如法语、加拿大法语
VersionableArtefact	超类为 <i>NameableArtefact</i> Base 继承子类为: <i>MaintainableArtefact</i>	为所有衍生对象提供版本信息
	version	遵循某种约定的版本字符串
	validFrom	版本开始生效的日期
	validTo	版本被替代的日期
MaintainableArtefact	继承自 <i>VersionableArtefact</i>	将机构维护的主要结构元数据产物进行组配的抽象类
	final	定义所维护产物是草稿还是终稿
	isExternalReference	如果此值设置为“真”,则表明此对象的内容在外部维护
	structureURL	包含外部对象的 SDMX-ML 文档的 URL
	serviceURL	符合 SDMX 规范的 web 服务的 URL,从该服务 URL 可以检索外部对象
	+maintainer	与负责维护该产物的 MaintenanceAgency(维护机构)的关联
Agency		参见“组织”章节

6.3 基本继承关系

6.3.1 源于 Base 的基本继承关系类图

图 8 给出了源于基础(Base)结构的基本继承关系类图。

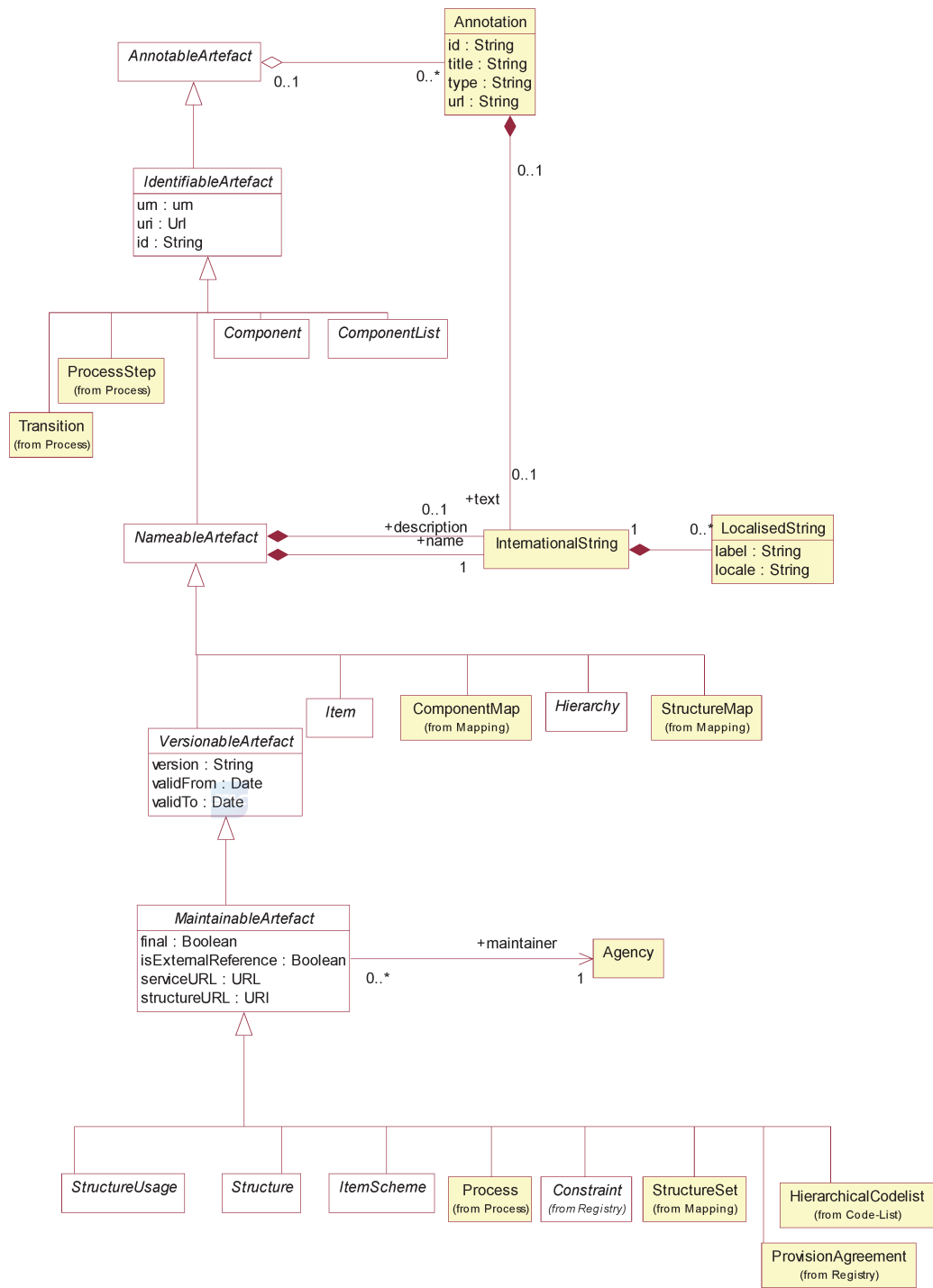


图 8 源于基础结构的基本继承关系类图

6.3.2 类图说明

图 8 给出了基础结构中的继承关系。在与它们相关联的特定包中引入并定义了具体类。

6.4 数据类型

6.4.1 类图

图 9 给出了基本数据类型的类图。

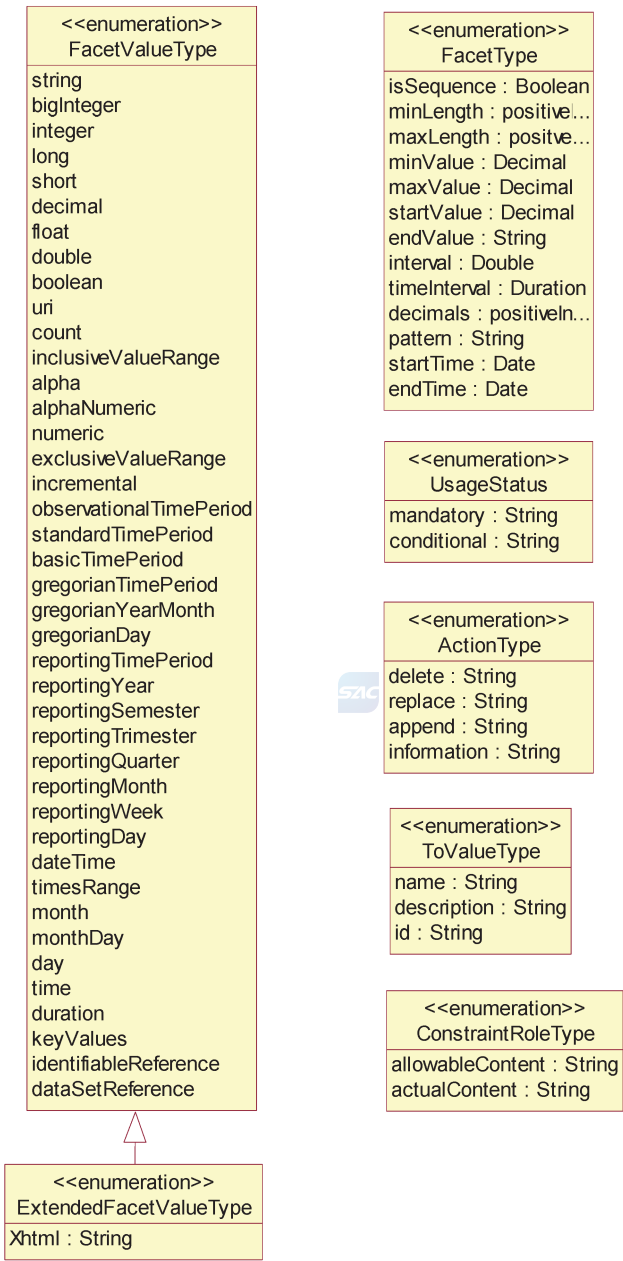


图 9 基本数据类型的类图

6.4.2 类图说明

UsageStatus(使用状态)枚举作为 DataAttribute(数据属性)(来自 DSD)的一种数据类型来使用,类实例中的属性值应取自 UsageStatus(使用状态)(即,强制型、条件型)。

当 FacetType(刻面类型)和 FacetValueType(刻面值类型)这些枚举用在 Structure(结构)中 Component(组件)[比如 DataStructureDefinition(数据结构定义)中的 dimension(维度)]时,这些枚举用来规定非枚举型的 Concept(概念)内容的有效格式,或用来规定 Concept(概念)的用法。不同类型的说明可以在 ConceptScheme(概念方案)(见 7.4)中找到。

在处理行为对象时,应采用 ActionType(行为类型)枚举进行规定。其枚举如下:

- 添加(Append)
现有数据/元数据集的更新,或者增加新的数据或新的属性值(以文档形式提供)。
- 替换(Replace)
数据/元数据被替换。
- 删除>Delete)
数据/元数据被删除。
- 解释信息(Information)
数据和元数据仅作为解释信息。

IdentifiableObjectType(可标识对象类型)枚举用来规定对象类型,此对象类型的类是 IdentifiableArtefact(可标识产物)的子类。

ToValueType(至数值类型)数据类型包含了可以支持转换的属性,转换在 StructureMap(结构映射)(见第 12 章)中进行了定义。

ConstraintRoleType(约束角色类型)数据类型包含了能够标识 Constraint(约束)用途的属性[allowableContent(允许内容)、actualContent(实际内容)]。

6.5 项方案模式

6.5.1 语境

Item Scheme(项方案)是一种基本的体系架构模式,它允许生成用于简单分类体系的方案列表。

ItemScheme(项方案)是 CategoryScheme(分类方案)、CodeList(代码表)、ConceptScheme(概念方案)、ReportingTaxonomy(报告分类)和 OrganisationScheme(组织方案)的基础。

6.5.2 类图

图 10 给出了项方案模式的类图。

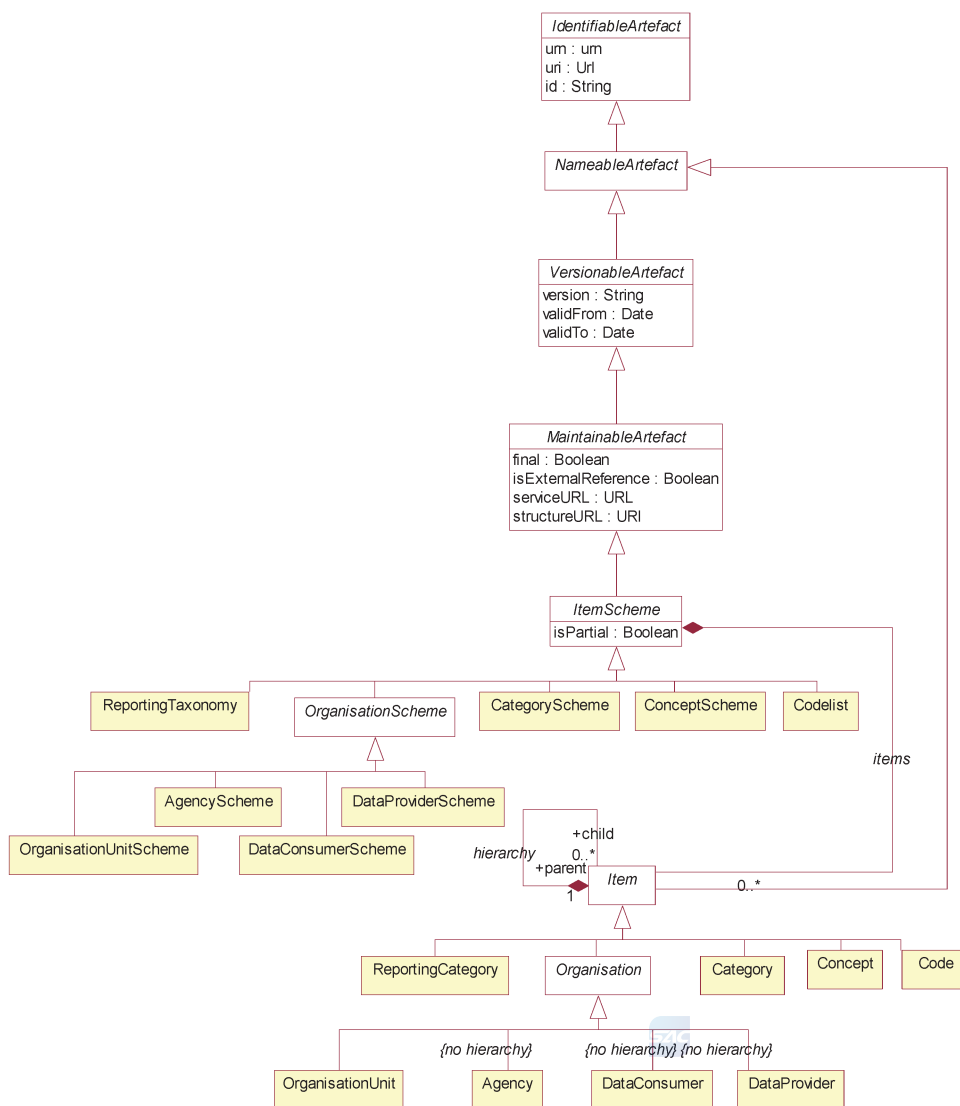


图 10 项方案模式类图

6.5.3 类图说明

6.5.3.1 描述

ItemScheme(项方案)是定义了一组项(*Item*)(项类也是抽象类)的抽象类,其主要目的是定义一种机制,这种机制可以用于创建划分 SDMX 信息模型其他部分的分类。项方案衍生自能够为其提供注释的 *MaintainableArtefact*(可维护产物),而 *MaintainableArtefact*(可维护产物)拥有标识、命名、版本控制并且与 *Agency*(机构)相关联。其具体类的一个范例就是 *CategoryScheme*(分类方案)。关联的 *Category*(分类)即为 *Items*(项)。

在数据交换环境中,一个 *ItemScheme*(项方案)可以包含受维护的 *ItemScheme*(项方案)类中 *Items*(项)的子集。如果 *ItemScheme*(项方案)随 *Items*(项)的子集进行发布,则 *ItemScheme*(项方案)的 *isPartial* 属性值为“真”。

“部分”*ItemScheme*(项方案)不能以部分形式进行独立维护,也就是说,部分的项方案不能包含完

整 *ItemScheme*(项方案)中并不存在的 *Items*(项),而且任何一个 *Item*(项)的内容(例如名称和说明)都不能偏离完整项方案中的内容。而且,“isPartial”属性值为“真”的 *ItemScheme*(项方案)的 id 与完整 *ItemScheme*(项方案)(维护机构、id、版本)的 id 相同。这一点之所以重要是因为此 id 在其他结构中也会被引用(例如,Codelist(代码表)在 DSD 中被引用),所以无论这个发布出去的 *ItemScheme*(项方案)是完整 *ItemScheme*(项方案)还是部分 *ItemScheme*(项方案),此 id 总是保持相同的。

部分 *ItemScheme*(项方案)的用途是支持子集 *ItemScheme*(项方案)的交换和发布,同时无需维护包含了相同 *Items*(项)的多个 *ItemScheme*(项方案)。例如,当 Codelist(代码表)用在 DataStructure-Definition(数据结构定义)中时,它可能只与 Codelist(代码表)中 Code(代码)的某一个子集相关。在这种情况下,通过使用本规范的约束机制来构建部分 Codelist(代码表)。

Item(项)继承自 *NameableArtefact*(可命名产物),这种继承赋予了项获得注释和标识的能力,因此项可以拥有 id、uri 和 urn 属性,以及 InternationalString(国际字符串)形式的名称和描述。与父 *ItemScheme*(项方案)不同,*Item*(项)本身并不属于 *MaintainableArtefact*(可维护产物),因此不能拥有独立的 Agency(机构)(即,项中隐含与 *ItemScheme*(项方案)相同的机构)。

Item(项)可以是有层级的,因此一个 *Item*(项)可以拥有多个子 *Items*(项)。层级关联的约束是一个子 *Item*(项)只能拥有一个父 *Item*(项)。

6.5.3.2 项方案、项及其属性说明

项方案、项及其属性如表 5 所示。

表 5 项方案、项及其属性表

类	特征/属性	说明
<i>ItemScheme</i>	继承自: <i>MaintainableArtefact</i> 直接子类为: <i>CategoryScheme</i> <i>ConceptScheme</i> <i>Codelist</i> <i>ReportingTaxonomy</i> <i>OrganisationScheme</i>	可根据对象共同拥有的特性将对象安排或划分成多个组的描述性信息
	isPartial	标明该 <i>ItemScheme</i> (项方案)是否包含受维护的项目中 <i>Items</i> (项)全集的子集
	Items	与方案中 <i>Items</i> (项)的关联
<i>Item</i>	继承自: <i>NameableArtefact</i> 直接子类为: <i>Category</i> <i>Concept</i> <i>Code</i> <i>ReportingCategory</i> <i>Organisation</i>	项(<i>Item</i>)是 <i>ItemScheme</i> (项方案)中的一个内容项。该项可以是某一分类或本体中的节点、某个代码表中的一个代码等。概念层级中的组织节点是没有层级的
	hierarchy	它允许项拥有一个或多个子项

6.6 结构模式

6.6.1 语境

结构模式是一种规定统计数据(诸如 DSD 数据结构定义和 MSD 元数据结构定义)中常见的复杂表格结构的基本架构模式。结构本身是有序列表的集合。目前已经开发了支持这种表格结构的模式,因此这些结构定义之间的共通性可以被通用软件和通用语法结构所支持。

6.6.2 类图

结构模式的类图、结构模式中的表示法分别如图 11、图 12 所示。

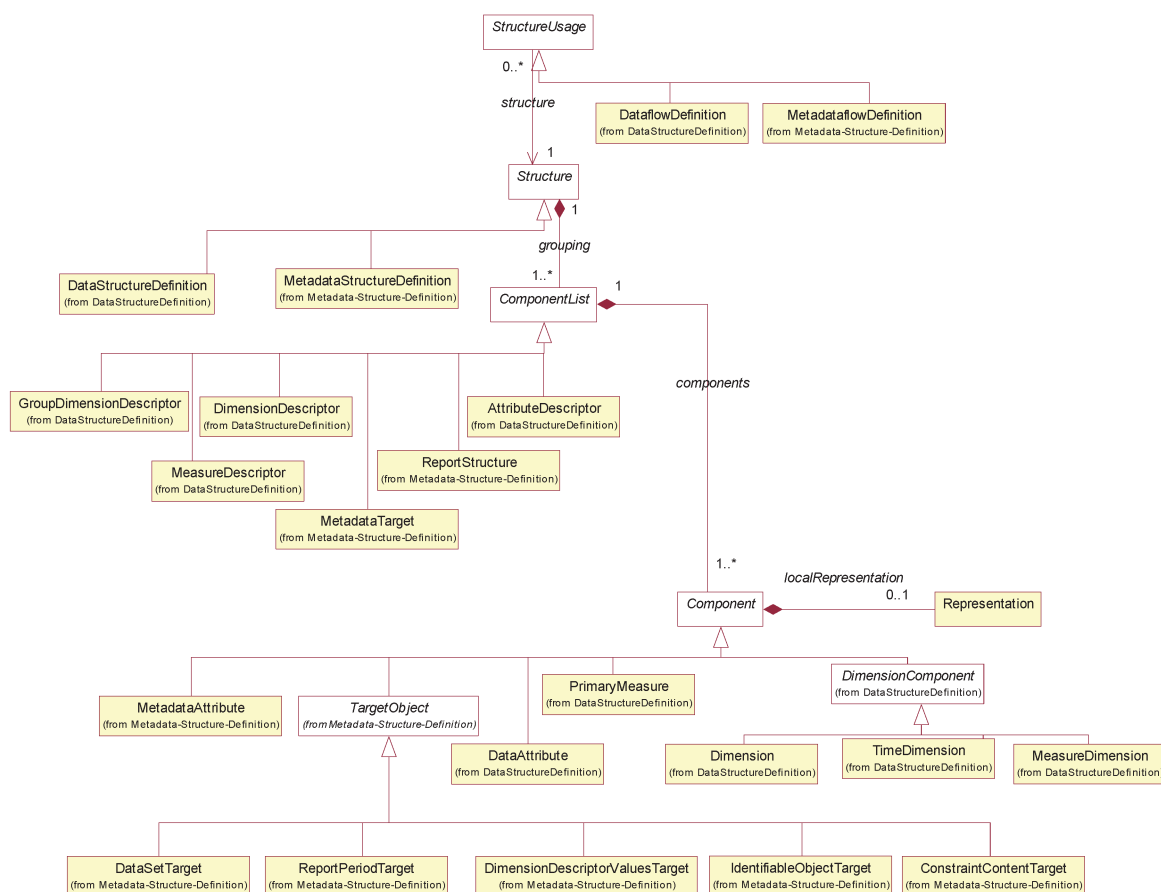


图 11 结构模式的类图

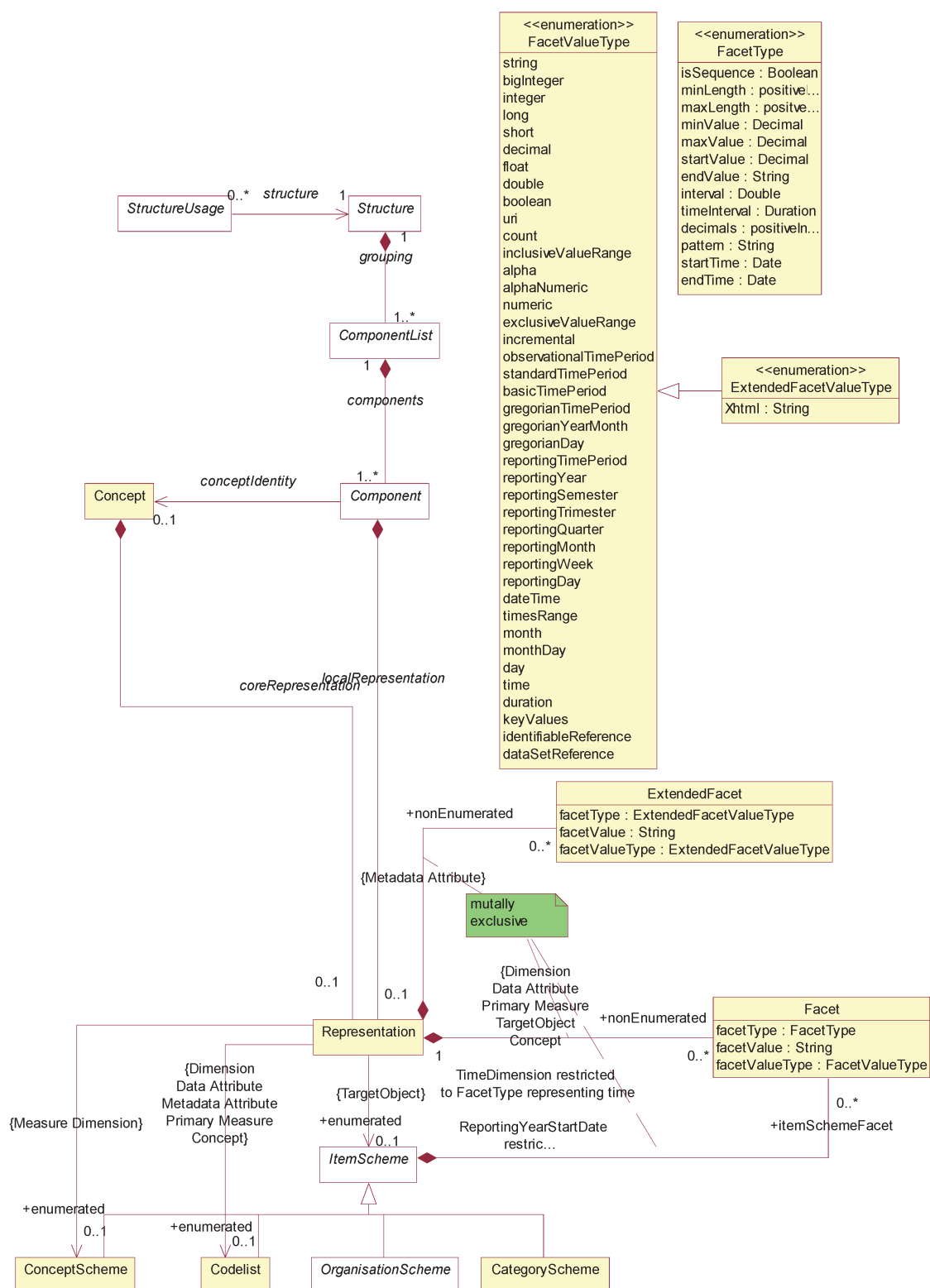


图 12 结构模式中的表示法



6.6.3 类图说明

6.6.3.1 描述

Structure(结构)是包含一组或多组 *ComponentList*(s)(组件列表)的一个抽象类, (*ComponentList* 也是抽象类)。 *Structure*(结构)的一个具体范例是 *DataStructureDefinition*(数据结构定义)。

ComponentList(组件列表)是包含了一个或多个 *Component*(s)(组件)的列表。 *ComponentList*(组件列表)拥有多个以其为基础的具体描述符类: *DataStructureDefinition*(数据结构定义)的 *DimensionDescriptor*(维度描述符)、 *GroupDimensionDescriptor*(组维度描述符)、 *MeasureDescriptor*(测量描述符)和 *AttributeDescriptor*(属性描述符)类, 以及 *MetadataStructureDefinition*(元数据结构定义)的 *ReportStructure*(报告结构)和 *MetadataTarget*(元数据目标)类。

Component(组件)包含在 *Componentlist*(组件列表)中。 *Componentlist*(组件列表)中的 *Component*(组件)类型依赖于下列 *Componentlist*(组件列表)中的具体类:

- *DimensionDescriptor*(维度描述符): *Dimension*(维度)、 *Measure Dimension*(度量维度)、 *Time Dimension*(时间维度);
- *GroupDimensionDescriptor*(组维度描述符): *Dimension*、 *Measure Dimension*、 *Time Dimension*;
- *MeasureDescriptor*(测量描述符): *PrimaryMeasure*(主要测量);
- *AttributeDescriptor*(属性描述符): *DataAttribute*(数据属性);
- *MetadataTarget*(元数据目标): *TargetObject*(目标对象)及其子类;
- *ReportStructure*(报告结构): *MetadataAttribute*(元数据属性)。

每个 *Component*(组件)都从 *ConceptScheme*(概念方案)中的某个 *Concept*(概念)取得其语义(并有可能还取得其表示法)。这可以由与 *Concept*(概念)相关联的 *conceptIdentity*(概念标识符)来表示。

Component(组件)也有可能拥有 *localRepresentation*(本地表示法), 这样就允许具体类[如 *Dimension*(维度)]来规定其表示法, 该表示法对于包含此组件的 *Structure*(结构)来说是本地的[对于 *Dimension*(维度)来说就是 *DataStructureDefinition*(数据结构定义)], 从而不考虑用于规定 *Concept*(概念)的任何 *coreRepresentation*(核心表示法)。

Representation(表示法)可以是枚举型或非枚举型。枚举型表示法的有效内容可以采用 *ItemScheme*(项方案)规定, 而 *ItemScheme*(项方案)可以是 *ConceptScheme*(概念方案)、 *Codelist*(代码表)、 *OrganisationScheme*(组织方案)、 *CategoryScheme*(分类方案)、 *ReportingTaxonomy*(报告分类)。非枚举型表示法的有效内容是一个或多个 *Facet*(刻面)(例如, 这些刻面可以规定最小值和最大值)。对于 *MetadataAttribute*(元数据属性)来说, 允许采用附加的 XHTML 表示法的一个或多个 *Extended Facet*(扩展刻面)来实现。

特定组件的有效表示法的类型可作为关联的约束来进行表达, 即:

- *MeasureDimension*(测量维度)应为枚举型并采用 *ConceptScheme*(概念方案);
- *Dimension*(维度)[但并非 *MeasureDimension*(测量维度)]、 *DataAttribute*(数据属性)、 *PrimaryMeasure*(主要测量)、 *MetadataAttribute*(元数据属性)都可以是枚举型, 如果这样, 则采用 *Codelist*(代码表);
- *TargetObject*(目标对象)可以是枚举型, 如果这样, 则采用 *ItemScheme*(项方案)(包括 *Codelist*、 *ConceptScheme*、 *OrganisationScheme*、 *Category*、 *ReportingTaxonomy*);
- *Dimension*(维度)[但并非 *MeasureDimension*(测量维度)]、 *Data Attribute*(数据属性)、 *PrimaryMeasure*(主要测量)、 *TargetObject*(目标对象)可以是非枚举型, 如果这样, 则采用一个或多个

个 Facet(剖面);

注: 适用于 TimeDimension(时间维度)的 FacetValueType(面值类型)仅限于表示时间的剖面值类型。

——MetadataAttribute(元数据属性)可以是非枚举型,如果这样,则采用一个或多个 ExtendedFacet(扩展剖面)。

Structure(结构)可以被一个或多个 StructureUsage(结构用法)所采用。它的具体类的一个范例是 DataflowDefinition(数据流定义)[(StructureUsage(结构用法)的子类)可以使用一个特定的 Data-StructureDefinition(数据结构定义)[结构(Structure)的子类],并且类似的结构也适用于 Metadata-flowDefinition(元数据流定义)[链接至 MetadataStructureDefinition(元数据结构定义)]。

6.6.3.2 结构用法、结构等的属性说明

结构用法、结构等的属性如表 6 所示。

表 6 结构用法、结构等的属性表

类	特征/属性	说明
StructureUsage	继承自: MaintainableArtefact 子类为: DataflowDefinition MetadataflowDefinition	其组件是由结构(Structure)所描述的一种产物。在具体类(子类)方面的一个范例是数据流定义(Dataflow Definition),该数据流定义(Dataflow Definition)链接至一个给定的结构——此处的结构即为数据结构定义(Data Structure Definition)
	structure	与规定了产物结构的 Structure(结构)相关联
Structure	继承自: MaintainableArtefact 子类为: DataStructure Definition MetadataStructure Definition	列表的抽象规定,以便定义复杂的表格结构。一个具体范例是统计概念、代码表及它们在数据或元数据结构定义中的组织,数据或元数据结构定义由某一机构所定义(通常用来与合作伙伴交换统计信息)
	grouping	与一个或多个组件列表的复合关联
ComponentList	继承自: IdentifiableArtefact 子类为: DimensionDescriptor GroupDimension Descriptor MeasureDescriptor AttributeDescriptor MetadataTarget ReportStructure	组件列表的抽象定义。一个具体范例是维度描述符(Dimension Descriptor),该描述符定义了数据结构定义(DSD)中维度的列表

表 6 (续)

类	特征/属性	说明
	components	与组成列表的一个或多个组件的聚合关联
Component	继承自: <i>IdentifiableArtefact</i> 子类为: <i>PrimaryMeasure</i> <i>DataAttribute</i> <i>DimensionComponent</i> <i>TargetObject</i> <i>MetadataAttribute</i>	组件是定义定性和定量数据和元数据项的抽象父类,而这些数据和元数据项属于组件列表(Component List)以及由此所组成的结构(Structure)。组件通过其子类进行细化
	conceptIdentity	与概念方案(Concept Scheme)中的概念(Concept)进行关联,该概念方案标识和定义了组件的语义
	localRepresentation	如果组件的表示法与概念的核心表示法(coreRepresentation),用于关联组件表示法
Representation		组件或概念的允许值或格式
	+ enumerated	与枚举型列表的关联,该列表包含了数据或元数据集中组件所采用的内容。具体组件的枚举型列表类型在关联约束中进行显示(例如,标识符组件)可以拥有项方案(Item Scheme)的子类,而测量维度(Measure Dimension)拥有概念方案(Concept Scheme)
	+ nonEnumerated	与一组刻面(Facets)的关联,该组刻面定义了数据或元数据集中组件内容的允许格式
Facet		定义了数据或元数据集中组件内容的格式
	facetType	由刻面类型(FacetType)枚举所约束的特定内容类型

表 6 (续)

类	特征/属性	说明
	facetValueType	数据或元数据集中组件值的格式,受刻面值类型(FacetValueType)枚举的约束
	+ itemSchemeFacet	定义了组件所使用的项方案中的标识符格式,通常用于定义标识符的字符数量(长度)
ExtendedFacet		与刻面(Facet)功能相同,但也允许采用 XHTML 表示法。在使用时,受元数据属性的约束

ComponentList(组件列表)和 Component(组件)子类内容和用法的规范可以在使用了它们的第 8 章 DataStructureDefinition(数据结构定义)和第 10 章 MetadataStructureDefinition(元数据结构定义)中找到。

6.6.3.3 表示法结构

大多数的 SDMX FacetValueTypes(刻面值类型)与 XML Schema 的刻面值类型相兼容,并且在当前绝大多数实现平台上都存在等效的数据类型,SDMX 刻面值类型、XML Schema 数据类型等见表 7。

表 7 SDMX、XML Schema、NET Framework 类型、Java 等数据类型表示法之间对应关系

SDMX 刻面值类型	XML Schema 数据类型	.NET Framework 类型	Java 数据类型
String	xsd:string	System.String	java.lang.String
Big Integer	xsd:integer	System.Decimal	java.math.BigInteger
Integer	xsd:int	System.Int32	int
Long	xsd:long	System.Int64	long
Short	xsd:short	System.Int16	short
Decimal	xsd:decimal	System.Decimal	java.math.BigDecimal
Float	xsd:float	System.Single	float
Double	xsd:double	System.Double	double
Boolean	xsd:boolean	System.Boolean	boolean
URI	xsd:anyURI	System.Uri	Java.net.URI 或 java.lang.String
DateTime	xsd:dateTime	System.DateTime	javax.xml.datatype.XMLGregorianCalendar
Time	xsd:time	System.DateTime	javax.xml.datatype.XMLGregorianCalendar

表 7（续）

SDMX 刻面值类型	XML Schema 数据类型	.NET Framework 类型	Java 数据类型
GregorianYear	xsd:gYear	System.DateTime	javax.xml.datatype. XMLGregorianCalendar
GregorianMonth	xsd:gYearMonth	System.DateTime	javax.xml.datatype. XMLGregorianCalendar
GregorianDay	xsd:date	System.DateTime	javax.xml.datatype. XMLGregorianCalendar
Day, MonthDay, Month	xsd:g*	System.DateTime	javax.xml.datatype. XMLGregorianCalendar
Duration	xsd:duration	System.TimeSpan	javax.xml.datatype. Duration

也有一些 SDMX 数据类型并没有与之直接对应的数据类型,通常这是因为它们是复合表示法或者是更宽泛的数据类型的约束。这些将在本标准的第 6 部分中详细说明。

Representation(表示法)由 Facets(刻面)构成,每个刻面表达了与值域定义相关的特性信息。经常需要一组刻面来表达所要求的语义。例如,一个序列最少要由 2 个刻面定义:一个刻面定义起始值,一个刻面定义间隔。表 8 给出了刻面类型的解释。

表 8 刻面类型

刻面类型	解释
isSequence	“isSequence”刻面指出相关数值是否有序,它可以与 interval(间隔)、startValue(起始值)和 endValue(结束值)刻面或 timeInterval(时间间隔)、startTime(起始时间)和 endTime(结束时间)刻面配合使用。如果该属性值为真,则应提供一个起始值或起始时间以及一个数值间隔或时间间隔。如果没有给出结束值,则此序列将无限继续下去
interval	interval(间隔)属性规定了序列中所允许的间隔(增量)。 如果使用该属性,则 isSequence 属性值应为真
startValue	startValue(起始值)刻面应与 isSequence(是否有序)和 interval(间隔)刻面一起使用(应在顺序设置时才使用该刻面)。该刻面用于数值序列,并指出了序列的起始点。在表达数值序列时,则该刻面值应为必选型
endValue	endValue(结束值)刻面与 isSequence 和 interval(间隔)刻面一起使用(应在顺序设置才使用该刻面)。该刻面用于数值序列,并指出序列的结束点(如存在的话)
timeInterval	timeInterval(时间间隔)刻面指出了时间序列中的允许间隔期。若使用该刻面,则 isSequence 刻面值为真
startTime	“starTime”(起始时间)刻面与 isSequence(是否有序)和时间间隔(timeinterval)刻面一起使用(应在顺序设置时才使用该刻面)。该属性用于时间序列,并指出序列的起始时间。在表达时间序列时,该值为必选型

表 8（续）

剖面类型	解释
endTime	endTime(结束时间)剖面与 isSequence(是否序列)和时间间隔(timeinterval)剖面一起使用(应在顺序设置时才使用该剖面)。该剖面用于时间序列,并且指出序列的结束点(如存在的话)
minLength	minLength(最小长度)剖面以字符形式规定了数值的最小长度
maxLength	maxLength(最大长度)剖面以字符形式规定了数值的最大长度
minValue	minValue(最小数值)剖面可用在包括和不包括的范围,并指出了范围的下边界。如果它与包括的范围一起使用,则有效值应大于或等于该数值。如果数据类型未说明是否包含边界值(例如,剖面与一个整型数据类型一起使用时),则该数值假设为包含边界值
maxValue	maxValue(最大数值)剖面可用于包括和不包括的范围,并指出了范围的上边界。如果它与包括的范围一起使用,则有效值应小于或等于该数值。如果数据类型未说明是否包含边界值(例如,剖面与一个整型数据类型一起使用时),则该数值假设为包含边界值
decimals	小数位数剖面规定了在小数点后所允许的字符个数
pattern	模式属性具有实现语法(例如 W3C XML Schema)所允许的规则表达式

7 具体项方案

7.1 简介

结构是基于特性的对象以层级形式或列表形式进行的组织,层级或列表作为从 ItemScheme 继承而来的组进行维护。ItemScheme 的具体类为:

- Codelist(代码表);
- ConceptScheme(概念方案);
- CategoryScheme(分类方案);
- OrganisationScheme(组织方案), AgencyScheme(机构方案)、DataProviderScheme(数值供应者方案)、DataConsumerScheme(数据使用者方案)、organisationUnitScheme(组织单元方案), 这些方案都继承自抽象类 OrganisationScheme(组织方案);
- Reporting Taxonomy(报告分类)。

7.2 继承视图

继承和关系视图将在每个类图中一同显示。

7.3 代码表

7.3.1 类图

代码表的类图如图 13 所示。

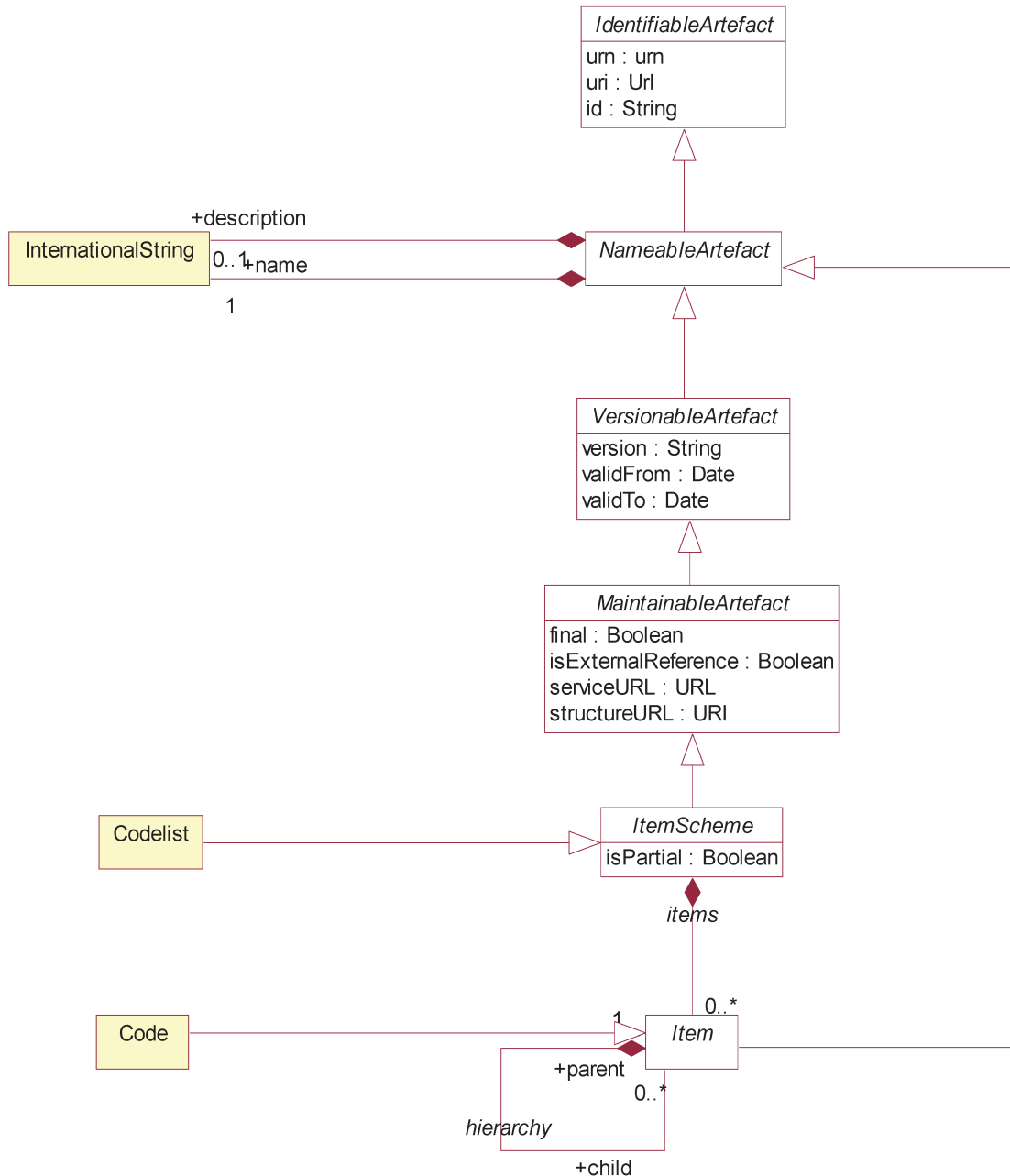


图 13 代码表的类图

7.3.2 类图说明

7.3.2.1 描述

Codelist(代码表)继承自 ItemScheme (项方案),因此具有以下属性:

- id
- uri
- urn
- version
- validFrom

- validTo
- isExternalReference
- serviceURL
- structureURL
- final
- isPartial

Code(代码)继承自 Item(项)并具有以下属性:

- id
- uri
- urn

Codelist(代码表)和 Code(代码)都与 InternationalString(国际字符串)相关联,以支持多语种名称、可选的多语种说明,并与 Annotation(注释)相关联以支持备注(图中未显示)。

通过继承,Codelist(代码表)包含了一个或多个 Codes(代码),而 Codes(代码)本身可以在(继承的)hierarchy(层级)关联中拥有零个或多个子代码。

注:在关联中,一个子 Code(代码)只能拥有一个父 Code(代码)。后文将描述一种更为复杂的 HierarchicalCodelist(层级代码表),它允许存在多个父代码和多种层级。

部分 Codelist(代码表)(其中的 isPartial 属性值为“真”)与 Codelist(代码表)一样,包含了 Code(代码)以及相关名称和说明,就像正常的代码表一样。然而,其内容只是完整 Codelist(代码表)的子集。这种方式在“ItemScheme(项方案)”中进行了说明。

7.3.2.2 代码表和代码的属性说明

代码表和代码的属性如表 9 所示。



表 9 代码表和代码的属性表

类	特征/属性	说明
Codelist	继承自 ItemScheme	某些统计概念(已编码的概念)可以取得其值的列表
Code	继承自 Item	与语种无关的字母、数字或符号集合,该集合可以表示其含义是由自然语言所描述的概念
	/hierarchy	将父代码与子代码关联起来

7.4 概念方案和概念

7.4.1 概念方案的继承关系类图

概念方案的类图如图 14 所示。

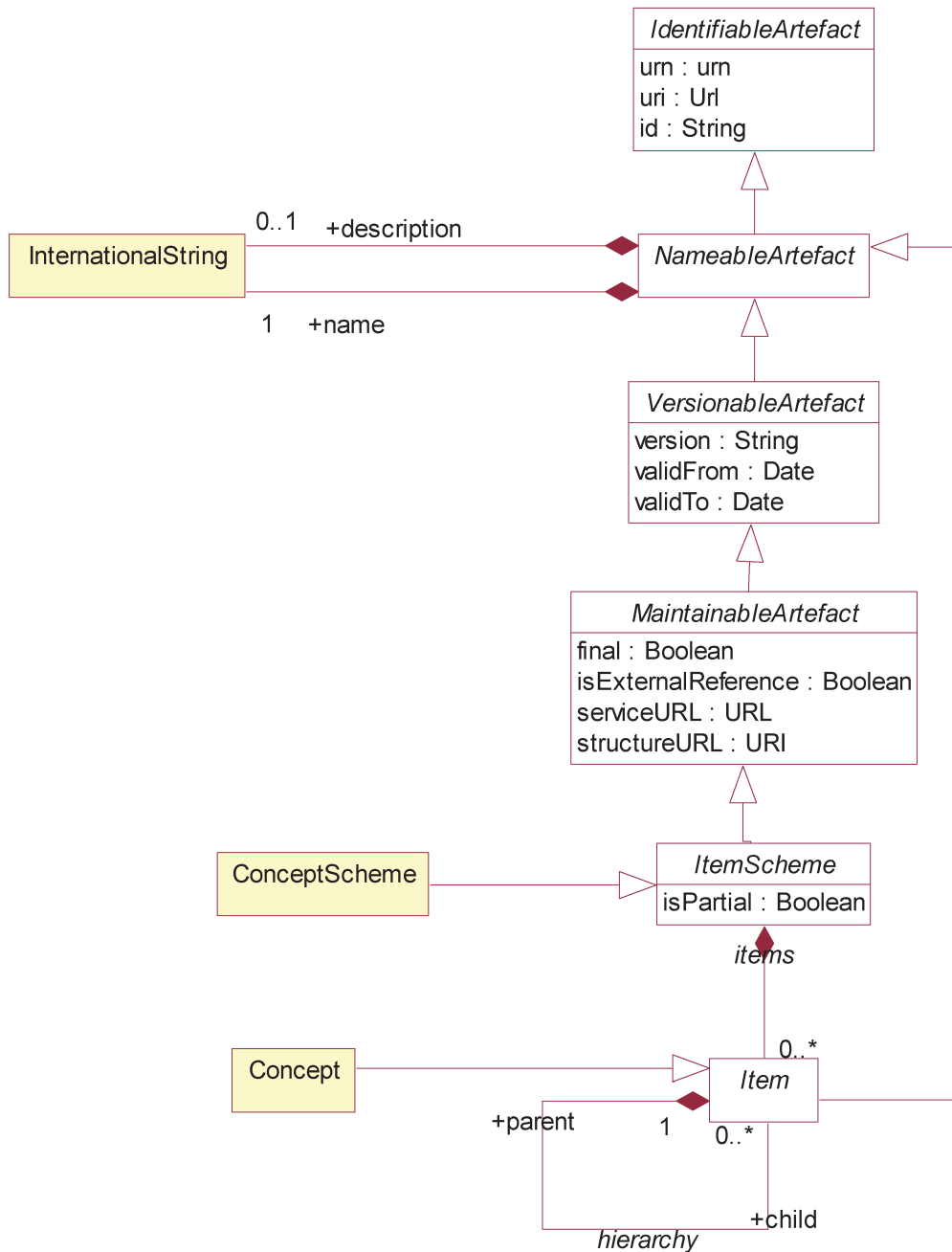


图 14 概念方案的继承关系类图

7.4.2 类图说明

ConceptScheme(概念方案)继承自 ItemScheme(项方案),因此具有以下属性:

- id
- uri
- urn
- version
- validFrom
- validTo
- isExternalReference

- registryURL
- structureURL
- repositoryURL
- final
- isPartial

Concept(概念)继承自 Item(项)并具有以下属性:

- id
- uri
- urn

通过来自于 *NameableArtefact*(可命名产物)的继承, *ConceptScheme*(概念方案)和 *Concept*(概念)都与 *InternationalString*(国际字符串)进行关联,以支持多语种名称、可选多语种说明,并与 *Annotation*(注释)进行关联以支持备注(图中未显示)。

通过来自 *ItemScheme*(项方案)的继承, *ConceptScheme*(概念方案)包含了一个或多个 *Concepts*(概念), *Concepts*(概念)本身可以在(继承的) *hierarchy*(层级)关联中拥有零个或多个子概念。

注: 在该关联中, 一个子 *Concept*(概念)只能拥有一个父 *Concept*(概念)。

一个部分 *ConceptScheme*(概念方案)(其中的 *isPartial* 属性值为“真”)与 *ConceptScheme*(概念方案)一样, 包含了 *Concept*(概念)以及相关名称和说明, 就像正常的 *ConceptScheme*(概念方案)一样。然而, 其内容只是完整 *ConceptScheme*(概念方案)的子集。该工作方式在 6.5.3.1 的“*ItemScheme*(项方案)”中进行了说明。

7.4.3 概念方案的关系类图

概念方案的关系类图如图 15 所示。

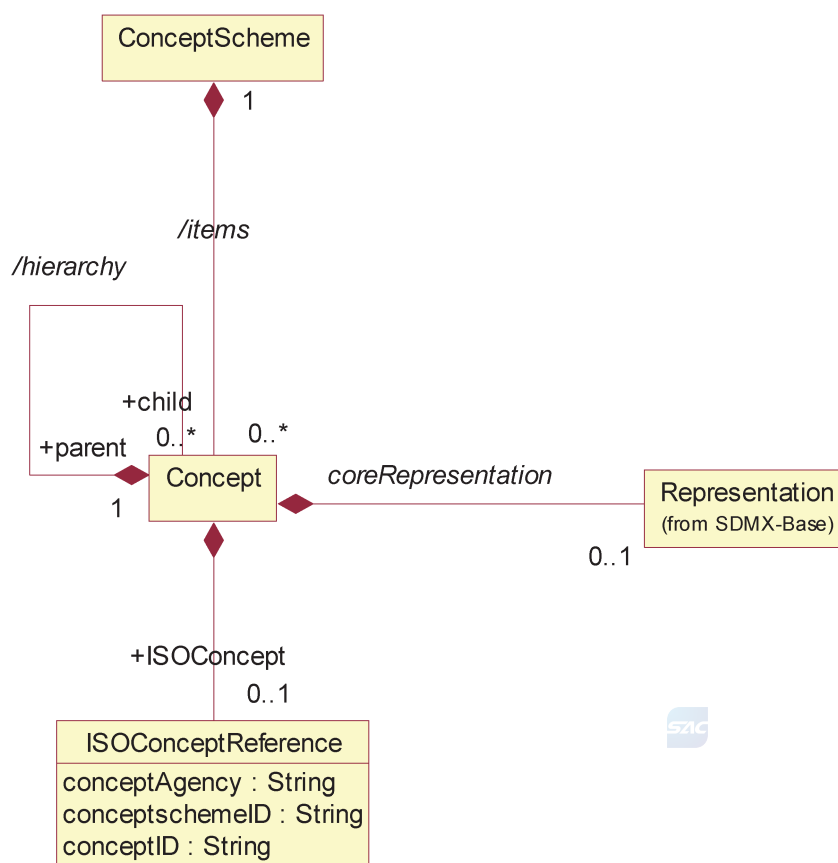


图 15 概念方案的关系类图

7.4.4 类图说明

7.4.4.1 描述

ConceptScheme(概念方案)可以拥有一个或多个 Concepts(概念集)。一个 Concept(概念)可以拥有零个或多个子 Concepts(概念集),这样就支持了 Concepts(概念集)的层级。

注:在该关联中,一个子 Concept(概念)只能拥有一个父 Concept(概念)。

层级的目的是将具有语义关系的概念联系起来。例如,Reporting_Country 概念和 Vis_a_Vis_Country 概念都以 Country(国家)概念作为父概念,而 CONTACT(联络人)概念则可以拥有 PRIMARY_CONTACT(主要联络人)作为子概念。这种方案的目的并不在于定义报告结构:这些报告结构在 MetadataStructureDefinition(元数据结构定义)中进行定义。

Concepts(概念)可与 coreRepresentation(核心表示法)关联起来。当 coreRepresentation(核心表示法)应用于诸如 DataStructureDefinition(数据结构定义)或 MetadataStructureDefinition(元数据结构定义)的结构时,coreRepresentation(核心表示法)是对 Concept(概念)的格式和值域的规定,除非 Representation(表示法)在相关结构定义中不予考虑。在层级 ConceptScheme(概念方案)中,除非子 Concept(概念)不考虑 Representation(表示法),否则 Representation(表示法)都从父 Concep(概念)处继承。

注: ConceptScheme(概念方案)在 DataStructureDefinition(见 8.3.2)中用作 MeasureDimension(测量维度)的 Representation(表示法)。ConceptScheme(概念方案)中的每个 Concept(概念)都是一个特定测量,每个 Concept(概念)都可以被赋予一个 coreRepresentation(核心表示法)。因此,在数据集中,对 MeasureDimension(测量维度)进行报告时,每个测量的观测值的有效格式将在 Concept(概念)中进行规定。这样每个测量可以采用不同的格式。更详细的描述见 8.3。

第 6 章对 Representation(表示法)进行了更详细的描述。

Concept(概念)与 GB/T 18391—2009(ISO/IEC 11179)中所描述的概念可能相关联。而 ISOConceptReference(ISO 概念引用)标识了该概念以及包含它的概念方案。

7.4.4.2 概念方案和概念等的属性说明

概念方案和概念等的属性如表 10 所示。

表 10 概念方案和概念等的属性表

类	特征/属性	说明
◆ ConceptScheme	继承自 <i>ItemScheme</i>	根据对象共有的特性,将概念进行分组的描述信息
◆ Concept	继承自 <i>Item</i>	概念是通过特征的独特组合所形成的知识单元
	◆ /hierarchy	将父概念与子概念关联起来
	◆ coreRepresentation	与 Representation 关联
	+ ISOConcept	关联到 ISO 概念引用
ISOConceptReference		ISO 概念定义的标识
	conceptAgency	该概念所在的概念方案的维护机构
	conceptSchemeID	概念方案的标识符
	conceptID	概念的标识符

7.5 分类方案

7.5.1 语境

本模型包定义了分类方案中能够支持分类的定义及其相互关系的结构。本模型包与概念方案的模型包类似。分类方案的一个例子是对数据进行分类的方案(有时候称为主题域方案或数据分类方案)。正如后文所描述的那样,分类方案中的每个节点可以关联到 Categorisation(加以分类)中的 IdentifiableArtefacts(可标识物)的集合。

7.5.2 分类方案的继承类图

分类方案的继承类图如图 16 所示。

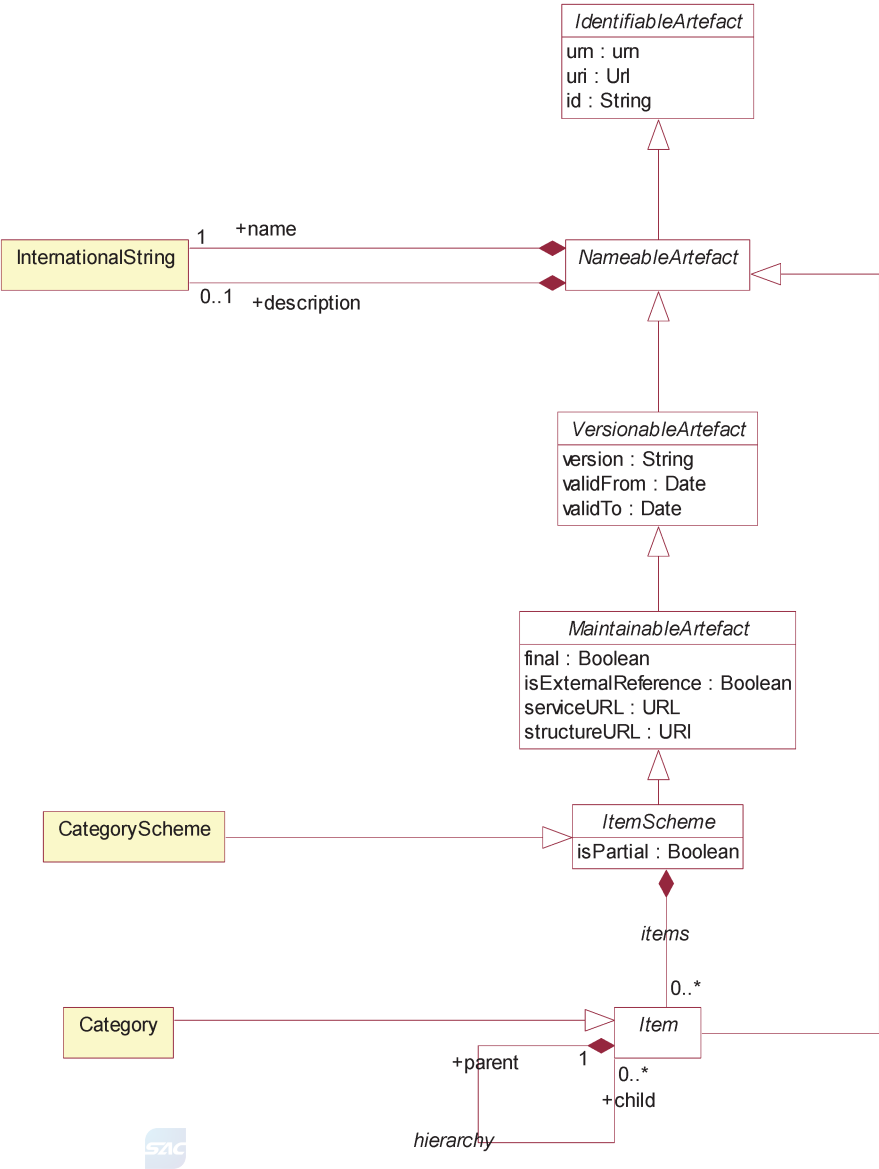


图 16 分类方案的继承类图

7.5.3 类图说明

分类被建模为层级的 ItemScheme(项方案)。CategoryScheme(分类方案)继承自 ItemScheme(项方

案)并具有以下属性:

- id
- uri
- urn
- version
- validFrom
- validTo
- isExternalReference
- structureURL
- serviceURL
- final
- isPartial

Category(分类)继承自 Item(项)并具有以下属性:

- id
- uri
- urn

CategoryScheme(分类方案)和 Category(分类)都与 InternationalString(国际字符串)相关联以支持多语种名称、可选的多语种说明,并且与 Annotation(注释)相关联以支持备注(本模型中未显示)。

通过继承,CategoryScheme(分类方案)包含了一个或多个 Category(分类),而 Category(分类)自身可以在(所继承的)hierarchy(层级)关联关系中拥有零个或多个子 Category(分类)。

注:在该关联中,一个子 Category(分类)只能有一个父 Category(分类)。

一个部分 CategoryScheme(分类方案)(其中的 isPartial 属性值为“真”)与 CategoryScheme(分类方案)一样,包含了 Category(分类)以及相关名称和说明,就像正常的 CategoryScheme(分类方案)一样。然而,其内容只是完整 CategoryScheme(分类方案)的子集。该工作方式在“ItemScheme(项方案)”中进行了说明。

7.5.4 分类方案的关系类图

7.5.4.1 类图

分类方案的关系类图如图 17 所示。

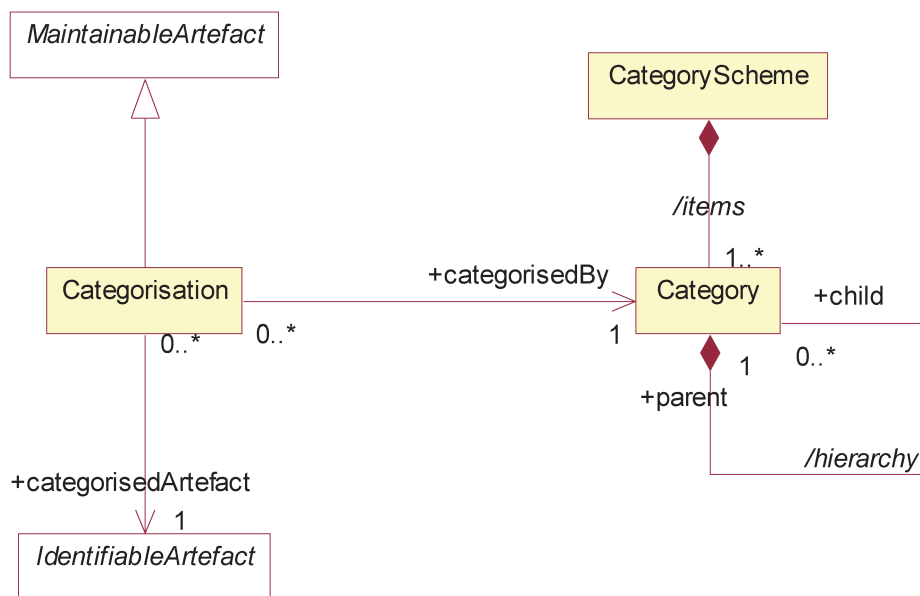


图 17 分类方案的关系类图

CategoryScheme(分类方案)可以拥有一个或多个 Categorys(分类集)。Category(分类)是可标识的并且拥有标识信息。Category(分类)可以拥有零个或多个子 Categorys(分类集),因此可以支持 Categorys(分类)层级。IdentifiableArtefact(可标识物)都可以由 Category(分类)进行 + categorisedBy(按...进行分类)。这可以通过 Categorisation(加以分类)方式达成。每个 Categorisation(加以分类)可以将一个 IdentifiableArtefact(可标识物)与一个 Category(分类)关联起来。可以采用多个 Categorisations(加以分类)来构建一组由同一 Category(分类)进行 + categorisedBy(按...进行分类)的 IdentifiableArtefact(可标识物)。

注：从 Category(分类)到 Categorisation(加以分类)不存在导航关系(也就是说没有嵌套引用)。从实现角度来看,这样做是有必要的,因为 Categorisation(加以分类)对 Category(分类)或 IdentifiableArtefact(可标识物)的版本控制都没有影响。

7.5.4.2 分类方案和分类等的属性说明

分类方案和分类等的属性如表 11 所示。

表 11 分类方案和分类等的属性表

类	特征/属性	说明
CategoryScheme	继承自 <i>itemScheme</i>	根据对象共有的特性,将概念进行分组的描述信息
	/items	与分类关联
Category	继承自 <i>Item</i>	分类(典型的是表格分类、章节、小节、部、分部、组、分组、类和子类)中任意层级的项
	/hierarchy	将父分类与子分类关联起来
Categorisation	继承自 MaintainableArtefact	将 IdentifiableArtefact(可标识物)与 Category(分类)关联起来
	示例 1: +categorisedArtefact	关联 IdentifiableArtefact(可标识物)
	示例 2: +categorisedBy	关联 Category(分类)

7.6 组织方案

7.6.1 类图

组织方案的类图如图 18 所示。

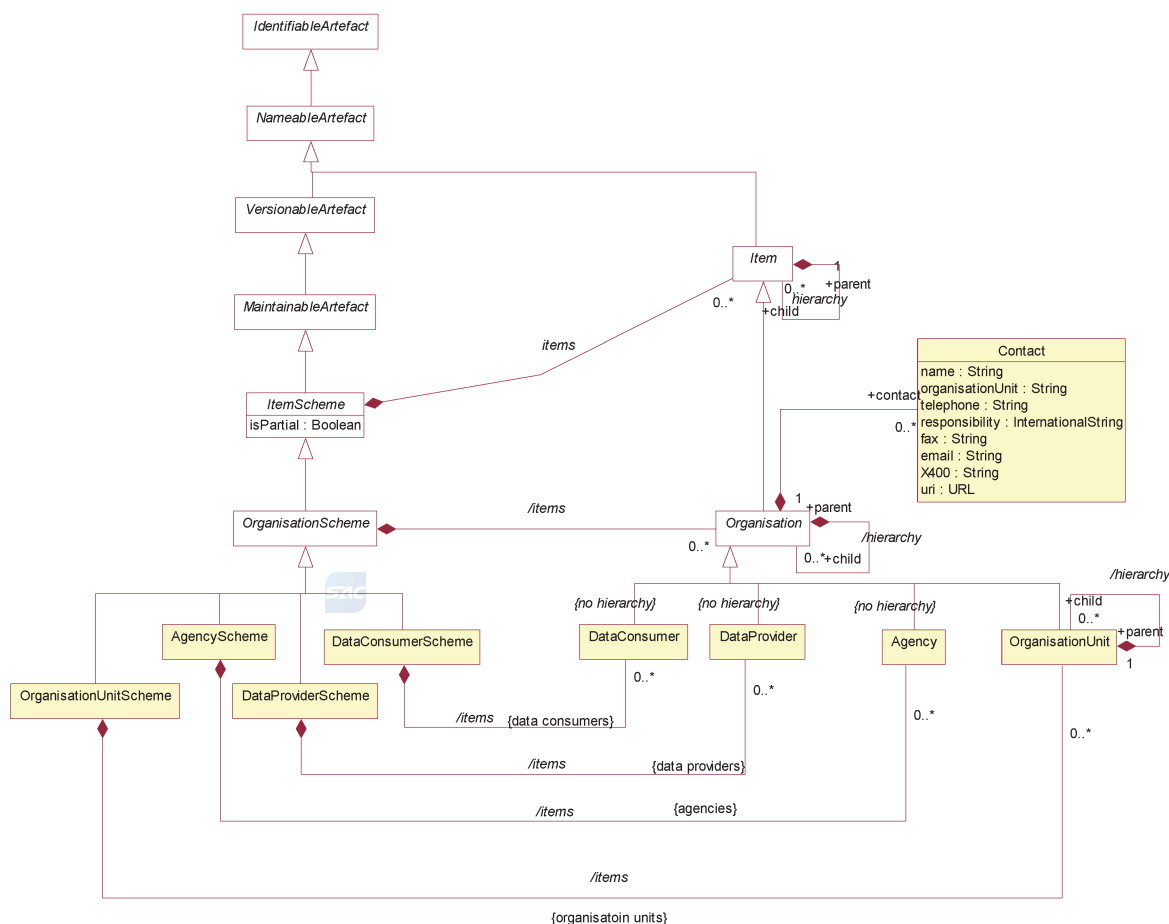


图 18 组织方案类图

7.6.2 类图说明

7.6.2.1 描述

OrganisationScheme(组织方案)是抽象类,它包含的 *Organisation*(组织)也是抽象类。*Organisation*(组织)可以具有子 *Organisation*(组织)。

OrganisationScheme(组织方案)可能是以下 4 种类型之一:

- a) AgencyScheme(机构方案)——包含了仅限于平行机构列表(也就是说,机构没有层级)中的机构。

注：[Maintenance(维护)]机构(Agency)的SDMX体系可以是层级机构，这将在本标准第7部分“技术说明”中详细说明。

- b) `DataProviderScheme`(数据提供者方案)——包含了仅限于平行机构列表(也就是说,机构没有层级)中的 `DataProvider`(数据提供者)。
- c) `DataConsumerScheme`(数据消费者方案)——包含了仅限于平行机构列表中(也就是说,机构没有层级)的 `DataConsumer`(数据使用者)。
- d) `OrganisationUnitScheme`(组织单元方案)——包含了从 `Organisation`(组织)继承/hierarchy(/层级)关联的 `OrganisationUnit`(组织单元)。

引用元数据通过元数据附加机制可以依附于 Organisation(组织)。该机制在本部分的“引用元数据”(见第 10 章)进行描述。这意味着,除了 Contact(联络人)信息外,该模型没有规定能够附加给 DataProvider(数据提供者)、DataConsumer(数据使用者)、OrganisationUnit(组织单元)或 Agency(机构)的具体引用元数据。

一个部分 OrganisationScheme(组织方案)(其中的 isPartial 属性值为“真”)与 OrganisationScheme(组织方案)一样,包含了 Organisation(组织)以及相关关联的名称和说明,就像正常的 OrganisationScheme(组织方案)一样,然而,其内容是完整 OrganisationScheme(组织方案)的子集。该工作方式在“ItemScheme(项方案)”中进行了说明。

7.6.2.2 组织方案和组织等的属性说明

组织方案和组织等的属性如表 12 所示。

表 12 组织方案和组织等的属性表

类	特征/属性	说明
OrganisationScheme	抽象类继承自 ItemScheme 子类是: AgencyScheme DataProviderScheme DataConsumerScheme OrganisationUnitScheme	Organisations(组织)的维护集合
	/items	关联到方案中的 Organisations(组织)
Organisation	继承自 Item 子类是: Agency DataProvider DataConsumer OrganisationUnit	组织是管理机构的独特框架,一个人或多个人在这个框架范围内为达到某些目标而实施行为,或被指派来实施行为
	+contact	关联到 Contact(联络)信息
	/hierarchy	关联到子 Organisations(组织)
Contact		个人或组织(或组织部分或组织人员)角色的实例,在规定的语境中,信息项、实体对象和/或人可以发送给他们,也可以从中获得
	name	Contact(联络)人的语言表达式
	organisationUnit	Contact(联络)人所工作的组织结构的语言表达式
	responsibility	组织角色中联络人的职能
	telephone	Contact(联络)电话号码
	fax	Contact(联络)的传真号码

表 12 (续)

类	特征/属性	说明
	email	Contact(联络)的因特网电子邮件地址
	X400	Contact(联络)的 X400(信息处理服务协议)地址
	uri	Contact(联络)的 URL 地址
AgencyScheme		Maintenace Agencies(维护机构)的集合
	/items	关联到方案中 Maintenace Agencies(维护机构)
DataProviderScheme		Data Providers(数据提供者)的集合
	/items	关联到方案中 Data Providers(数据提供者)
DataConsumerScheme		Data Consumers(数据使用者)的集合
	/items	关联到方案中 Data Consumers(数据使用者)
OrganisationUnitScheme		Organisation Units(组织单元)的集合
	/items	关联到方案中 Organisation Units(组织单元)
Agency	继承自 Organisation	负责维护产物(诸如统计分类)、术语表、结构元数据(诸如 DSD(数据结构定义)和 MSD(元数据结构定义))、Concepts(概念)和 Code lists(代码表)的机构
DataProvider	继承自 Organisation	生成数据或引用元数据的组织
DataConsumer	继承自 Organisation	将数据作为输入以进一步处理的组织
OrganisationUnit	继承自 Organisation	组织结构中的指派
	/hierarchy	关联到子 Organisation Units(组织单元)

7.7 报告分类

7.7.1 类图

报告分类的类图如图 19 所示。

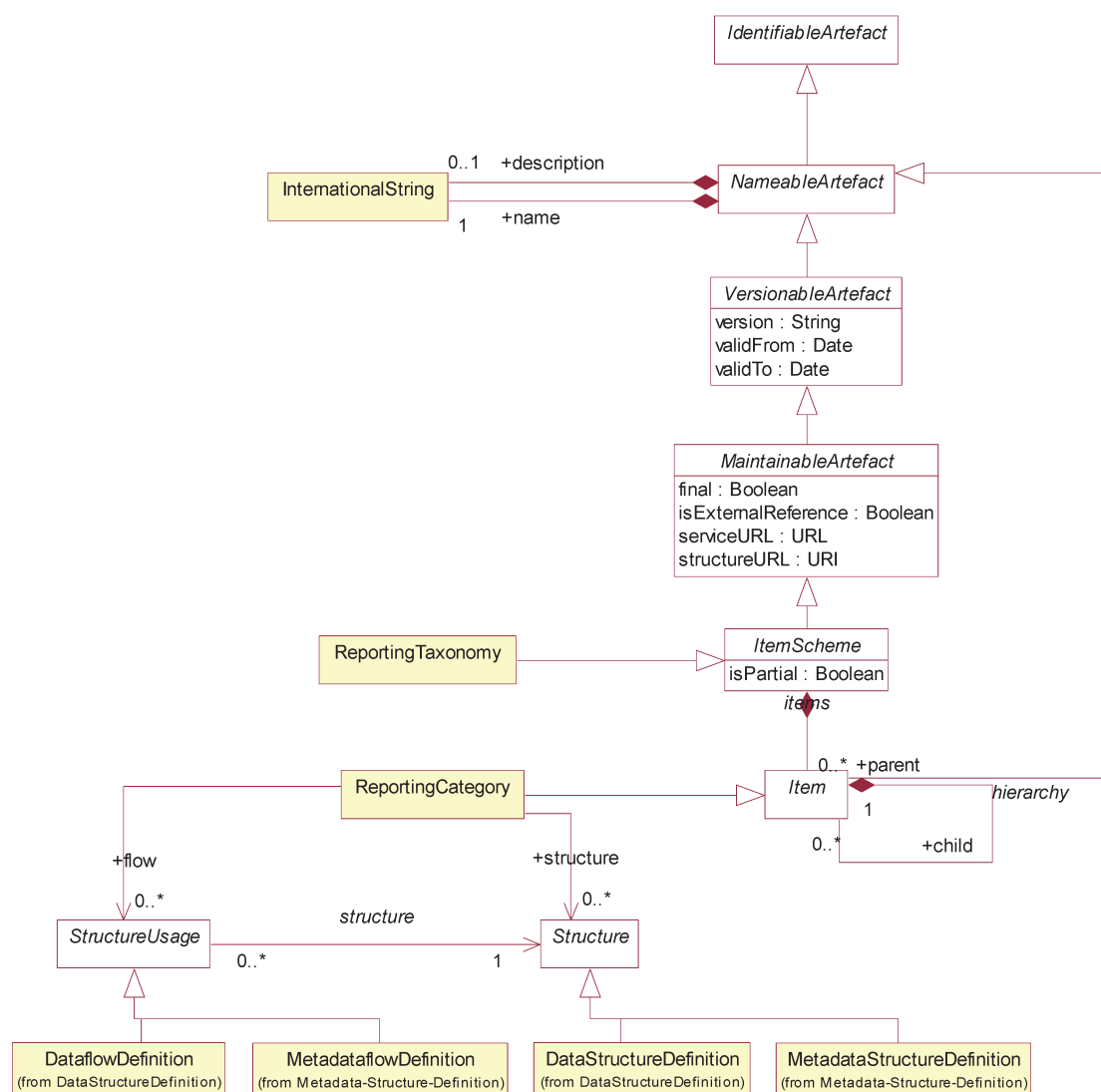


图 19 报告分类的类图

7.7.2 类图说明

7.7.2.1 描述

在数据报告,特别是一些主要报告中,可能包含多种不同类型的数据,而每种数据都由不同的 *Structure*(结构)进行描述。同样,一份特定发布或发布的报告也可能包含多种不同类型结构的数据。所关联的子报告集合的定义由 *ReportingTaxonomy*(报告分类法)支持。

ReportingTaxonomy(报告分类法)是 *ItemScheme*(项方案)的一种特殊化形式。*ReportingTaxonomy*(报告分类法)的每一种 *ReportingCategory*(报告分类)都可以链接至一个或多个 *StructureUsage*(结构用法),而 *StructureUsage*(结构用法)本身可以是一种 *DataflowDefinition*(数据流定义)或 *MetadataflowDefinition*(元数据流定义);同时可以连接至一种或多种 *Structure*(结构),而 *Structure*(结构)本身可以是一种 *DataStructureDefinition*(数据结构定义)或 *MetadataStructureDefinition*(元数据结构定义)。在一个特定的 *ReportingTaxonomy*(报告分类法)中,以这种方式链接的每个 *Category*(分类)都将链接至同一个类(例如,方案中的所有 *Category*(分类)将链接至 *DataflowDefinition*(数据

流定义))。

注：ReportingCategory(报告分类)可以拥有子 ReportingCategory(报告分类)，而通过这种方式就可能定义层级 ReportingTaxonomy(报告分类法)。

在该分类体系中，定义的某些 ReportingCategory(报告分类)可能仅给出了报告结构。例如：

第 1 部分：

- 1) 链接至 DataflowDefinition_1(数据流定义_1)；
- 2) 链接至 DataflowDefinition_2(数据流定义_2)。

第 2 部分：

- 1) 链接至 DataflowDefinition_3(数据流定义_3)；
- 2) 链接至 DataflowDefinition_4(数据流定义_4)。

在此处，第 1 部分和第 2 部分的节点将不会链接至 DataflowDefinition(数据流定义)，但其他部分的节点将会链接至 DataflowDefinition(数据流定义)[并因此链接至 DataStructureDefinition(数据结构定义)]。

一个部分 ReportingTaxonomy(报告分类法)(其中的 isPartial 属性值为“真”)与 ReportingTaxonomy(报告分类法)一样，包含了 ReportingCategory(报告分类)以及相关名称和说明，就像正常的 ReportingTaxonomy(报告分类法)一样。然而，其内容是完整 ReportingTaxonomy(报告分类法)的子集。该工作方式在“ItemScheme(项方案)”中进行说明。

7.7.2.2 报告分类法和报告分类等的属性说明

报告分类法和报告分类等的属性如表 13 所示。

表 13 报告分类法和报告分类等的属性表

类	特征/属性	说明
ReportingTaxonomy	继承自 ItemScheme	定义了数据报告的复合结构的方案，其中的每个组件都可以用独立的 Dataflow Definition(数据流定义)或 Metadataflow Definition(元数据流定义)来进行描述
	items	关联 ReportingCategory(报告分类)
ReportingCategory	继承自 Item	将结构赋予报告并链接至数据和元数据的组件
	hierarchy	关联子 Reporting Category(报告分类)
	+ flow	关联到数据流和元数据流，而数据流和元数据流链接到元数据以及定义了它们的结构上，而这些元数据是有关供应和相关数据和元数据集的
	+ structure	关联到 DSD(数据结构定义)和 MSD(元数据结构定义)，DSD 和 MSD 定义了描述数据和元数据的结构元数据，而所描述的数据和元数据包含在该报告中

8 数据结构定义和数据集

8.1 简介

DataStructureDefiniton(数据结构定义)是用于数据的结构定义类名。某些组织将该类型的定义称为“Key Family(键族)”,因此这两个名称是同义的。本部分采用数据结构定义(也称为 DSD)。

模型中该层的许多结构继承自 SDMX 基础层。因此,有必要研究继承图和关系图,以便理解各个包的功能。在简单的子模型中,继承图和关系图在同一图中显示,但是为了清晰起见,在较为复杂的子模型中则对它们进行省略。在这种情况下,图 20 继承图显示了与数据结构定义相关的类的完整继承树形图。

除在图 20 继承图所示的子模型外,几乎没有其他类。换句话说,根据关联和属性,SDMX 基础层给出了该子模型的绝大多数结构。本部分所示的关系图清晰地表明了这些关联是何时从 SDMX 基础层处继承而来。

继承自 SDMX 基础层结构的类应具有以下功能:

- 注释: *AnnotableArtefact* (可注释产物);
- 标识: *IdentifiableArtefact* (可标识物);
- 命名: *NameableArtefact* (可命名产物);
- 版本控制: *VersionableArtefact* (版本可控产物);
- 维护: *MaintainableArtefact* (可维护产物)。

8.2 继承视图

8.2.1 类图

数据结构定义和数据集包中的类继承类图如图 20 所示。

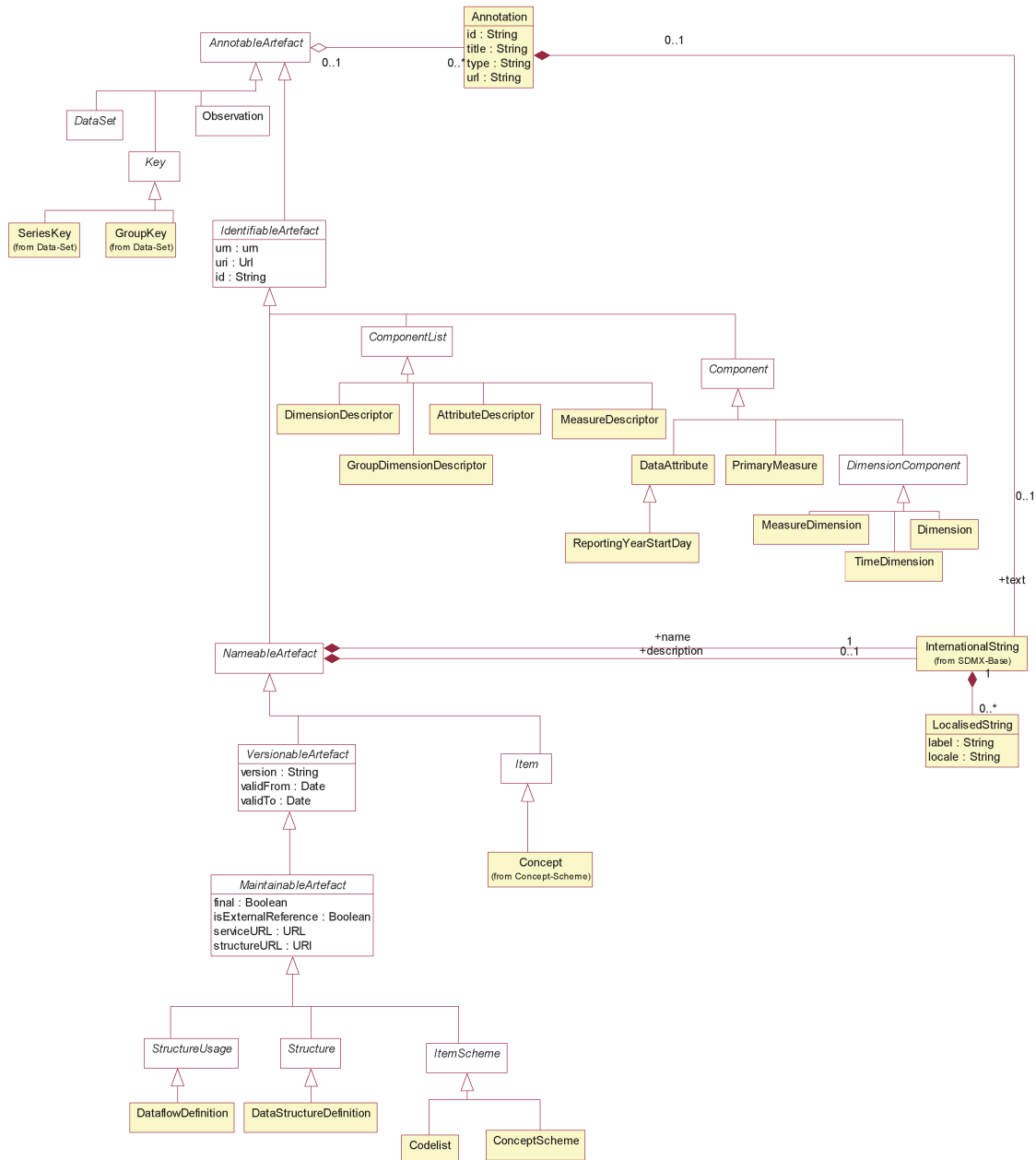


图 20 数据结构定义和数据集包中的类继承

8.2.2 类图说明

SDMX 元模型中要求注释的类从 AnnotableArtefact(可注释产物)继承而来。这些类为:

——IdentifiableArtefact(可标识物);

——DataSet(数据集)(包括: StructureSpecificDataSet(特定结构数据集)、GenericDataSet(通用数据集)、GenericTimeSeriesDataSet(通用时间序列数据集)、StructureSpecificTimeSeriesDataSet(特定结构时间序列数据集);

——Key(键)[包括: SeriesKey(序列键)和 GroupKey(组键)]。

SDMX 元模型中要求注释和全局标识的类衍生自 IdentifiableArtefact(可标识物)类。这些

类为：

- NameableArtefact* (可命名产物)；
- ComponentList* (组件列表)；
- Component* (组件)。

SDMX 元模型中要求注释、全局标识、多语种名称和多语种描述类衍生自 *NameableArtefact* (可命名产物) 类。这些类为：

- VersionableArtefact* (版本可控产物)；
- Item* (项)。

SDMX 元模型中要求注释、全局标识、多语种名称和多语种描述以及版本控制的类衍生自 *VersionableArtefact* (版本可控产物) 类。这些类都为：

- MaintainableArtefact* (可维护产物)。

维护机构所维护的信息的抽象类全部继承自 *MaintainableArtefact* (可维护产物)，这些抽象类也继承了 *VersionableArtefact* (版本可控产物) 的所有特征，这些类为：

- StructureUsage* (结构用法)；
- Structure* (结构)；
- ItemScheme* (项方案)。

上述所有类均为抽象类。理解本章的类图关键在于继承自这些抽象类的具体类。

Agencies (机构) 所维护元模型的 SDMX DSD (数据结构定义) 和 Dataset (数据集) 包中的具体类全部继承 (通过其他抽象类) 自 *MaintainableArtefact* (可维护产物)，这些类为：

- DataflowDefinition* (数据流定义)；
- DataStructureDefinition* (数据结构定义)。

作为列表的组件结构直接从 *Structure* (结构) 类继承。*Structure* (结构) 包含了多个组件列表。继承自 *Structure* (结构) 的具体类为：

- DataStructureDefinition* (数据结构定义)。

DataStructureDefinition (数据结构定义) 包含维度列表、测量列表和属性列表。

继承自 *ComponentList* (组件列表) 并且是 *DataStructureDefinition* (数据结构定义) 子组件的具体类为：

- DimensionDescriptor* (维度描述符)：内容为 *Dimension* (维度)、*MeasureDimension* (测量维度) 和 *Time Dimension* (时间维度)；
- DimensionGroupDescriptor* (维度组描述符)：内容是与 *Dimension* (维度)、*MeasureDimension* (测量维度)、*TimeDimension* (时间维度) 的关联；
- MeasureDescriptor* (测量描述符)：内容是 *PrimaryMeasure* (主要测量)；
- AttributeDescriptor* (属性描述符)：内容为 *DataAttribute* (数据属性)。

继承自 *Component* (组件) 的类为：

- PrimaryMeasure* (主要测量)；
- DimensionComponent* (维度组件) 以及 *Dimension* (维度)、*MeasureDimension* (测量维度) 和 *TimeDimension* (时间维度) 的子类；
- DataAttribute* (数据属性)。

继承自 *DataAttribute* (数据属性) 的类为：

- ReportingYearStartDay* (报告年度起始日)。

以上所列的具体类是定义 *DataStructureDefinition* (数据结构定义) 的元模型所要求的大部分类。本章的其他类图和说明显示了这些具体类是如何关联的，以便支持所要求的功能。

8.3 数据结构定义的关系视图

8.3.1 类图

不含表示法的数据结构定义的关系类图如图 21 所示。

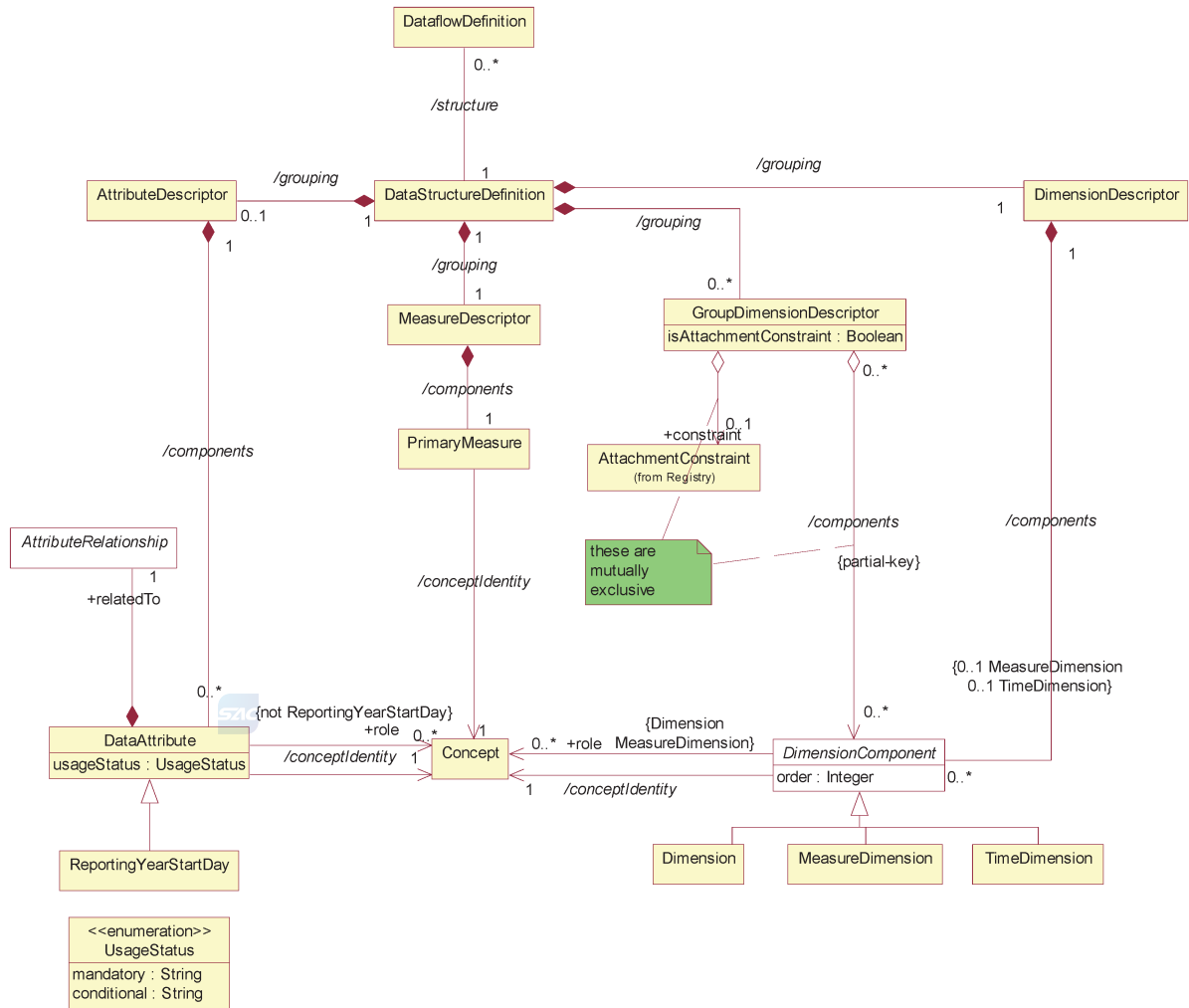


图 21 不含表示法的数据结构定义的关系类图

8.3.2 类图说明

8.3.2.1 描述

DataStructureDefinition(数据结构定义)定义了 Dimensions(维度集)、MeasureDimension(测量维度)、TimeDimension(时间维度)、DataAttributes(数据属性)和 PrimaryMeasure(主要测量)以及相关的 Representation(表示法),而它们包含 DataSet(数据集)中的数据的有效结构和相关属性,而 DataSet(数据集)则由 DataflowDefinition(数据流定义)进行定义。

DataflowDefinition(数据流定义)也可以拥有附加的其他元数据,该元数据定义了定性信息以及使用 DSD(数据结构定义)的有关 Constraints(约束),诸如在 Dimension(维度)中所使用的 CodeS(代码)子集(这部分内容见第 13 章,第 14 章)。每个 DataflowDefinition(数据流定义)最多拥有一个 DSD(数据结构定义),这个 DSD(数据结构定义)定义了将被报告/发布的 DataSets(数据集)的结构。

这里存在三种类型的维度,每种维度都与 Concept(概念)有着共同的关联:

- Dimension(维度);
- MeasureDimension(测量维度);
- TimeDimension(时间维度)。

注: 此处, *DimensionComponent* (维度组件) 可以是它的子类或全部子类, 也就是 *Dimension* (维度)、*MeasureDimension* (测量维度)、*TimeDimension* (时间维度), 而术语 *DataAttribute* (数据属性) 既指 *DataAttribute* (数据属性), 也指其子类 *ReportingYearStartDate* (报告年度起始日期)。

DimensionComponent (维度组件)、*DataAttribute* (数据属性) 和 *PrimaryMeasure* (主要测量) 链接至定义了其名称和语义的 *Concept* (概念) (/conceptIdentity 关联到 *Concept* (概念))。 *DataAttribute* (数据属性)、*Dimension* (维度) 和 *MeasureDimension* (测量维度) (而不是 *TimeDimension* (时间维度)) 可以选择拥有与 *Concept* (概念) 相关联的 +conceptRole (概念角色), 而概念标识出其在 DSD (数据结构定义) 中的角色。因此, *Concept* (概念) 所允许的角色在 *ConceptScheme* (概念方案) 中进行维护。角色的实例有: 地理、实体、计数、测量单位。这些角色的作用是使应用程序能够采用有意义的方式来处理数据 (例如, 将维度值关联到映射矢量)。团体 (例如官方统计团体) 将协调角色与角色所在的团体之间的关系, 以便数据以有意义的方式在团体中进行交换和共享。

如果 *DimensionComponent* (维度组件)、*PrimaryMeasure* (主要测量) 或 *DataAttribute* (数据属性) 的有效值在 *DataSetDefinition* (数据结构定义) 中使用, 则它们由 *Representation* (表示法) 进行定义。 *Representation* (表示法) 在 *DataSetDefinition* (数据结构定义) 的 *localRepresentation* (本地表示法) 中没有体现时, 则 *Representation* (表示法) 通过 *coreRepresentation* (核心表示法) 关系取自于 *Concept* (概念) 定义, 见图 21。

注 1: 对于 *MeasureDimension* (测量维度) 来说, *Representation* (表示法) 应是 *ConceptScheme* (概念方案), 同时, *Representation* (表示法) 应从 *MeasureDimension* (测量维度) 引用而来, 且不能默认为 /conceptIdentity (概念标识) 所关联的 *Concept* (概念) 的 *Representation* (表示法)。

注 2: *TimeDimension* (时间维度) 和 *ReportingYearStartDate* (报告年度起始日期) 均约束到具体的 *FacetValueTypes* (刻面值类型)。

总是存在标识出包含了完整键的所有 *Dimension* (维度) 的 *DimensionDescriptor* (维度描述符) 组。同时, *Dimensions* (维度集) 还规定了 *Observation* (观测) 的键。

DimensionComponent (维度组件) 可以有选择性地由多个 *GroupDimensionDescriptor* (组维度描述符) 进行分组, 而每个描述符都标识出了构成部分键的 *Dimensions* (维度) 组。 *GroupDimensionDescriptors* (组维度描述符) 应标识为 *GroupDimensionDescriptor.id*, 并且该描述符用在 *DataSet* (数据集) 的 *GroupKey* (组键) 中, 以声明在该数据集的该组层级上报告哪个 *DataAttributes* (数据属性)。

在 *DimensionDescriptor* (维度描述符) 中最多可以规定一个 *MeasureDimension* (测量维度)。 *MeasureDimension* (测量维度) 的用途在于规定测量的含义 (因为 *PrimaryMeasure* (主要测量) 通常拥有一个通用含义, 如观测值), 还可以在 *StructureSpecificDataSet* (结构特定数据集) 中定义和报告多个测量。 *MeasureDimension* (测量维度) 引用 *ConceptScheme* (概念方案) 作为其 *Representation* (表示法) (参见下文), 然而, *Dimension* (维度) 可以有枚举型 [*Codelist* (代码表)] 表示法或非枚举型 [*Facet* (刻面)] 表示法。对于 *MeasureDimension* (测量维度) 来说, *ConceptScheme* (概念方案) 中的 [*Concept* (概念)] 构成了允许测量值的列表。这使得每一个测量 [*Concept* (概念)] 的表示法可以作为 *Concept* (概念) 的 *coreRepresentation* (核心表示法), 从而不考虑 *MeasureDimension* *Concept* (测量维度概念) 观测值的 *PrimaryMeasure* (主要测量) 所规定的 *Representation* (表示法)。

DimensionDescriptor (维度描述符) 中最多可以规定一个 *TimeDimension* (时间维度)。 *TimeDimension* (时间维度) 规定了数据集中表达观测时间周期的 *Concept* (概念)。 *TimeDimension* (时间维度) 应包含有效的时间表示法并且不应对其进行编码。

PrimaryMeasure(主要测量)是可观测的,尽管只存在一个 PrimaryMeasure(主要测量),但为了与 ComponentList(组件列表)/Component(组件)模式保持一致,PrimaryMeasure(主要测量)由 MeasureDescriptor(测量描述符)进行分组。

DataAttribute(数据属性)定义了所采集或所发布的数据的特性,并且根据单个 AttributeDescriptor(属性描述符)在 DSD(数据结构定义)中进行分组。DataAttribute(数据属性)可以规定为必选型或条件型,正如 usageStatus(使用状态)中所定义的那样。DataAttribute(数据属性)可在结构中扮演一种特定的角色,这是由关联到 Concept(概念)的 +role(角色)来规定,而该概念标识了它的角色。

DataAttribute(数据属性)规定为 +relatedTo(关联到)某个 AttributeRelationship(属性关系),而 AttributeRelationship(属性关系)定义了 DataSet(数据集)中所出现的 DataAttribute(数据属性)的结构。DataAttribute(数据属性)可以规定与以下产物之一相关联,如图 22 所示:

- 数据集[NoSpecifiedRelationship(未规定的关联关系)];
- 维度或维度集[DimensionRelationship(维度关系)];
- 由 GroupKey(组键)所规定的维度集[GroupRelationship(组关系):保留此术语的目的是为了保持兼容性,或 DimensionRelationship(维度关系)的 +groupKey(组键)];——观测[PrimaryMeasureRelationship(主要测量关系)]。

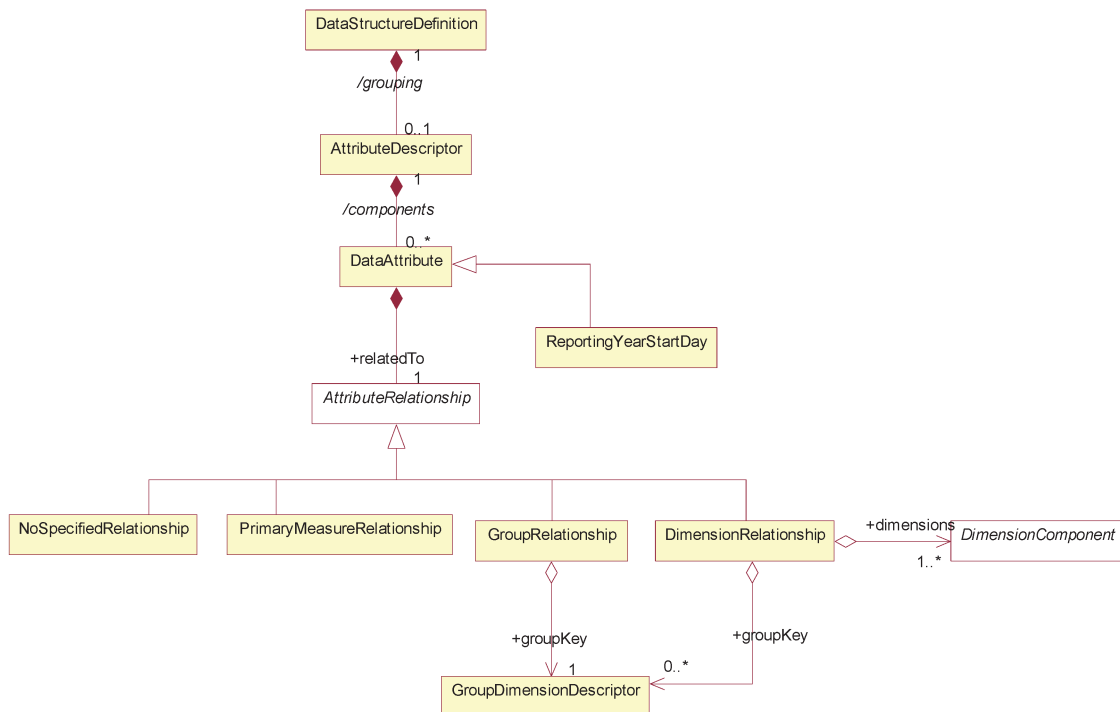


图 22 数据结构定义中的属性

表 14 详细列出了 DataAttribute(数据属性)规定的所有可能关系。

注：这些关系之间是相互排斥的,因此只能规定以下关系中的一项。

表 14 DataAttribute(数据属性)的所有可能关系

关系	含义	数据集中报告属性的位置
无	属性值不会随其他组件值的变化而改变	该属性在 Dataset Attribute(数据集属性)层级进行报告
维度 (1..n)	属性值将随着引用维度的值而变化。在这种情况下,可以有选择性地规定属性应当依附的组	属性在其所关联的维度的最低层级上进行报告,否则,如果规定了 Attachment Groups(附件组),则在该 Attachment Groups(附件组)层级上进行报告
组	属性值随着组所包含的全部维度值的组合而变化。增加属性是为了便于列出所有维度和附件组,但是,只有在属性值基于所有组维度值进行变化时才进行使用	属性在 Group(组)层级上进行报告
Primary Measure(主要测量)	属性值随着观测值而变化	属性在 Observation(观测)层级进行报告





Dimension(维度)、MeasureDimension(测量维度)、TimeDimension(时间维度)、PrimaryMeasure(主要测量)和 DataAttribute(数据属性)都可具有已规定的 Representation(表示法)[采用 localRepresentation(本地表示法)关联]。如果 Representation(表示法)未在 DataStructureDefinition(数据结构定义)中进行规定,则采用对 Concept(概念)coreRepresentation(核心表示法)进行规定的表示法。对于 MeasureDimension(测量维度)来说,每个测量的表示法在 MeasureDimension(测量维度)所引用的 ConceptScheme(概念方案)中为 Concept(概念)作出了规定。DSD 组件的表示法如图 23 所示。

DataStructureDefinition(数据结构定义)可以进行扩展以形成衍生的 DataStructureDefinition(数据结构定义)。这种扩展得到了 StructureMap(结构映射)的支持。

8.3.2.2 结构用法和数据流定义等属性说明

结构用法和数据流定义等属性如表 15 所示。

表 15 结构用法、数据流定义等属性表

类	特征/属性	说明
StructureUsage		见“SDMX Base”
DataflowDefinition	继承自 <i>StructureUsage</i>	供应者在不同的引用期限而提供的 数据流的抽象概念(也就是没有任何 数据的结构)
	<i>/structure</i>	将 Dataflow Definition(数据流定义) 关联到 DSD(数据结构定义)
DataStructureDefinition		用于采集或发布数据的元数据概念、 结构以及用法的集合
	<i>/grouping</i>	关联到一组元数据概念,这些元数据 概念在 DSD(数据结构定义)中具有 清晰的结构化角色
Group DimensionDescriptor	继承自 <i>ComponentList</i>	一组元数据概念,这些元数据概念定 义了部分键,而部分键衍生自 DSD (数据结构定义)中的 DimensionDe- scriptor(维度描述符)
	+constraint	标识了 Attachment Constraint(附件 约束),这个约束规定了属性可以依 附的维度、测量或属性值的子集
	<i>/components</i>	关联到构成组的 Dimension(维度)以 及 Measure Dimension(测量维度) 组件
DimensionDescriptor	继承自 <i>ComponentList</i>	元数据概念的有序集合,这些概念组 合起来对统计序列进行进行分级,而 且它们的值在诸如数据集这样的实 例中(与键)组合时,对特定的观测进 行了唯一标识

表 15 (续)

类	特征/属性	说明
	/components	关联到构成 Key Descriptor(键描述符)的 Dimension(维度)、Measure Dimension(测量维度)和 Time Dimension(时间维度)
AttributeDescriptor	继承自 ComponentList	定义了 DSD(数据结构定义)属性的一组元数据概念
	/components	关联到 Data Attribute(数据属性)组件
MeasureDescriptor	继承自 ComponentList	定义了 DSD(数据结构定义)测量的元数据概念
	/components	关联到测量组件
Dimension	继承自 Component	对统计序列进行分级的元数据概念(很有可能与其他元数据概念一同使用),例如,指出了某一经济活动或某一地理参照区域的统计概念
	/role	关联到规定了 Dimension(维度)在 DSD(数据结构定义)中所扮演的角色的 Concept(概念)
	/conceptIdentity	关联到定义了 Dimension(维度)语义的元数据概念
MeasureDimension	继承自 Dimension	标识了键结构中具有枚举型测量列表的组件的统计概念。此维度如同其表示法那样,具有枚举型测量概念的 Concept Scheme(概念方案)
TimeDimension	继承自 Dimension	标识了键结构中拥有“时间”角色的组件的元数据概念
DataAttribute	继承自 Component 子类 ReportingYear StartDay	对象或实体的特性
	/role	关联到规定了 Data Attribute(数据属性)在 DSD(数据结构定义)中所扮演的角色的 Concept(概念)
	usageStatus	定义了由数据类型 UsageStatus(使用状态)所约束的使用状态

表 15 (续)

类	特征/属性	说明
	+relatedTo	关联到 Attribute Relationship (属性关系)
	/conceptIdentity	关联到定义了组件语义的 Concept (概念)
ReportingYearStartDay 	继承自 DataAttribute	具体化的数据属性,该属性值与 Time Dimension(时间维度)中预先定义的报告期限一起使用。如果该属性值不存在,则 Time Dimension(时间维度)的所有报告期限值默认以报告年度的 1 月 1 日为起始日期
PrimaryMeasure	继承自 Component	元数据概念是数据集中可以进行测量的现象。在数据集中,测量的实例通常称为观测
	/conceptIdentity	关联到测量值的 Concept(概念)
AttributeRelationship	抽象类 子类 NoSpecified Relationship PrimaryMeasure Relationship GroupRelationship Dimension Relationship	规定了 DataSet(数据集)中 DataAttribute(数据属性)所依附的产物类型
NoSpecifiedRelationship		与任何具体构造无关的 DataAttribute(数据属性)
PrimaryMeasure Relationship		与 Primary Measure(主要测量)构造相关的 Data Attribute(数据属性)
GroupRelationship		与 Group Dimension Dimension Descriptor(组维度描述符)构造相关的 Data Attribute(数据属性)
	+groupKey	关联到 Group Dimension Dimension Descriptor(组维度描述符)
DimensionRelationship		与一组 Dimension(维度)相关的 Data Attribute(数据属性)
	+dimensions	关联到 Data Attribute(数据属性)所相关的 Dimension(维度)集
	+groupKey	关联到规定了 Data Attribute(数据属性)所依附的 Dimension(维度)集的 Group Dimension Dimension Descriptor(组维度描述符)

构成 Representation(表示法)的类、属性以及关联的解释在第 6 章中进行了描述

8.4 数据集的关系视图

8.4.1 语境

数据集包含了数据值与相关元数据的集合,它们都按照确定的 DSD(数据结构定义)进行采集或发布。

8.4.2 类图

数据集类图如图 24 所示。

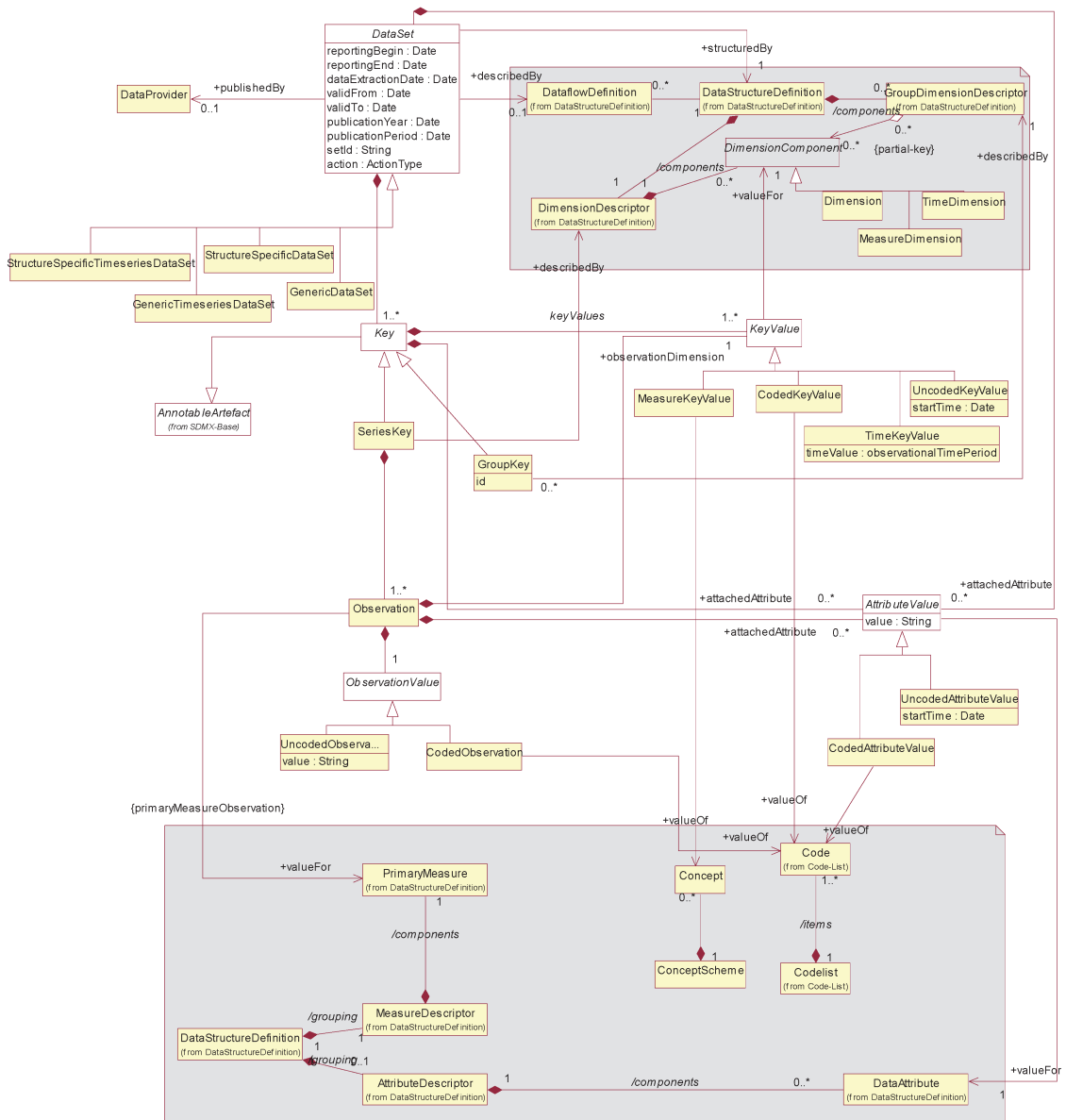


图 24 数据集类图

8.4.3 类图说明

8.4.3.1 描述

DataSet(数据集)是“数据的实例”,然而,*DataSet*(数据集)应符合与 *DataflowDefinition*(数据流定义)相关联的 *DSD*(数据结构定义)。虽然该模型显示了与 *DSD*(数据结构定义)类之间的关联,但与 *DSD*(数据结构定义)的链接仅具有概念意义。在数据集的实际交换中,应具有一个引用,该引用指向 *DSD*(数据结构定义),并可选择地指向 *DataflowDefinition*(数据流定义),但是 *DSD*(数据结构定义)并不一定应与数据进行交换。因此,*DSD*(数据结构定义)类在灰色区域进行显示,因为在 *DataSet*(数据集)交换时,它们并不是 *DataSet*(数据集)的一部分。然而,*DSD*(数据结构定义)中的结构元数据可以被应用程序根据 *DSD*(数据结构定义)中的 *Representation*(表示法)所定义的 *KeyValue*(键值)的有效内容来验证 *DataSet*(数据集)的内容。

DataProvider(数据提供者)组织可以负责一个或多个 *DataSet*(数据集)。

DataSet(数据集)既可以格式化为一种通用数据集(*GenericDataSet*、*GenericTimeSeriesDataSet*),也可格式化为某一种 *DSD* 特定的数据集(*StructureSpecificDataSet*、*StructureSpecificTimeSeriesDataSet*)。无论 *DataSet*(数据集)表达的是哪个 *DSD*(数据结构定义),通用数据集都以相同的方式进行构建。结构化的数据集按照一个特定的 *DSD*(数据结构定义)进行构建。依赖于所选定的实施语法,结构化的数据集应在语法层级更好地进行验证。

DataSet(数据集)是一组 *Observations*(观测)的集合,这些观测具有相同的维度,这些维度由 *DSD*(数据结构定义)的 *DimensionDescriptor*(维度描述符)中所定义的一组独特的组件(*Dimension*(维度)、*MeasureDimension*(测量维度)、*TimeDimension*(时间维度))以及相关联的 *AttributeValues*(属性值)来规定,而这些属性值定义了它们所依附的产物[即 *DataSet*(数据集)、*Observations*(观测)、*Dimension*(维度)集]的具体特性。数据集依据 *Observations*(观测)向其所报告的 *SeriesKey*(序列键)来进行结构化。

Observations(观测)可以是变量值,这些变量值是用于测量 *Concept*(概念)的,而 *Concept*(概念)与 *DSD*(数据结构定义)的 *MeasureDescriptor*(测量描述符)中的 *PrimaryMeasure*(主要测量)相关联。这种情况在没有规定每个 *Observation*(观测)的准确含义的 *MeasureDimension*(测量维度)的情况下是成立的。每个 *Observation*(观测)都将一个 *ObservationValue*(观测值)与一个 *KeyValue*(键值)(+*observationDimension*)(观测维度)相关联,此键值是“观测层级维度”所用的值。任何维度都可规定为“观测层级维度”,并且在 *DataSet*(数据集)层级上进行规定(也就是,整个数据集的维度都是相同的)。

如果“观测层级维度”属于 *MeasureDimension*(测量维度),则有可能(而不是必选地)用一个或多个 *Concept*(概念)的明确标识来报告 *Observation*(观测),而这些概念包含在 *MeasureDimension*(测量维度)按照它的 *Representation*(表示法)所引用的 *ConceptScheme*(概念方案)中。换句话说,实际 *ConceptS*(概念)在观测中得到了清晰的阐述。

如果明确要求规定 *DataSet*(数据集)为时间序列,则使用某个 *GenericTimeSeriesDataSet*(通用时序序列数据集)或 *StructureSpecificTimeSeriesDataSet*(结构特有时序序列数据集),并且 +*observationDimension*(观察维度)的 *KeyValue*(键值)应为一个 *TimeKeyValue*(时序键值)。在 *GenericDataSet*

(通用数据集)和 StructureSpecificDataSet(结构特定数据集)中,允许存在任意维度作为 +observationDimension(观测维度),包括 TimeDimension(时间维度)。

Keyvalue(键值)是 DSD(数据结构定义)中所规定的某个 MeasureDimension(测量维度)、TimeDimension(时间维度)或 Dimension(维度)的值。如果是 Dimension(维度),则可以对其编码(CodedKey-Value(代码键值)或不编码(UncodedKey-Value(无代码键值))。如果是 MeasureDimension(测量维度),则键值为 MeasureKey-Value(测量键值)。如果是 TimeDimension(时间维度),则键值为 TimeKey-Value(时间键值)。CodedDimensionValue(编码维度值)可取的实际值应是根据 DSD(数据结构定义)中 Dimension(维度)的 Representation(表示法)所规定的 Codelist(代码表)中的某个 Codes(代码)。MeasureDimensionValue(测量维度值)可取的实际值应是 MeasureDimensionValue(测量维度值)所要关联(+valueFor)的 ConceptScheme(概念方案)中的 Concept(概念)所规定的有效表示法。

ObservationValue(观测值)可以被编码——这就是 CodedObservation(编码观测);也可以不编码,也就是 UncodedObservation(未编码观测)。

GroupKey(组键)是拥有与 SeriesKey(序列键)相同维度的 Key(键)的子单元,但它定义了 SeriesKey(序列键)的 KeyValues(键值集)的一个子集。其子维度结构在与 GroupKey(组键)相同的 id 所标识的 DSD(数据结构定义)的 GroupDimensionDescriptor(组维度描述符)中进行定义。该 id 标识了一种“类型”的组,GroupKey(组键)的用途在于报告包含在该组层级上的一个或多个 AttributeValue(属性值)。当 GroupDimensionDescriptor(组维度描述符)与 DSD(数据结构定义)中的 GroupRelationship(组关系)有关联时,则 GroupKey(组键)出现。在同一个 DataSet(数据集)中可以存在许多类型的组。如果 Group(组)与 DSD(数据结构定义)中的 DimensionRelationship(维度关系)相关联时,则 AttributeValue(属性值)将与 SeriesKey(序列键)或 Observation(观测)中的适当的维度一同报告。

通过这种方式,每个 DataSet(数据集)、SeriesKey(序列键)、GroupKey(组键)和 Observation(观测)都可以拥有零个或多个 AttributeValue(属性值),这些属性值定义了它所关联的对象的元数据。所允许的 ConceptS(概念)和这些元数据所关联(依附)的对象在 DSD(数据结构定义)中进行定义。

AttributeValue(属性值)链接到其自身所关联的对象类型(DataSet(数据集)、SeriesKey(序列键)、GroupKey(组键)、Observation(观测))。

8.4.3.2 数据集等类及其属性说明

数据集等类及其属性如表 16 所示。

表 16 数据集等类及其属性表

类	特征/属性	说明
<i>DataSet</i>	抽象类 子类 <i>GenericDataSet</i> <i>StructureSpecificDataSet</i> <i>GenericTime</i> <i>SeriesDataSet</i> <i>StructureSpecificTime</i> <i>SeriesDataSet</i>	组织化的数据集合
	<i>reportingBegin</i>	确定时间周期体系中一段特定时间周期,标识了报告的起始期
	<i>reportingEnd</i>	确定时间周期体系中一段特定时间周期,标识了报告的结束期
	<i>dataExtractionDate</i>	特定的时间段,标识了从数据源抽取数据的日期和时间
	 <i>validFrom</i>	指明了数据集中信息有效性的起始时间
	<i>validTo</i>	指明了数据集中信息有效性的结束时间
	<i>publicationYear</i>	规定了数据或元数据的发布年份,而不论使用了什么供应协议
	<i>publicationPeriod</i>	规定了数据或元数据的发布周期,而不论使用了什么供应协议
	<i>setId</i>	给出了数据集的标识
	<i>action</i>	定义了接收系统所采取的行动(更新、添加、删除)
	<i>describedBy</i>	将数据流定义以及 DSD(数据结构定义)关联到数据集
	+ <i>structuredBy</i>	关联到定义了 <i>DataSet</i> (数据集)结构的 DSD(数据结构定义)。 注: 这个 DSD(数据结构定义)与关联到 <i>Dataflow Definition</i> (数据流定义)的 DSD(数据结构定义)(非必选型)相同

表 16 (续)

类	特征/属性	说明
	+publishedBy	关联到报告/出版数据的数据提供者
	+attachedAttribute	关联到与 DataSet(数据集)相关的 Attribute Values(属性值)
GenericDataSet		包含了遵循某一 DSD(数据结构定义)的数据的数据格式结构
StructureSpecificDataSet		包含了遵循某一特定 DSD(数据结构定义)的数据的数据格式结构
GenericTimeseriesDataSet		包含了遵循某一 DSD(数据结构定义)的时间序列数据的数据格式结构
StructureSpecific TimeseriesDataSet		包含了遵循某一特定 DSD(数据结构定义)的时间序列数据的数据格式结构
Key	抽象类 子类 SeriesKey GroupKey	包含了唯一标识 Observation(观测)的维度的向量值
	keyValues	关联到组成 Key(键)的每个 Key-Values(键值)
	+attachedAttribute	关联到与 SeriesKey(序列键)或 GroupKey(组键)相关联的 Attribute Values(属性值)
KeyValue	抽象类 子类 MeasureKeyValue TimeKeyValue CodedKeyValue UncodedKeyValue	键组件的值,例如 DSD(数据结构定义)的 Dimension Descriptor(维度描述符)中的 Dimension(维度)实例的值
	+valueFor	关联到 DSD(数据结构定义)中的键组件,且此 KeyValues(键值)对于该 DSD(数据结构定义)来说是有效表示法 注:这是概念性关联,这是因为该键组件在数据集中明确地进行了标识
MeasureKeyValue	继承自 KeyValue	键的 Measure Dimension(测量维度)组件的值。该值就是此类所关联的 Concept(概念)

表 16 (续)

类	特征/属性	说明
	+value	关联到 Concept(概念) 注:这是概念性关联,它表明 Concept(概念)应存在于与 DSD 中的 Measure Dimension(测量维度)相关联的 Concept Scheme(概念方案)之中。而该测量维度源于数据结构定义。在实际 DataSet(数据集)中,此 Concept(概念)的值是存在于 KeyValue(键值)
TimeKeyValue	继承自 KeyValue	键的 Time Dimension(时间维度)组件的值
CodedKeyValue	继承自 KeyValue	键的已编码组件的值。该值就是此类所关联的 Code(代码)
	+value	关联到 Code(代码) 注:这是概念性关联,它表明 Code(代码)应存在于与 DSD 中的 Measure(维度)相关联的 Codelist(代码表)之中。在实际 DataSet(数据集)中,此 Code(代码)值存在于 KeyValue(键值)
UnCodedKeyValue	继承自 KeyValue	键的未编码组件的值
	value	键组件的值
	startTime	只有属性的 textFormat(文本格式)属于 DSD(数据结构定义)中的 Timespan(时间跨度)类型时,才会使用该属性(在这种情况下,值域是一个时间段)
	+valueFor	将 Dimension(维度)、MeasureDimension(测量维度)或 TimeDimension(时间维度)关联到键值,从而关联到作为 Dimension(维度)或 TimeDimension(时间维度)语义的 Concept(概念)
GroupKey	继承自 Key	为了能够依附 DataAttribute(数据属性),KeyValues(键值)集组成部分键,而部分键与 Time Series Key(时间序列键)具有相同维数
	+describedBy	关联到 DSD(数据结构定义)中的 Group Dimension Descriptor(组维度描述符)

表 16 (续)

类	特征/属性	说明
SeriesKey	继承自 Key	包含所有 KeyValues (键值) 的向量值, 与 + observation Dimension (观测维度) 的键值一起, 对 Observation (观测) 进行了唯一标识
	+ describedBy	关联到 DSD (数据结构定义) 中所定义的 Dimension Descriptor (维度描述符)
Observation		在 KeyValues (键值) 组成键语境下所观测到的值
	+ valueFor	关联到 DSD (数据结构定义) 中所定义的 Primary Measure (主要测量)
	+ attachedAttribute	关联到与 Observation (观测) 相关联的 Attribute Values (属性值)
	+ observationDimension	与拥有“观测层级维度”值的 Key Value (键值) 的关联
ObservationValue	抽象类 子类 UncodedObservation CodedObservation	
UncodedObservation	继承自 ObservationValue	具有文本值的观测
	value	Uncoded Observation (未编码观测) 的值
CodedObservation	继承自 ObservationValue	从代码表中的代码取得其值的观测
	+ value	关联到作为 Observation (观测) 值的 Code (代码) 注: 这是概念性关联, 它表明 Code (代码) 应与 DSD 中的 Primary Measure (主要测量) 或 Measure Dimension (测量维度) 相关联的 Code list (代码表) 之中。在实际 Data Set (数据集) 中, 此 Code (代码) 的值是存在于 Observation (观测) 中
AttributeValue	抽象类 子类 UncodedAttributeValue CodedAttributeValue	属性值, 例如在诸如 DSD (数据结构定义) 等结构中的 Coded Attribute (已编码的属性) 或 Uncoded Attribute (未编码的属性) 的实例

表 16 (续)

类	特征/属性	说明
	value	属性值
	+ valueFor	关联到 DSD(数据结构定义)所定义的 DataAttribute(数据属性) 注：这是概念性关联,这是因为该 Concept(概念)在数据集中明确地进行了标识
UncodedAttribute Value	继承自 AttributeValue	具有文本值的属性值
	startTime	只有属性的 textFormat(文本格式)属于 DSD (数 据 结 构 定 义) 中 的 Timespan(时间跨度)类型时,才会使用此属性(在这种情况下,值域取一个时间段)
CodedAttribute Value	继承自 AttributeValue	从 Code list(代码表)中的 Code(代码)取得其值的属性
	+ value	关联到 AttributeValue(属性值)的值的代码 注：这是概念性关联,它表明 Code (代码)应存在于与 DSD 中的 Data Attribute(数据属性)相关联的 Code list (代码表)之中。在实际 DataSet(数据集)中,此 Code (代码)值存在于 At-tributeValue(属性值)

9 数据立方体

9.1 语境

某些统计系统基于“立方体”结构创建了数据视图。实质上,数据立方体就是一个 n 维的对象,且对象中每一维的值都可从层级代码表中获得。这种立方体系统的用途在于它可以“向上综合”或“向下细分”每个维度的每个层次级别,以给出数据“视图”所要求的粒度等级。有些维度可能是向上综合,有些维度是向下细分。随着数据聚合,维度位置向上综合,这样,这些系统给出了数据的动态视图。例如,单个国家可以向上综合到诸如欧盟的经济区域,或诸如欧洲的地理区域,而在另一个维度上,诸如“道路类型”,则可以向下细分至更低级别。所获得的测量(诸如“事故数量”)将是每个国家特定道路类型的值的集合。

这样的数据立方体系统不依赖于简单的代码表,而是依赖于层级代码集(见第 11 章)。

9.2 支持信息模型中的数据立方体

采用 DSD(数据结构定义)结构(每个维度值如果被编码,则其值应取自于平行代码表)所报告的数据可以由数据立方体定义来描述,并且可以由具备处理数据立方体能力的系统来进行处理。SDMX-IM

通过以下方式支持此类数据立方体的定义：

- a) HierarchicalCodeList(层级代码表)定义了(通常是复杂的)代码的层级；
- b) 如果必要,StructureSet(结构集)可以：
 - 1) 将描述数据立方体的 DataStructureDefinition(数据结构定义)进行归组；
 - 2) 在 DataStructureDefinition(数据结构定义)所使用的平行代码表中的代码与 HierarchicalCodeList(层级代码表)之间提供了一种映射机制,此处,HierarchicalCodeList(层级代码表)所使用的代码表并没有在该 DSD(数据结构定义)中进行使用。

10 元数据结构定义和元数据集

10.1 语境

SDMX 元模型允许元数据：

- a) 无需将其嵌入到它所描述的对象中就可以进行交换；
- b) 与其所描述的对象隔离存储,同时还链接到元数据[例如,系统或服务可以通过机构的元数据仓库对元数据所属的对象进行访问。这种方式在 web 中是常见的,即通过点击元数据所依附的对象“信息”图标,可以查看(并下载)更多的元数据]；
- c) 通过索引支持搜索(例如:注册服务可以处理元数据报告,提取结构化信息,形成元数据目录,以支持用户查询)；
- d) 根据已定义的结构进行报告。

为了达到这个目标,对以下结构进行建模：

- a) 元数据结构定义具有以下组件：
 - 元数据所关联(依附)的对象类型；
 - 共同包含了元数据所关联的对象类型的唯一键的组件；
 - 包含了能够依附到不同对象类型的元数据属性(这些属性可以以层级进行结构化),以及可能适用的约束(例如,元数据中的属性可以与代码标中的有效值进行关联)的报告结构。
- b) 包含了所报告元数据的集合。

10.2 继承

10.2.1 简介

与 Data Structure Definition Structure(数据结构定义结构)一样,模型中该层的许多结构都继承自 SDMX 基础层。因此,有必要从继承图和关系图来理解各个模型包的功能。图 25 显示了有关 MetadataStructureDefinition(元数据结构定义)和 MetadataSet(元数据集)类的完整继承树形图。

MetadataStructureDefinition(元数据结构定义)包中的类几乎都是继承自 SDMX 基础层中的类。换句话说,SDMX 基础层根据关联和属性给出了该子模型的绝大多数结构。本章的关系图清晰地表明了这些关联是继承自 SDMX 基础层。

注：用于 MetadataStructureDefinition(元数据结构定义)的 SDMX Base 结构与用于 DataStructureDefinition(数据结构定义)的 SDMX Base 结构是相同的,即使其用法稍有不同,但定义 MetadataStructureDefinition(元数据结构定义)的根本方式也与用于 DataStructureDefinition(数据结构定义)的方式类似。

10.2.2 继承类图

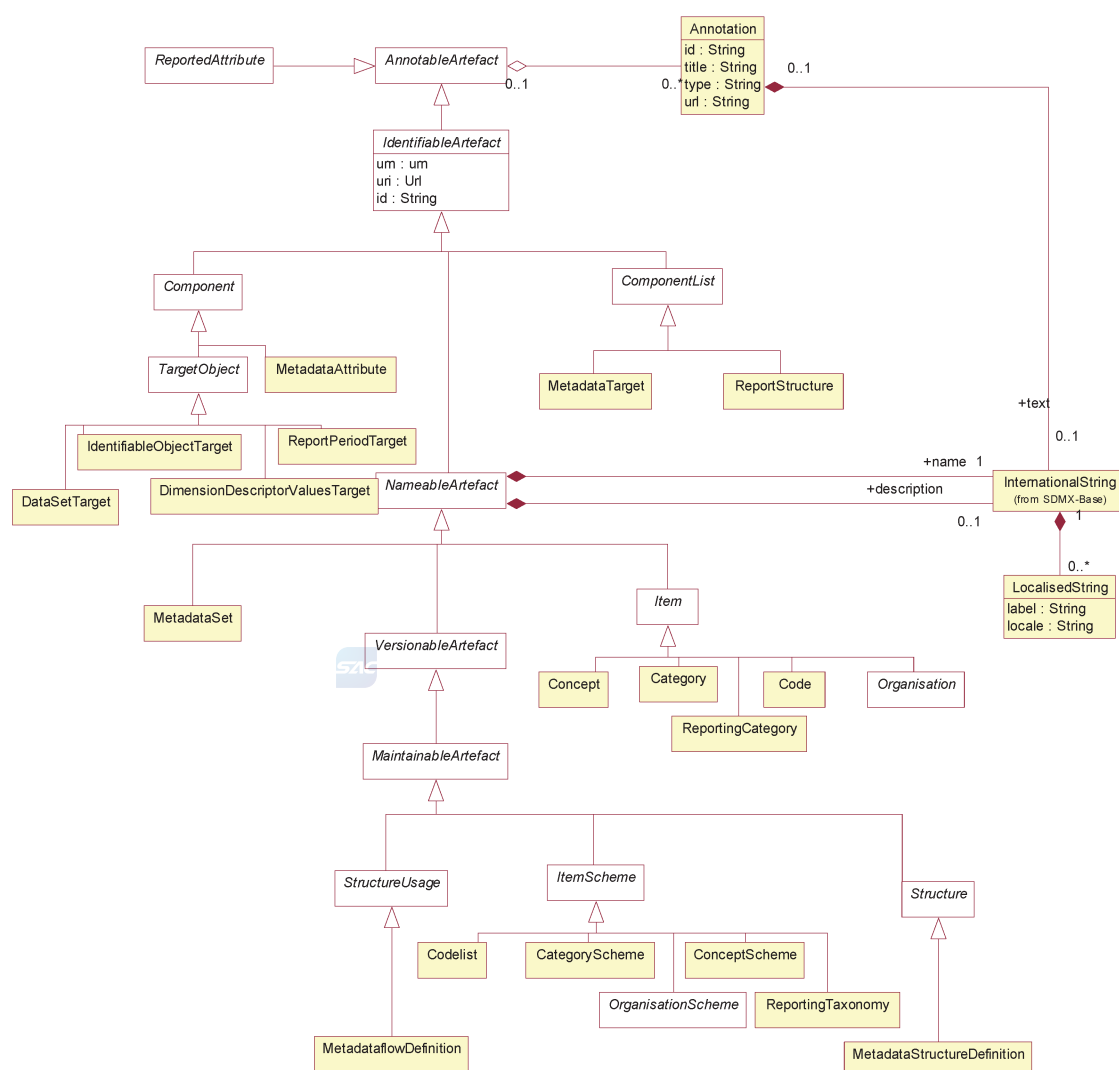


图 25 元数据结构定义的继承类图

10.2.3 类图说明

本章的关系类图旨在标识从抽象类继承而来的具体类。

SDMX 元模型中的这些具体类要求由 Maintenance Agencies(维护机构)进行维护,而且全部从 MaintainableArtefact(可维护产物)类继承而来。这些可维护产物是:

- StructureUsage(结构用法)(具体类为 MetadataflowDefinition);
- Structure(结构)(具体类为 MetadataStructureDefinition(元数据结构定义))。

这些类还继承了 IdentifiableArtefact(可标识物)、NameableArtefact(可命名物)和 VersionableArtefact(版本可控制物)的标识和版本控制的刻画。

Structure(结构)包含了多个组件列表。从 ComponentList(组件列表)继承而来且本身为 MetadataStructureDefinition(元数据结构定义)的子组件的具体类为:

- MetadataTarget(元数据目标);
- ReportStructure(报告结构)。

ComponentList(组件列表)包含 ComponentS(组件集)。从 Component(组件)继承而来的类有:

- *TargetObject* (目标对象) 类的子类;
- *MetadataAttribute* (元数据属性)。

10.3 元数据结构定义

10.3.1 简介

本章其余部分的类图和说明表明了这些具体类之间是如何关联以支持所要求的功能。

10.3.2 结构

MetadataStructureDefinition (元数据结构定义) 使用以下 *ItemScheme* (项方案), 或用作模型中的明确具体类, 或用作包含 *TargetObject* (目标对象) 值域的可能列表。

- *CategoryScheme* (分类方案);
- *ConceptScheme* (概念方案);
- *Codelist* (代码表);
- *OrganisationScheme* (组织方案);
- *Reporting Taxonomy* (报告分类法)。

10.3.3 关系类图

元数据结构定义的关系类图如图 26 所示。

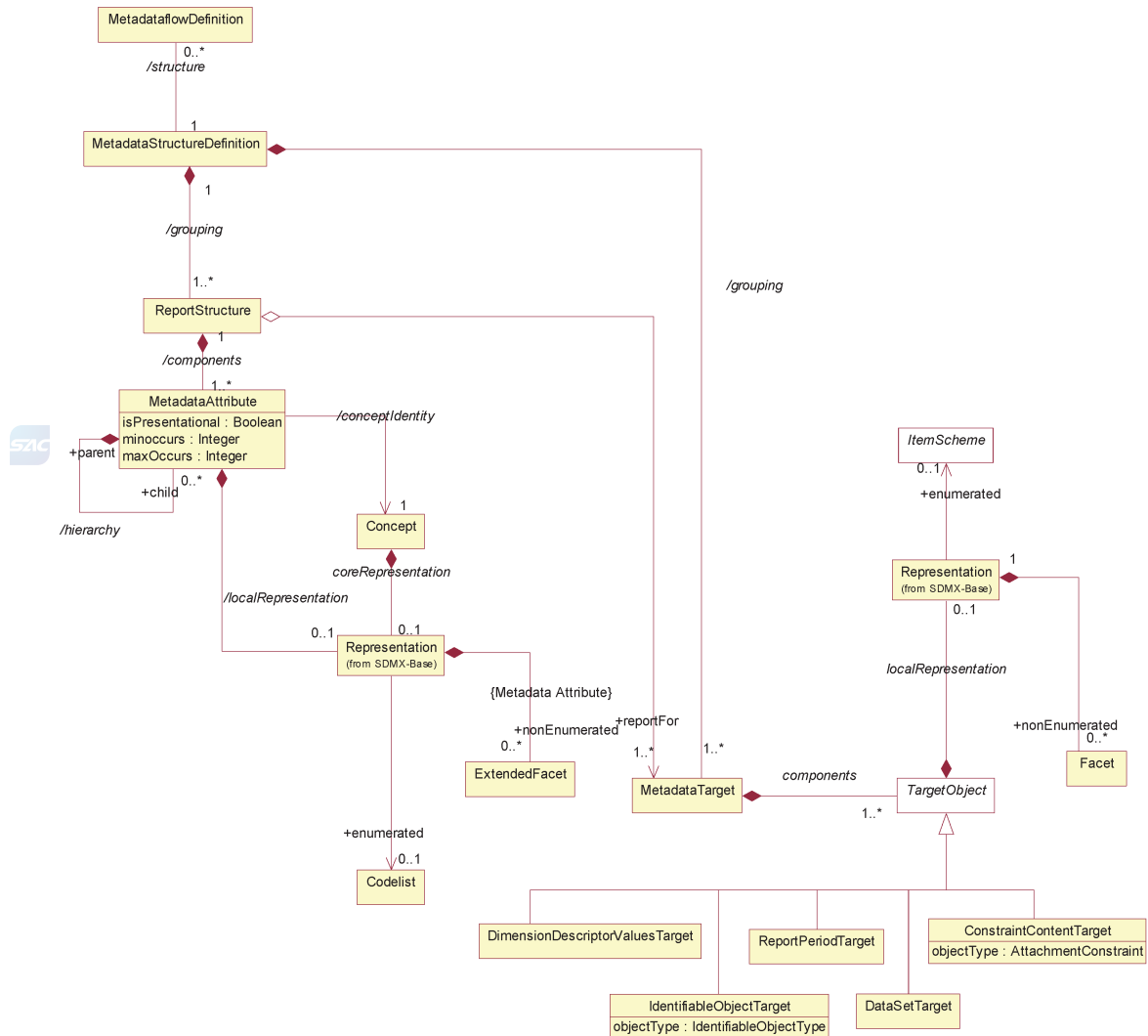


图 26 元数据结构定义的关系类图

10.3.4 类图说明

10.3.4.1 描述

简而言之, MetadataStructureDefinition(MSD)(元数据结构定义)做了以下定义:

——MetadataTarget(元数据目标): 定义了组件(TargetObject(目标对象))及其 Representation(表示法), 它们对该 MSD(元数据结构定义)是有效的, 并且是一个或多个 ReportStructure(报告结构)的元数据目标对象;

——ReportStructures(报告结构): 包含与所引用的 MetadataTarget(元数据目标)中所标识的对象类型相关联的 MetadataAttributes(元数据属性), 还包含了属性的层级结构。

MetadataTarget(元数据目标)包含一个或多个 TargetObjects(目标对象)。TargetObjects(目标对象)的组合标识了 MetadataSet(元数据集)中元数据所依附的特定对象类型。

TargetObjects(目标对象)是以下之一:

——DimensionDescriptorValuesTarget(维度描述值目标), 可在 MetadataSet(元数据集)中规定完整或部分键(如同数据集中所使用的那样)作为目标对象;

——identifiableObjectTarget(可标识对象目标), 定义特定的对象类型, 此对象类型可以是任意的 IdentifiableArtefact(可标识物);

——DataSetTarget(数据集目标), 规定了目标对象为 DataSet(数据集);

——ReportPeriodTarget(报告期目标)——规定了报告期应存在于 MetadataSet(元数据集)中;

——ConstraintContentTarget(约束内容目标), 规定了目标对象就是 AttachmentConstraint(依附约束)的内容, 也就是, 由 AttachmentConstraint(依附约束)的内容所标识的数据集或元数据集的一部分。

当 TargetObject(目标对象)在 MetadataSet(元数据集)中进行报告时, TargetObject(目标对象)的有效内容以 Representation(表示法)形式进行定义。这可以是枚举型表示法(也就是说, 对 ItemScheme(项方案)子类的引用: Codelist(代码表)、ConceptScheme(概念方案)、OrganisationScheme(组织方案)、CategoryScheme(分类方案)或 ReportingTaxonomy(报告分类法));也可以是非枚举型表示法。

因此, 对于一组离散的相关联的对象类型可以只定义一个 MetadataStructureDefinition(元数据结构定义)。例如, 只构建一个定义来定义可依附到 DSD(数据结构定义)任意部分的元数据, 或者定义可依附到与质量元数据的报告相关产物的元数据(例如数据提供者和(数据)分类)。MetadataTarget(元数据目标)规定了 MetadataSet(元数据集)中元数据所依附的特定对象类型的标识特性。例如, 在 DSD(数据结构定义)中, MetadataTarget((元数据目标)可以是一个 Dimension(维度), 因此 TargetObject(目标对象)能够唯一地标识 Dimension(维度)。这将同时包含 DataStructureDefinition(数据结构定义)和 Dimension(维度)(两者均为 IdentifiableArtefact(可标识物), 并将使用 IdentifiableObjectTarget(可标识对象目标), 这是因为需要使用这两个 TargetObjectS(目标对象), 以便唯一标识 Dimension(维度))。

ReportStructure(报告结构)包含一组 MetadataAttributeS(元数据属性)——这些属性可以是层级。每个 MetadataAttributeS(元数据属性)都标识了一个 Concept(概念), 该概念将在采用了 DSD(元数据结构定义)的 MetadataSet(元数据集)(/conceptIdentity(概念标识))中进行报告或发布。在同一个 ReportStructure(报告结构)中, 不同的 MetadataAttributeS(元数据属性)可以使用不同 ConceptSchemeS(概念方案)的 Concepts(概念)。

注: MetadataAttribute(元数据属性)不能链接至 DSD(元数据结构定义)中定义其角色的 Concept(概念)(也就是, MetadataAttribute(元数据属性)并不担任角色)。

MetadataAttribute(元数据属性)可以出现多次, 可以为必选型(minOccurs=1 或更多)或条件型(minOccurs=0)。ReportStructure(报告结构)层级可以通过为 MetadataAttribute(元数据属性)的层

级来进行定义。

ReportStructure(报告结构)与一个或多个 MetadataTargetS(元数据目标)相关联,当它们在 MetadataSet(元数据集)中进行报告时,元数据目标规定了 ReportStructure(报告结构)中 MetadataAttributeS(元数据属性)所依附的对象。

由此可以看出,对 MetadataAttribute(元数据属性)所依附的对象类型的规定是间接的:MetadataAttribute(元数据属性)在 ReportStructure(报告结构)中进行定义,而报告结构本身依附于一个或多个 MetadataTarget(元数据目标),而由 TargetObjectS(目标对象)所标识的实际对象组成了 MetadataTarget(元数据目标)。这样就给出了一种灵活的机制,通过这种机制,实际对象类型无需在该模型中以具体术语进行定义,但是可以在 MSD(元数据结构定义)中进行动态定义,其定义的方式与数据观测在 DSD(数据结构定义)中所“依附”的键非常相似。通过这种方式,MSD(元数据结构定义)可以用于定义 MetadataAttribute(元数据属性)的任意集合,以及这些元数据属性可以依附的对象类型任意集合。

每个 MetadataAttribute(元数据属性)可以有一个已规定好的 Representation(表示法)(采用/localRepresentation 关联)。如果没有在 MSD(元数据结构)中进行规定,则 Representation(表示法)将采用为 Concept(概念)所定义的表示法(coreRepresentation 关联)。

在 Base 构造规范中存在不同类型 Representation(表示法)的定义。

注:如果 Representation(表示法)是非枚举型,则关联指向 ExtendedFacet(扩展刻面)(此刻面允许 xhtml 作为 FacetValueType(刻面值类型))。如果 Representation(表示法)是枚举型的,则表示法应使用 Codelist(代码表)。

MSD(元数据结构定义)与 MetadataflowDefinition(元数据流定义)相链接。MetadataflowDefinition(元数据流定义)除了从 Base 类继承而来的属性外没有任何其他属性。

10.3.4.2 结构用法等类及其属性说明

结构用法等类及其属性如表 17 所示。

表 17 结构用法等类及其属性表

类	特征/属性	说明
StructureUsage		参见“SDMX 基础层”
Metadataflow Definition	继承自: StructureUsage	由供应者为不同引用期而提供的元数据流的抽象概念(即没有任何元数据的结构)
	/structure	关联了 Metadata Structure Definition(元数据结构定义)
MetadataStructure Definition		当用于采集或发布引用元数据时,它是元数据概念、元数据结构以及元数据用法的集合
	/grouping	关联到 Metadata Target(元数据目标)或 Report Structure(报告结构)
MetadataTarget	继承自 ComponentList	定义了元数据可能依附的对象类型键的一组组件
	/components	关联到定义了 Metadata Target(元数据目标)键的 Target Object(目标对象)组件

表 17 (续)

类	特征/属性	说明
<i>TargetObject</i>	抽象类 子类 <i>DimensionDescriptorValues</i> <i>Target</i> <i>IdentifiableObjectTarget</i> <i>DataSetTarget</i> <i>ReportPeriodTarget</i>	
	 /localRepresentation	将 Representation (表示法) 关联到 Target Object (目标对象), 该目标对象在 Metadata Set (元数据集) 中进行标识时应采用该目标对象。这种表示法可以是枚举型, 也可以是非枚举型
<i>DimensionDescriptorValuesTarget</i>	继承自 <i>TargetObject</i>	目标对象是数据序列的键
<i>IdentifiableObjectTarget</i>	继承自 <i>TargetObject</i>	此目标对象是已规定的对象类型
	objectType	标识对象类型
<i>DataSetTarget</i>	继承自 <i>TargetObject</i>	目标对象是 DataSet (数据集)
<i>ReportPeriodTarget</i>	继承自 <i>TargetObject</i>	该目标是一个报告期 注: 这并没有描述对象的用法, 而是作为一种用于元数据报告的唯一元数据键。依附到某一特定对象的元数据报告可以随时间的变化而变化, 且这个时间标识符组件可以消除各个报告的歧义, 其作用非常类似于数据序列中区分观测的时间维度
<i>ConstraintTarget</i>	继承自 <i>TargetObject</i>	目标对象是在附件约束 (Attachement Constraint) 内容中所标识的数据或引用元数据
<i>ReportStructure</i>	继承自: <i>ComponentList</i>	定义了由将要进行报告的元数据属性 (Metadata Attributes) 所组成的一组概念
	/components	关联到与 Report Structure (报告结构) 相关的 Metadata Attribute (元数据属性)

表 17 (续)

类	特征/属性	说明
	+reportFor	与使用了 Report Structure (报告结构) 的 Metadata Targets (元数据目标) 相关联
MetadataAttribute		在 Metadata Set (元数据集) 中, 标识出能够报告其值的概念 (Concept)
	/hierarchy	关联到一个或多个子 Metadata Attribute (元数据属性)
	/conceptIdentity	关联到定义了属性语义的概念
	isPresentational	指出了 Metadata Attribute (元数据属性) 的存在是为了结构化目的 (也就是说, 它拥有子属性), 而且在使用了 Report Structure (报告结构) 的 Metadata Set (元数据集) 中预期不会对属性值进行报告
	minOccurs maxOccurs	规定了在 Metadata Report (元数据报告) 的某点, Metadata Attribute (元数据属性) 的出现次数
ConceptUsage		将 Concept (概念) 作为 Metadata Attribute (元数据属性)
	concept	关联到 ConceptScheme (概念方案) 中的 Concept (概念)
	/localRepresentation	关联到 Representation (表示法), 该表示法不考虑用于规定 Concept (概念) 本身的任何核心表示法
Representation		Metadata Attribute (元数据属性) 的表示法

10.4 元数据集

10.4.1 类图

元数据集的关系类图如图 27 所示。



义的 MetadataTarget(元数据目标),也包含了 TargetObjectKey(目标对象键)和 ReportedAttributeS(报告属性)。

所标识的 ReportStructure(报告结构)规定了哪些 MetadataAttribute(元数据属性)预期成为 ReportedAttribute(报告属性)。所标识的 MetadataTarget(元数据目标)规定了 TargetObjectKey(目标对象键)的预期内容,也就是说,它规定了标识对象的必要信息,并且为该对象报告 ReportedAttribute(“报告属性”)。

TargetObjectValue(目标对象值)可以是以下之一:

——TargetDataKey(目标日期键),其中包含:

- 1) SeriesKey(序列键):维度值的集合;
- 2) SeriesKey(序列键)以及 TimeDimension(时间维度)的一个或多个值[TimeDimensionValue(时间维度值)];
- 3) TimeDimension(时间维度)值中的一个值。

——TargetIdentifiableObject(可标识的目标对象),用于标识任意可标识的对象(包括既可维护又可标识的对象);

——TargetDataSet(目标数据集),标识 DataSet(数据集);

——TargetReportPeriod(目标报告期),用于规定 Report(报告)的时期。

ReportedAttribute(报告属性)的简单文本值使用 ReportAttribute(报告属性)的 NonEnumeratedAttributeValue(非枚举型属性值)子类,而代码值使用 EnumeratedAttributeValue(枚举型属性值)子类。

NonEnumeratedAttributeValue(非枚举型属性值)可以是以下之一:

——XHTMLAttributeValue(XHTML 属性值):内容为 XHTML;

——TextAttributeValue(文本属性值):内容为文本型,并有可能包含多语种 text(文本);

——OtherNonEnumeratedAttributeValue(其他非枚举型属性值)内容是字符串值,此值应符合相关 ReportStructure(报告结构)的 MSD(元数据结构定义)中 MetadataAttribute(元数据属性)所规定的 Representation(表示法)。

EnumeratedAttributeValue(枚举型属性值)包含了一个 Code(代码)值,这个代码在相关 ReportStructure(报告结构)的 MSD(元数据结构定义)中作为 MetadataAttribute(元数据属性)的 Representation(表示法)被规定。

10.4.2.2 元数据集等类及其属性说明

元数据集等类及其属性如表 18 所示。

表 18 元数据集等类及其属性表

类	特征/属性	说明
MetadataSet		元数据的任意有组织的集合
	reportingBegin	确定时间周期体系中的一段特定时间周期,标识了报告的起始期
	reportingEnd	确定时间周期体系中的一段特定时间周期,标识了报告的结束期
	dataExtractionDate	特定的时间段,标识了从数据源抽取数据的日期和时间

表 18 (续)

类	特征/属性	说明
	validFrom	指明了元数据集中信息有效性的起始时间
	validTo	指明了元数据集中信息有效性的结束时间
	publicationYear	规定了数据或元数据的发布年份,而不论使用了什么供应协议
	publicationPeriod	规定了数据或元数据的发布周期,而不论使用了什么供应协议
	setId	给出了元数据集的标识
	action	定义了接收系统所采取的行动(更新、添加、删除)
	+describedBy	将 Metadataflow(元数据流定义)关联到 Metadata Set(元数据集)
	+structuredBy	与定义了 MetadataSet(元数据集)结构的 MSD(元数据结构定义)相关联 注:与 MSD(元数据结构定义)的关联和关联到(非强制性的)Metadata-flow Definition(元数据流定义)是相同的
	+publishedBy	与报告/出版了元数据的数据提供者相关联
	+describedByV	对 Report Structure(报告结构)的引用
MetadataReport		MSD(元数据结构定义)的 Report Structure(报告结构)中所定义的 Metada Attribute(元数据属性)的一系列值
	+attachesTo	将对象键关联到元数据所依附的对象
	+target	关联定义了目标对象的 Metadata Target(元数据目标),该目标对象是元数据将要关联的
	+metadata	关联 Reported Attribute(报告属性)值,此值与 Target Object Key(目标对象键)所标识的对象相关联

表 18 (续)

类	特征/属性	说明
TargetObjectKey		标识了元数据将要依附的对象的键
	+valueFor	与标识了对象类型和 Target Object Key (目标对象键) 组件结构的 Metadata Target(元数据目标)相关联 注: 这是与 MSD 结构链接的概念性关联。
	+keyValues	关联了 Target Object Key(目标对象键)的 Target Object Values(目标对象值)
TargetObjectValue	抽象类 子类为 TargetDataKey TargetIdentifiableObject TargetDataSet TargetReportPeriod	在 MSD(元数据结构定义)的 Metadata Target(元数据目标)中所规定的单一对象类型的键
	+valueFor	关联提供该值的 Target Object(目标对象) 注: 这是与 MSD 结构链接的概念性关联。
TargetDataKey	继承自 TargetObjectValue	组件的标识和值,它们组成了数据键或元数据键
ComponentValue		共同包含了组件的标识以及构成数据键的值
value		键值
	+valueFor	与声明了该值的组件相关联
TimeDimensionValue		包含了时间维度的标识及其值
TargetIdentifiable Object	继承自 TargetObjectValue	规定了可标识对象的标识
StructureRef		包含了可标识对象的标识
	structureType	目标对象的对象类型
Maintainable ArtefactRef Identifiable ArtefactRef		通过目标对象标识符结构形成目标对象的标识,即,目标对象标识是机构 ID、Maintainable Object(可维护对象)的 id、可维护对象的版本;以及 Identifiable Object(可标识对象)的 id
	+containedObject	关联到 Identifiable Object(可标识对象)层级中所包含的对象,如 Process Step(过程步骤)中的 Transition(转变)

表 18 (续)

类	特征/属性	说明
TargetDataSet	继承自 <i>TargetObjectValue</i>	包含了 Data Set(数据集)的标识
TargetReportPeriod	继承自 <i>TargetObjectValue</i>	包含了 Metadata Report(元数据报告)所涵盖的时间段
<i>ReportedAttribute</i>	抽象类 子类为 <i>NonEnumeratedAttributeValue</i> <i>EnumeratedAttributeValue</i>	Metadata Attribute(元数据属性)的值
	+valueFor	与 MSD(元数据结构定义)中 Metadata Attribute(元数据属性)的关联,该元数据结构定义标识了用于 Reported Attribute(报告属性)的 Concept(概念)和允许的 Representation(表示法) 注:这是与 MSD 结构链接的概念性关联。用于 Reported Attribute(报告属性)的语法将以某些形式声明 Metadata Attribute(元数据属性)的 id。
	+child	与符合层级的子 Reported Attribute(报告属性)相关联,该层级在 Metadata Attribute(元数据属性)的 Report Structure(报告结构)中进行定义,而 Metadata Attribute(元数据属性)子元素是 Reported Attribute(报告属性)
<i>NonEnumeratedAttributeValue</i>	继承自 <i>ReportedAttribute</i> 子类: <i>XHTMLAttributeValue</i> <i>TextAttributeValue</i> <i>OtherNonEnumeratedAttributeValue</i>	Reported Attribute(报告属性)的内容为文本型
XHTMLAttributeValue		它包含了 XHTML
	value	XHTML 的字符串值
TextAttributeValue		内容适应于人类阅读的文本的 Reported Attribute(报告属性)值
	text	该字符串值为文本。它可以以多语种版本的形式存在
OtherNonEnumeratedAttributeValue		内容不适应于人类阅读的文本的 Reported Attribute(报告属性)值

表 18 (续)

类	特征/属性	说明
	value	文本字符串,其格式与 Metadata Attribute (元数据属性) 的 Representation(表示法)中所定义的格式相符,而 ReportedAttribute(报告属性)是用于该元数据属性
EnumeratedAttribute Value	继承自 <i>MetadataAttributeValue</i>	Reported Attribute(报告属性)的内容取自于 Code list(代码表)中的 Code(代码)
	value	Reported Attribute (报告属性)的 Code(代码)值
	+ value	与 Metadata Attribute(元数据属性)的 Representation(表示法)中所规定的 Code list(代码表)中的 Code(代码)相关联,而该 Report Attribute(报告属性)是此 Metadata Attribute(元数据属性)的值 注: 这是与 Item(项)的概念性关联,项就是此值。在实际中,该值本身将包含在 Enumerated Attribute Value (枚举型属性值)中

11 层级代码表

11.1 范围

本章关于结构化定义所描述的 Codelist(代码表)支持 Code(代码)的简单层级结构,并且将任意一个子 Code(代码)限定为只能拥有一个父 Code(代码)。虽然这种结构对支撑 DSD(数据结构定义)和 MSD(元数据结构定义)的需求是有用的,但对于编码方案(如分类方案)中经常遇到的代码之间的更复杂的关联关系,其支撑作用则显然还不太够。通常,在 DSD(数据结构定义)中所使用的 Codelist(代码表)是从更为复杂的编码方案中衍生而来的。而调用这样的编码方案可以帮助应用程序(如 OLAP 应用程序或数据可视化系统),这比 DSD(数据结构定义)中所使用的简单的 Codelist(代码表)提供更多的数据视图。

注: 层级代码表并不一定是平衡树形图。平衡树形图中的层级是预先定义并且是固定的,(也就是说,同一层级总是拥有相同的代码集,而任何代码都拥有与其他代码之间固定的父关系和子关系)。统计分类是平衡树形图的一个实例,在特定情况下,对平衡层级的支持是层级代码表的子集。

Hierarchical Codelist(层次代码表)的主要特征为:

- 一个子代码可以拥有一个以上的父代码;
- 可以存在多个没有父代码的代码(也就是说,存在一个以上的“根节点”);
- 依据代码之间的关联关系,可以定义许多层级(或“视图”)。每个层级都在数据的报告、分析或发布中发挥特定作用;
- 层级的层次可以是明确的,也可以是隐含的(也就是,它们在编码结构中只作为父/子关系而存

在)。

11.2 继承

11.2.1 类图

层级代码表的继承类图如图 28 所示。

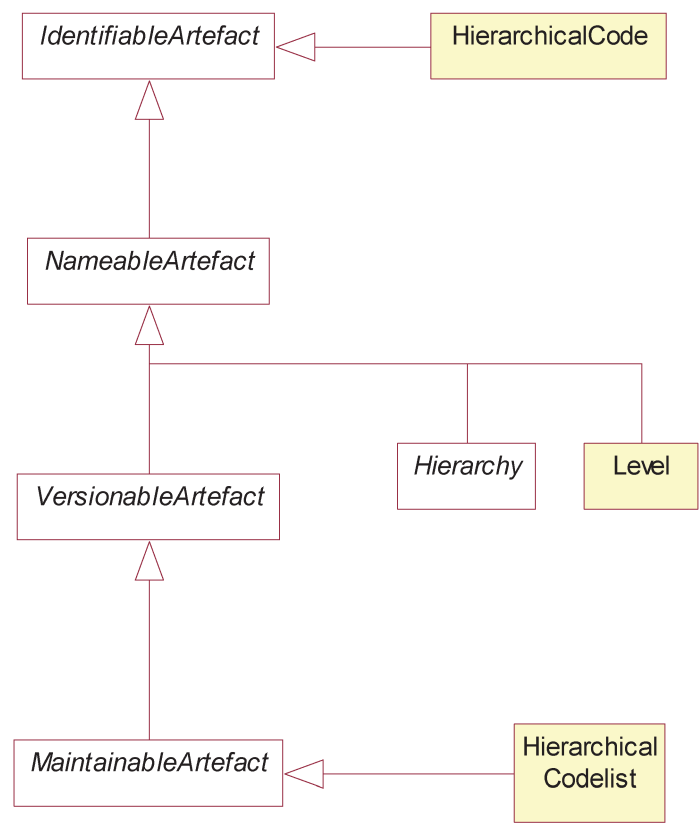


图 28 层级代码表的继承类图

11.2.2 类图说明

HierarchicalCodelist(层级代码表)继承自 MaintainableArtefact(可维护产物),因此拥有标识、命名、版本控制和维护机构。Hierarchy(层级)和 Level(层次)均为 NameableArtefact(可命名产物),因此拥有 id(标识)、多语种名称和多语种描述。而 HierarchicalCode(层级代码)是 IdentifiableArtefact(可标识产物)。

重要的是:用于 HierarchicalCodelist(层级代码表)的代码本身并不包含在此列表中,一它们被该列表引用,并在一个或多个 Codelist(代码表)中进行维护。这在图 27 的关系类图中进行了解释。

11.3 给出了各种类、属性和关联的定义。

11.3 关系

11.3.1 类图

层级代码方案的关系类图如图 29 所示。

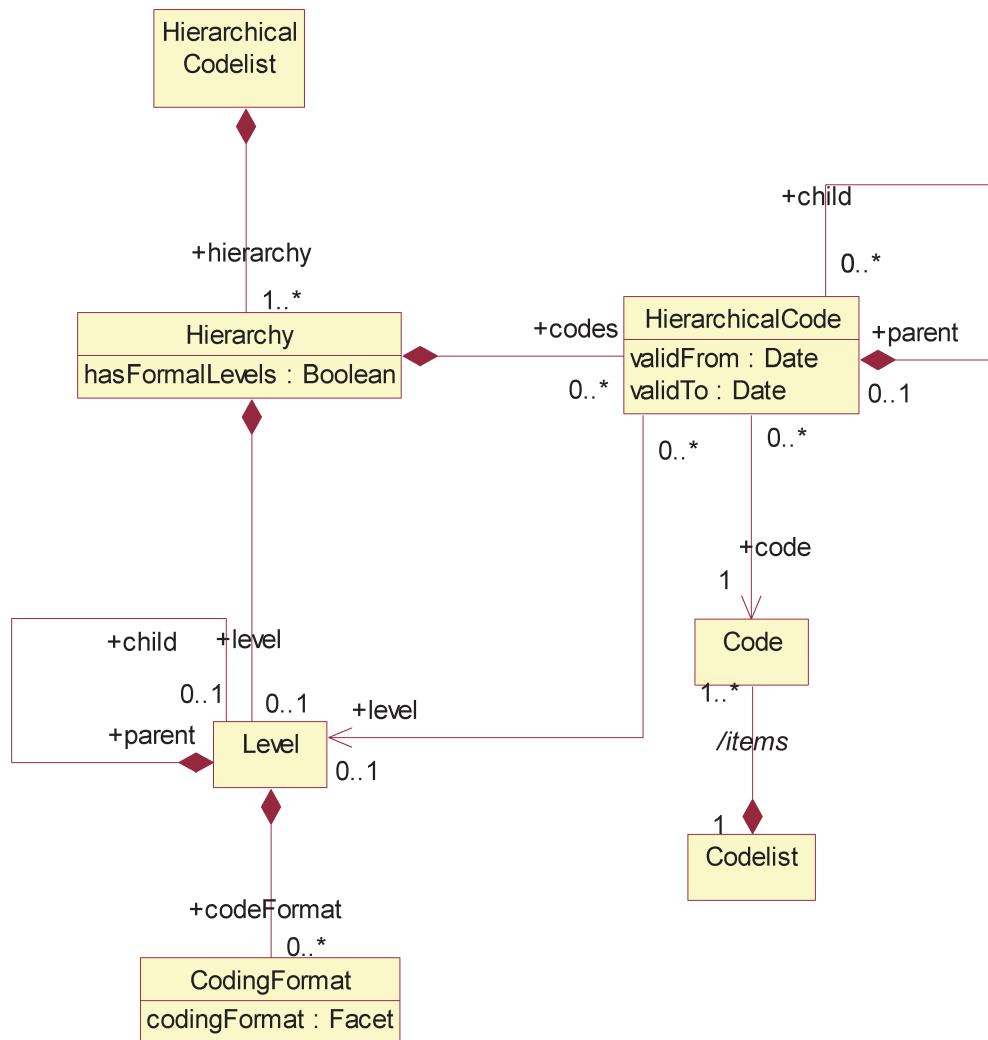


图 29 层级代码方案的关系类图

11.3.2 类图说明

11.3.2.1 描述

HierarchicalCodelist(层级代码表)的基本原则是：

- HierarchicalCodelist(层级代码表)依据一种或多种 Hierarchy(层级),是由分类方案中的 Codes(代码)及 Codes(代码)结构的规范所组成；
- HierarchicalCodelist(层级代码表)中的 Codes(代码)本身可以不是方案的一部分,相反,它们是对一个或多个外部代码表中代码的引用；
- 任意单独的代码可以出现在多个 HierarchyS(层级)中,这样便于给出 HierarchicalCodelist(层级代码表)的结构；
- Codes(代码)的 HierarchyS(层级)在 HierarchicalCode(层级代码)中进行规定。这引用了 Code(代码)以及其直接子 HierarchicalCode(层级代码)。

Hierarchy(层级)可以拥有正式的层次(hasFormalLevels="true")。然而,即使出现 hasFormalLevels="false",Hierarchy(层级)仍然可以拥有一个或多个相关联的层次,以便用于记录与 HierarchicalCodeS(层级代码)相关的信息。

如果出现“hasFormalLevels = "false"”，则 Hierarchy(层级)是包含代码的并且没有正式 Level(层次)的“基于值的”。如果“hasFormalLevels = "true"”，则层次是“基于层级的”，而每个层级都是(层级代码表)中的正式 Level(层次)，如统计分类中的层级。在“基于层次”的层级中，每个 HierarchicalCode(层级代码)均链接至其自身所处的 Level(层次)(它应与 HierarchicalCode(层级代码)处于相同的 Hierarchy(层级))。预计由 + parent/+ child 关联所定义的所有处于同一层级的 HierarchicalCodeS(层级代码)都将链接至相同 Level(层次)。

注 1：如果 HierarchicalCode(层级代码)的层级层次(HierarchicalCode(层级代码)父/子关联隐指)与层次层级(由 Level(层次)父/子关联隐指)不一致时，则应规定 + level 关联。

注 2：希望符合统计分类公认模型的组织机构应当确保 id 就是与 Level(层次)相关联的编号，而 Level(层次)是从最高层次的第一层次 1 开始连续编号。

Level(层次)可以具有已定义的 CodingFormat(编码格式)信息(例如，处于该层次的编码类型)。

11.3.2.2 层级代码表等类及其属性说明

层级代码表等类及其属性如表 19 所示。

表 19 层级代码表等类及其属性表

类	特征/属性	说明
HierarchicalCodelist	继承自： <i>MaintainableArtefact</i>	代码有组织的集合，这些代码有可能与方案中的其他代码涉及父/子关系，正如方案的一个或多个层级所定义那样
	+ hierarchy	与代码层级的关联
Hierarchy	继承自： <i>NameableArtefact</i>	按照详尽程度从最宽泛的层次至最详尽的层次排列的一种分类结构
	hasFormalLevels	如果值为“真”，则表示其结构按照详尽程度从最宽泛的层次至最详尽的层次排列的层级。 如果值为“假”，则表示层级结构中的项没有正式层次结构
	+ codes	与层级中高阶层级代码的关联
	+ level	与层级中高阶层次的关联
Level	继承自 <i>NameableArtefact</i>	在“基于层次”的层级中，这描述了按照同样编码进行描述特性的一组代码，并且组中每个代码的父代码均处于层级的同一层次。 在“基于值”的层级中，这描述了在规定嵌套层的层级代码的相关信息
	+ codeFormat	与编码格式的关联
	+ child	与 Level(层次)的子层次的关联
CodingFormat		规定了层级中该层次代码的格式信息，如处于此层次的代码是字母的、数字的，还是字母数字的，以及代码长度

表 19 (续)

类	特征/属性	说明
HierarchicalCode		代码引用的层级结构
	validFrom	生效日期
	validTo	废止日期
	+code	关联到用在层级中特定点的代码
	+child	关联到层级中的子代码
	+level	与 Level(层次)的关联,而层次是用于定义 Hierarchy(层级)
Code		用于层级中该点的代码
	/items	关联到包含该代码的代码表
Codelist		包含该代码的代码表

12 结构集和映射

12.1 范围

StructureSet(结构集)允许某一结构中的组件映射到相同类型的另一个结构中的组件。在此语境下,术语“结构”泛指包含 *ItemScheme*(项方案)类型、*Structure*(结构)类型以及 *StructureUsage*(结构用法)类型。在这些结构中,可以映射的结构及其组件见表 20。

表 20 结构类型和组件类型之间的映射

结构类型	组件类型
Codelist(代码表)	Code(代码)
Category Scheme(分类方案)	Category(分类)
Concept Scheme(概念方案)	Concept(概念)
Organisation Scheme(组织方案)	组织—允许映射任意类型的组织至另一任意类型的组织(例如,映射一个数据供应方至一个组织单元)
Hierarchical Codelist(层级代码表)	层级代码至代码或反之
Data Structure Definition(数据结构定义)	Dimension(维度)、测量维度(MeasureDimension)、时间维度(TimeDimension)、DataAttribute(数据属性)、Primary Measure(主要测量)
Metadata Structure Definition(元数据结构定义)	Target Object(目标对象)、Metadata Attribute(元数据属性)
Dataflow Definition(数据流定义)	无
Metadataflow Definition(元数据流定义)	无

StructureSet(结构集)可以包含一个或多个“映射”,并可以定义相关结构(通过关联+related-Structure),该结构可以归组相关的 DataStructureDefinitions(数据结构定义集)、MetadataStruc-

tureDefinitions(元数据结构定义集)、MetadataflowDefinintions(元数据流定义集)。

12.2 结构集

12.2.1 继承类图

结构集的继承类图如图 30 所示。

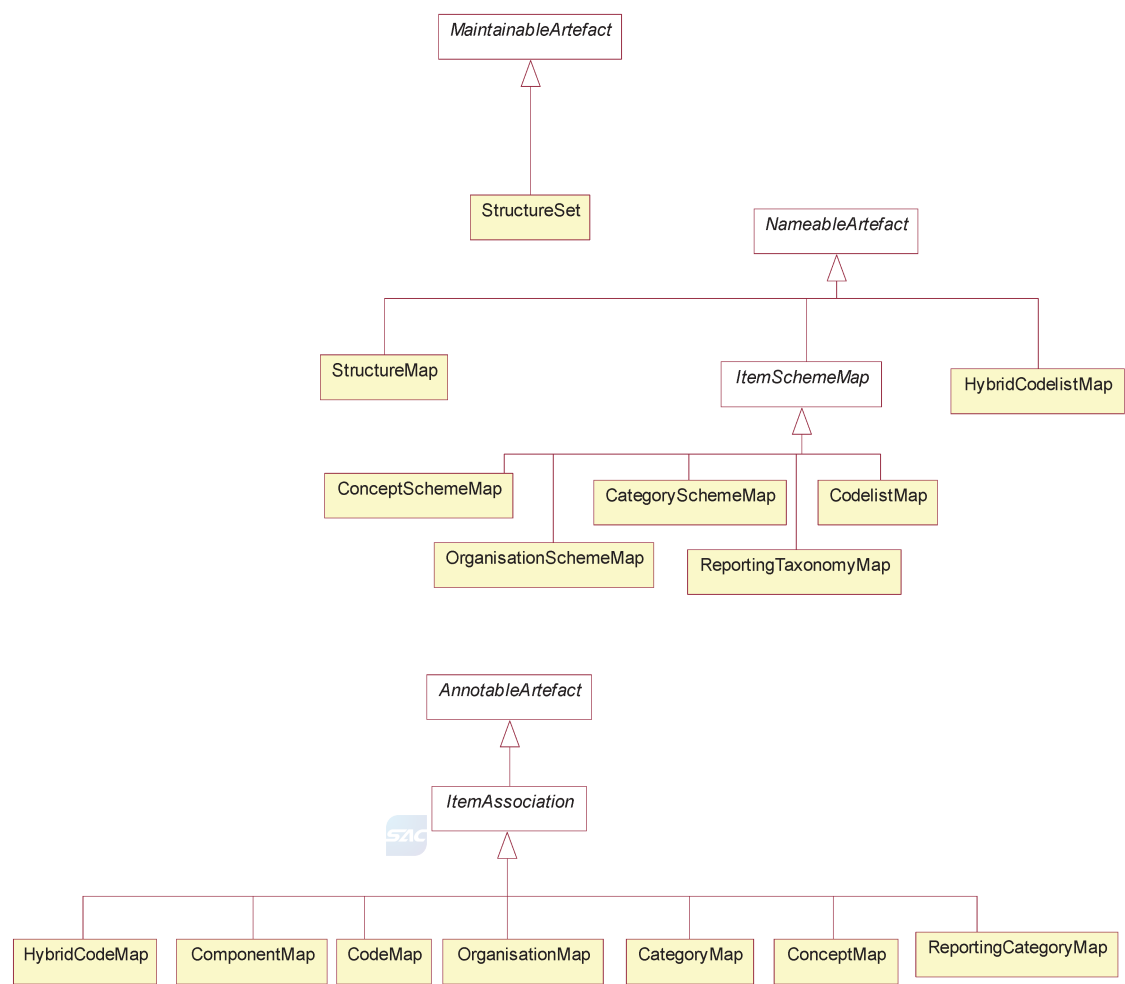


图 30 结构集的继承类图

12.2.2 关系类图

结构集的关系类图如图 31 所示。

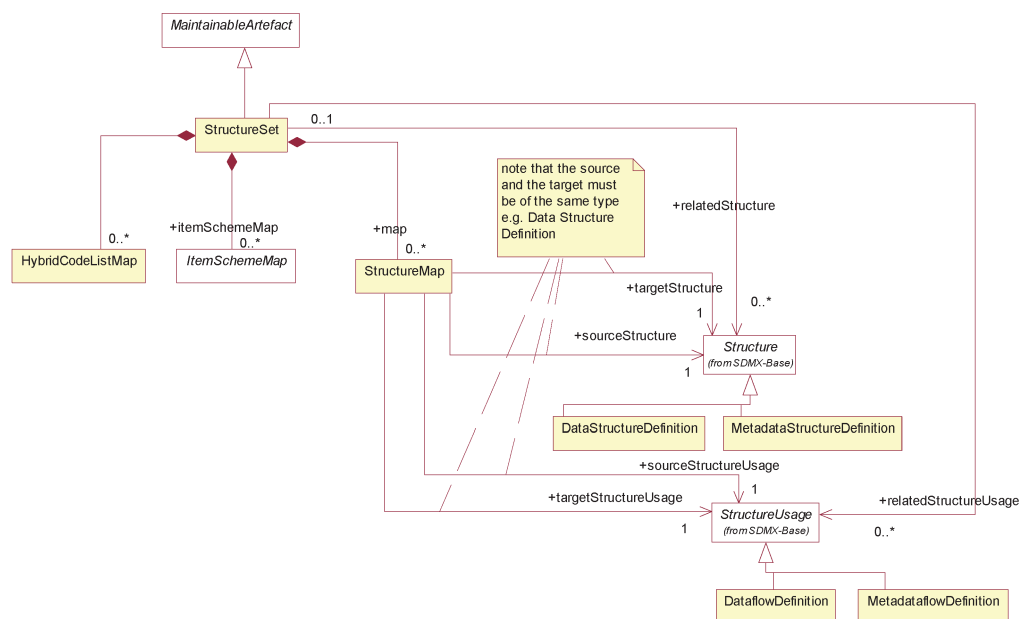


图 31 结构集的关系类图

12.2.3 类图说明

12.2.3.1 描述

StructureSet(结构集)是一种 *MaintainableArtefact*(可维护产物)。它可以包含：

- 一组对 *Structure*(结构)和 *StructureUsage*(结构用法)(DSD(数据结构定义)、MSD(元数据结构定义)、DataflowDefinition(数据流定义)或 MetadataflowDefinition(元数据流定义))的具体子类的引用,用于指明它们之间的关系。例如,这可以是一组 DSD(数据结构定义),它们共同构成了一个数据立方体的定义,而每一个 DSD(数据结构定义)定义着该数据立方体的一部分;
- 一组 *StructureMaps*(结构映射)定义了一个结构中的哪些组件等同于另一个 *ComponentMap*(组件映射)中的组件;
- 一组 *ItemSchemeMaps*(项方案映射),它定义了 *ItemScheme*(项方案)中的两个具体类之间的映射,还定义了这些项方案中的 *Item*(项)的映射,例如两个代码表(CodelistS)中的代码(Codes)之间的映射;
- 一组 *HybridCodelistMaps*(混合代码表映射),它定义了 *Codelist*(代码表)与 *HierarchicalCodelist*(层级代码表)之间的映射。

structureMap(结构映射)引用了两个 *StructureS*(结构)或 *StructureUsageS*(结构用法)。具体来说,这些将引用到 DSD(数据结构定义)、MSD(元数据结构定义)、DataflowDefinitions(数据流定义)或 MetadataflowDefinitionS(元数据流定义)。

12.2.3.2 结构集等类及其属性说明

结构集等类及其属性如表 21 所示。

表 21 结构集等类及其属性表

类	特征/属性	说明
StructureSet	继承自 <i>MaintainableArtefact</i>	结构化映射的可维护集合,它以源/目标关系将各组件链接起来,其中在源组件和目标组件之间存在着语义上的等价性
	+ relatedStructure	与一组 DSD(数据结构定义)和 MSD(元数据结构定义)的关联
	+ relatedStructureUsage	与一组数据流定义和元数据流定义的关联
	+ map	与结构映射的关联
	+ itemSchemeMap	与项方案映射的关联
StructureMap	继承自 <i>NameableArtefact</i>	源结构与目标结构之间的链接,其中在源结构与目标结构之间存在着语义上的等价性
	sourceStructure	与源结构的关联
	targetStructure	与目标结构的关联,而目标结构的类型应与源结构的类型相同
	sourceStructureUsage	与源结构用法的关联
	targetStructureUsage	与目标结构用法的关联,而目标结构用法类型应与源结构用法类型相同

12.3 结构映射

12.3.1 类图

结构映射类图如图 32 所示。

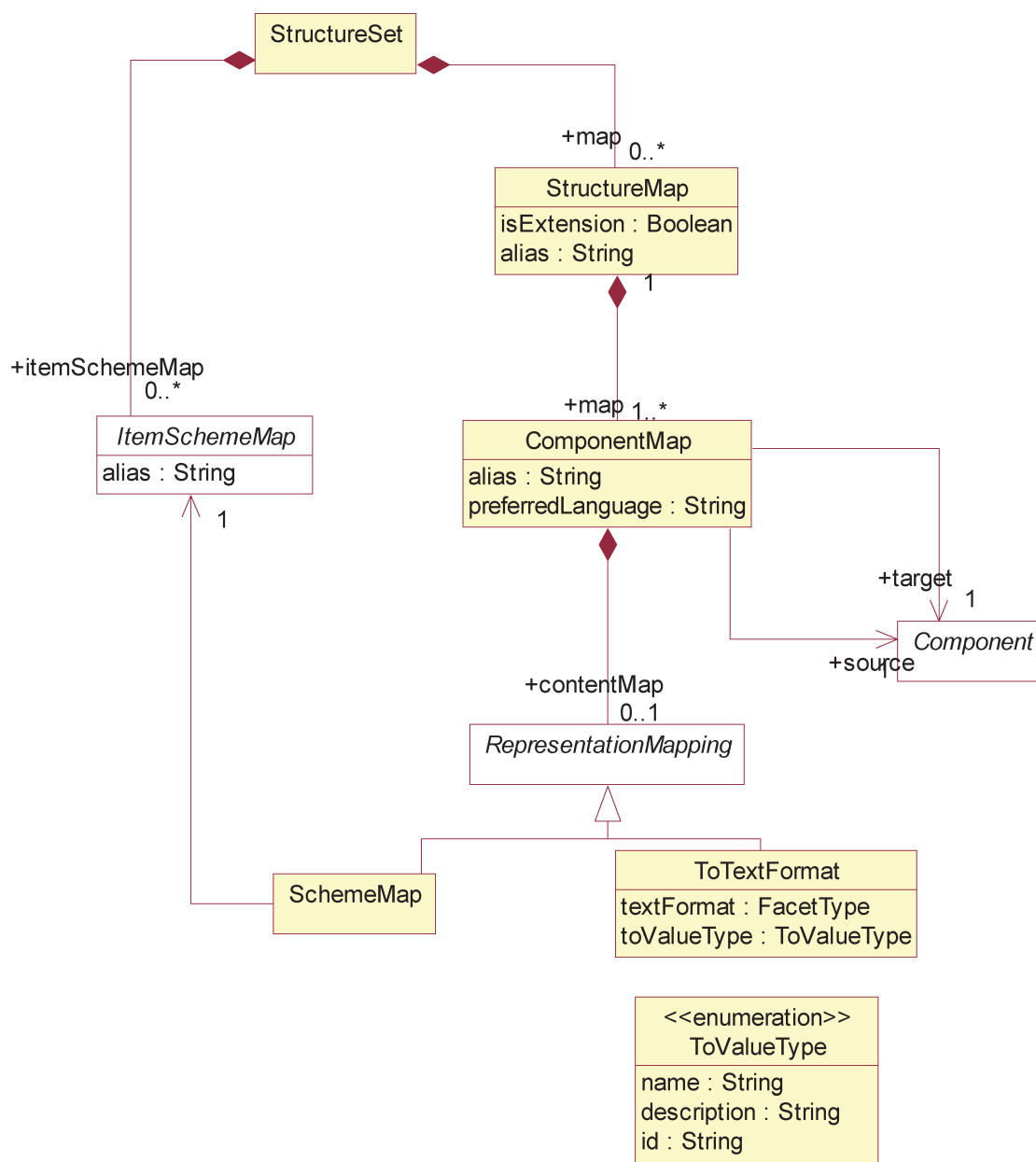


图 32 结构映射类图

12.3.2 类图说明

12.3.2.1 描述

StructureMap(结构映射)包含了一组 ComponentMapS(组件映射),而每个组件映射都指出了所引用 Structure(结构)的 Component(组件)之间的等价性。ComponentMap(组件映射)拥有一个 RepresentationMapping(表示法映射),它可以是 ItemSchemeMap(项方案映射)的一个具体类[例如,对于 Dimension(维度)来说,这可能是 CodelistMap(代码表映射)],也可以是取如下值的 ToTextFormat(至文本格式):id、name、description。这使得映射工具应使用已编码组件的标识符、名称或描述来确定与未编码组件值之间的等价性。

ComponentMap(组件映射)的一个实例是将源 DSD(数据结构定义)(在 structureMap 中所标识的)中

的 Dimension (维度) 的源 Component (组件) 链接到目标 DSD (数据结构定义) 中 Dimension (维度) 的等价目标 Component (组件)。

12.3.2.2 结构映射等类及其属性说明

结构映射等类及其属性如表 22 所示。

表 22 结构映射等类及其属性表

类	特征/属性	说明
StructureMap	继承自 <i>NameableArtefact</i>	源结构与目标结构之间的链接, 此处, 在源结构与目标结构之间存在着语义等价性
	alias	映射的另一种标识, 通过共享该值就可以表达多个映射关系
	+ map	与组件映射的关联
ComponentMap	继承自 <i>AnnotableArtefact</i>	源组件与目标组件之间的链接, 此处, 在源组件与目标组件之间存在着语义等价性
	alias	映射的另一种标识, 通过共享此值就可以表达多个映射关系
	preferredLanguage	规定了用作 RepresentationMap (表示法映射) 的 To Text Format (至文本格式) 选项内容的语种
	+ source	与源组件的关联
	+ target	与目标组件的关联
	+ contentMap	与映射了组件内容的构造的关联——这可以是项方案的一个子类, 或者是对文本的一个映射
<i>Representation Mapping</i>	抽象类 子类: <i>SchemeMap ToTextFormat</i>	定义了源组件内容与目标组件内容之间的映射
<i>SchemeMap</i>	继承自 <i>RepresentationMapping</i>	关联了项方案映射
<i>ToTextFormat</i>	继承自 <i>RepresentationMapping</i>	定义文本格式
	textFormat	文本格式类型
	toValueType	在将源组件的内容映射到目标组件的内容时, 标识了取自源组件的项中的构造
ToValueType		项中构造的枚举

12.4 项方案映射

12.4.1 语境

ItemSchemeMap(项方案映射)用于关联两个不同 ItemScheme(项方案)中的 Item(项)。存在一种通用的机制来映射 Item(项)。ItemScheme(项方案)中的 Item(项)之间存在语义等价性时,存在适合这些映射方案的特定模型。该模型支持相同类型的任意两个 ItemScheme(项方案)之间的映射。这些项方案是:

- ConceptScheme(概念方案);
- CategoryScheme(分类方案);
- OrganisationScheme(组织方案);
- Codelist(代码表);
- ReportingTaxonomy(报告分类)。

12.4.2 类图

项方案映射类图如图 33 所示。

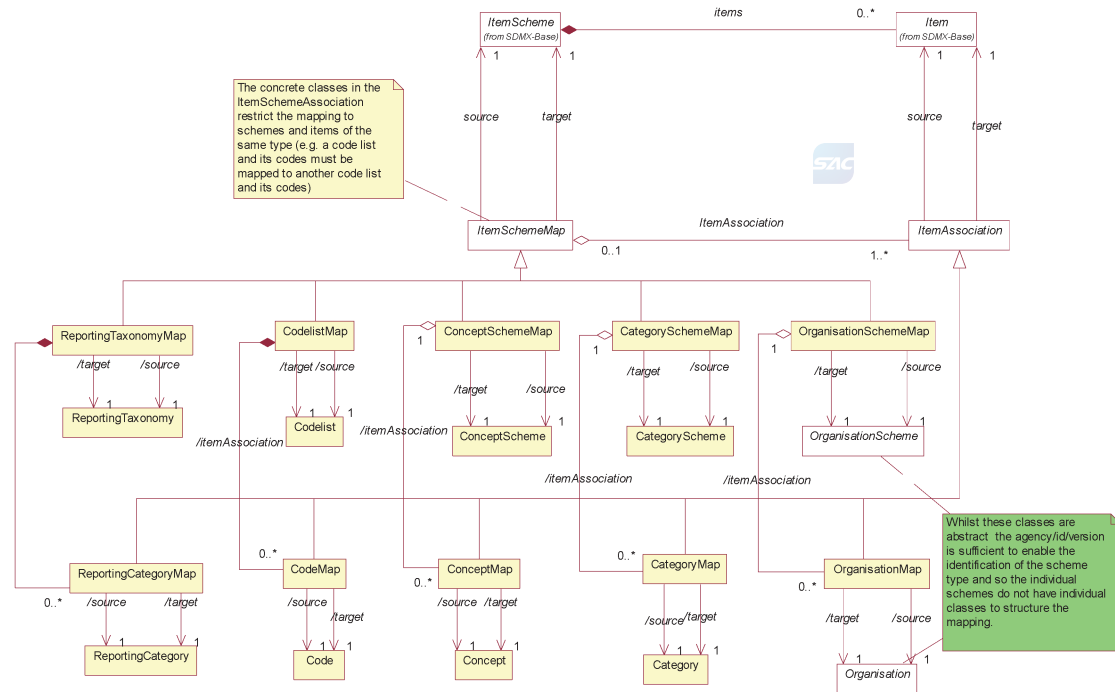


图 33 项方案映射类图

12.4.3 类图说明

12.4.3.1 描述

ItemSchemeMap(项方案映射)和 ItemAssociation(项关联)都继承自 NameableArtefact(可命名产物)。

每个 ConceptSchemeMap(概念方案映射)、CategorySchemeMap(分类方案映射)、CodelistMap(代码表映射)和 OrganisationSchemeMap(组织方案映射)、ReportingTaxonomyMap(报告分类映射)为方案中

的各个项 (Concept、Category、Code、Organization、ReportCategory) 提供了一种规定语义等价性的机制。

注 1: 任何类型的 OrganisationScheme(组织方案)和 Organisation(组织)都可以进行映射(例如, AgencySchme(机构方案)中的 Agency(机构)可以映射至 OrganisationUnitScheme(组织单元方案)中的 OrganisationUnit(组织单元))。

每个方案映射都标识了 +source(源)和 +target(目标)方案,且方案的内容得以映射。

注 2: 许多方案都可以通过一组配对映射组合在一起。ConceptMap(概念映射)、CategoryMap(分类映射)、CodelistMap(代码表映射)、OrganisationMap(组织映射)和 ReportingTaxonomyMap(报告分类映射)指明了哪些 ConceptS、CategoryS、CodeS、OrganisationS、ReportingCategoryS ReportingCategory)在语义上是等价的,并且可以规定一个共享的别名来引用一组映射的概念以便于查询。

12.4.3.2 项方案映射等类及其属性说明

项方案映射等类及其属性如表 23 所示。

表 23 项方案映射等类及其属性表

类	特征/属性	说明
<i>ItemSchemeMap</i>	继承自 <i>NameableArtefact</i> 子类 ConceptSchemeMap CategorySchemeMap CodelistMap OrganisationSchemeMap ReportingTaxonomySchemeMap	关联了两个项方案
	alias	映射的另一种标识,通过共享此值就可以表达多个映射关系
	source	关联到源项方案
	target	关联到目标项方案
	ItemAssociation	关联到项关联(Item Association)
<i>ItemAssociation</i>	继承自 <i>AnnotableArtefact</i> 子类 ConceptMap CategoryMap CodeMap OrganisationMap ReportingCategoryMap	
	source	关联到源项
	target	关联到目标项
ConceptSchemeMap	继承自 <i>ItemSchemeMap</i>	关联了源和目标概念方案

表 23 (续)

类	特征/属性	说明
	/source	关联到源概念方案
	/target	关联到目标概念方案
ConceptMap	继承自 <i>ItemAssociation</i>	关联了源和目标概念
	/source	关联到源概念
	/target	关联到目标概念
CodelistMap	继承自 <i>ItemSchemeMap</i>	关联了源和目标代码表
	/source	关联到源代码表
	/target	关联到目标代码表
CodeMap	继承自 <i>ItemAssociation</i>	关联源和目标代码
	/source	关联到源代码
	/target	关联到目标代码
CategorySchemeMap	继承自 <i>ItemSchemeMap</i>	关联了源和目标分类方案
	/source	关联到源分类方案
	/target	关联到目标分类方案
CategoryMap	继承自 <i>ItemAssociation</i>	关联了源和目标分类
	/source	关联到源分类
	/target	关联到目标分类
OrganisationSchemeMap	继承自 <i>ItemSchemeMap</i>	关联了源和目标组织方案
	/source	关联到源组织方案
	/target	关联到目标组织方案
OrganisationMap	继承自 <i>ItemAssociation</i>	关联源和目标组织
	/source	关联到源组织
	/target	关联到目标组织
ReportingTaxonomyMap	继承自 <i>ItemSchemeMap</i>	关联源和目标报告分类

表 23 (续)

类	特征/属性	说明
	/source	关联到源报告分类
	/target	关联到目标报告分类
ReportingCategoryMap	继承自 <i>ItemAssociation</i>	关联源和目标报告分类
	/source	关联到源报告分类
	/target	关联到目标报告分类

12.5 混合代码表映射

12.5.1 类图

混合代码表映射的类图如图 34 所示。

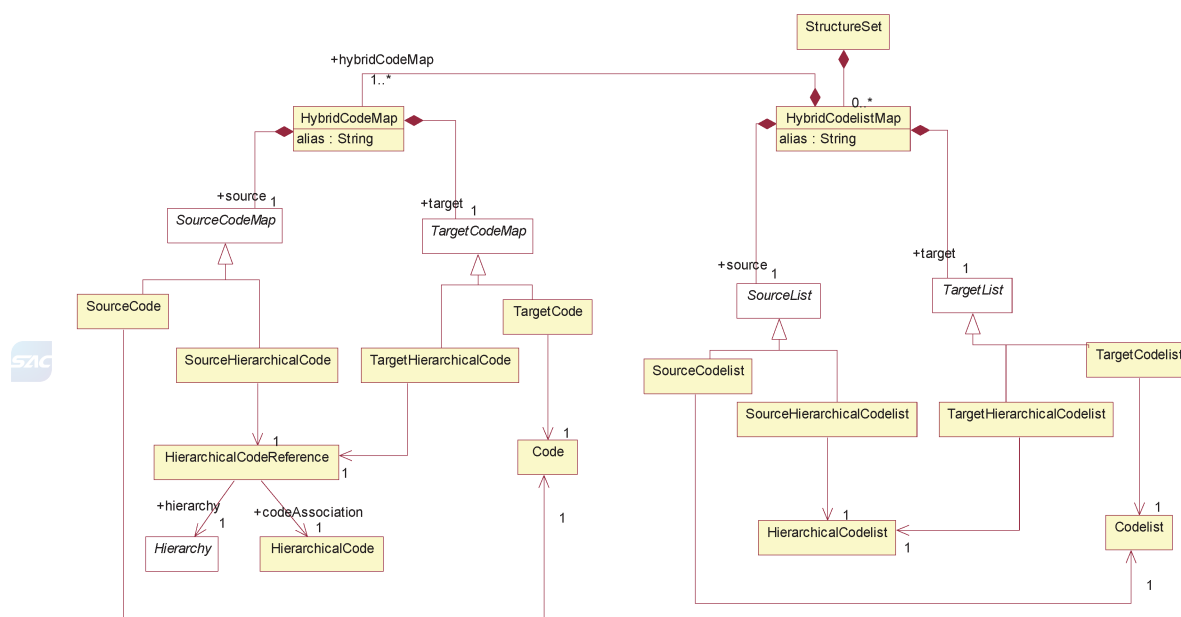


图 34 混合代码表映射的类图

12.5.2 类图说明

12.5.2.1 描述

HybridCodeListMap(混合代码表映射)映射 CodeList(代码表)和 HierarchicalCodeList(层级代码表)的内容。它包含了两个方案(HybridCodeMap(混合代码映射))中代码之间的映射。HybridCodeMap(混合代码映射)可以将一个 Code(代码)或 HierarchicalCode(层级代码)映射到另一个 Code(代码)或 HierarchicalCode(层级代码)。HierarchicalCode(层级代码)由 Hierarchy(层级)和 HierarchicalCode(层级代码)的组合进行标识。

12.5.2.2 混合代码表映射等类及其属性说明

混合代码表映射等类及其属性如表 24 所示。

表 24 混合代码表映射等类及其属性表

类	特征/属性	说明
HybridCodelistMap	继承自 <i>NameableArtefact</i>	关联代码表和层级代码表
	alias	映射的另一种标识,通过共享此值就可以表达多个映射关系
	+ source	关联到源列表
	+ target	关联到目标列表
	+ hybridCodeMap	关联到混合代码表映射中的混合代码映射的集合
SourceList	抽象类 子类 SourceCodelist SourceHierarchical Codelist	
TargetList	抽象类 子类 TargetCodelist TargetHierarchical Codelist	
SourceCodelist		标识了作为映射源的代码表
TargetCodelist		标识了作为映射目标的代码表
SourceHierarchical Codelist		标识了作为映射源的层级代码表
TargetHierarchical Codelist		标识了作为映射目标的层级代码表
HybridCodeMap	继承自可注释产物 (<i>AnnotableArtefact</i>)	关联了混合代码表映射中的源和目标代码
	+ source	关联了源代码映射
	+ target	关联了目标代码映射
SourceCodeMap	抽象类 子类 SourceCode SourceHierarchical Code	

表 24 (续)

类	特征/属性	说明
<i>TargetCodeMap</i>	抽象类 子类 TargetCode TargetHierarchical Code	
SourceCode		标识了作为映射源的代码
TargetCode		标识了作为映射目标的代码
SourceHierarchical Code		标识了作为映射源的层级代码
TargetHierarchical Code		标识了作为映射目标的层级代码
HierarchicalCode Reference		同时引用了层级代码表中的层级和 层级代码
	+ hierarchy + codeAssociation	关联了层级代码表的层级中的层级 代码

13 约束

13.1 范围

本章描述元模型对数据源的内容和访问的支持。信息可以存储在一个资源中,如通过应用程序所使用的注册库,此应用程序可以通过因特网定位数据和元数据。约束还适用于规定 Codelist(代码表)的子集,该子集可用作代码列表的一部分,而这与约束所依附的产物语境相关,例如,数据结构定义、数据流、供应协议。

注:在该元模型中,数据源这个术语指数据和元数据的源,数据提供者指数据和元数据的供应者。

数据源可以是数据或元数据的简单文件(以 SDMX-ML 格式),也可以是数据库或元数据仓库。数据源可以包含适用于许多数据或元数据流(在本模型中称为 DataflowDefinition(数据流定义)和 MetadataflowDefinition(元数据流定义)的数据,本章所描述的机制允许组织准确地规定数据源在注册时其内容范围(SimpleDataSource(简单数据源)、QueryDataSource(查询数据源))。

DataflowDefinition(数据流定义)和 MetadataflowDefinition(元数据流定义)本身可以规定为仅包含衍生自 DSD(数据结构定义)或 MSD(元数据结构定义)的所有可能键的子集。

在本模型中它们被称为“约束”。

13.2 继承

13.2.1 可约束产物的继承类图

可约束和供应产物的继承类图如图 35 所示。

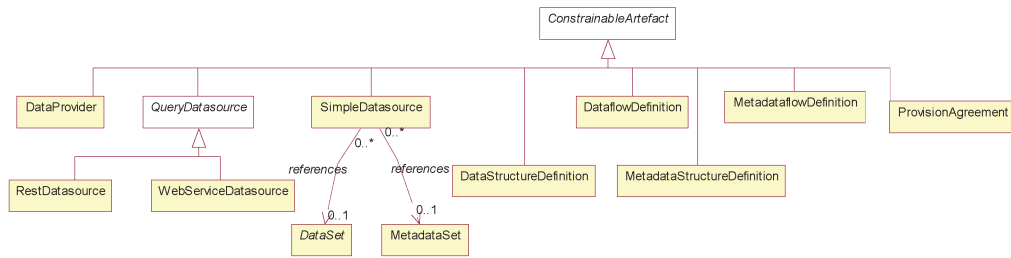


图 35 可约束和供应产物的继承类图

13.2.2 类图说明

任何衍生自 *ConstrainableArtefact* (可约束产物) 的产物都具有已定义好的约束。具有约束元数据的产物为：

- DataflowDefinition* (数据流定义)；
- ProvisionAgreement* (供应协议)；
- DataProvider* (数据提供者)：受限于发布日期；
- MetadataflowDefinition* (元数据流定义)；
- DataStructureDefinition* (数据结构定义)；
- MetadataStructureDefinition* (元数据结构定义)；
- DataSet* (数据集)；
- SimpleDataSource* (简单数据源)，已注册的数据源，其中的注册引用了实际的数据集或元数据集；
- QueryDataSource* (查询数据源)。

注：由于 *Constraint* (约束) 可以规定某一具体 *Structure* (结构) (如某一个具体的 DSD 或 MSD) 所隐含的组件值的子集，所以 *ConstrainableArtefact* (可约束产物) 应与某一个具体的 *Structure* (结构) 相关联。尽管 *Constraint* (约束) 本身可以不直接链接到 DSD 或 MSD，但受约束的产物将链接到 DSD 或 MSD。由于数据提供者不链接到任何一个具体的 DSD 或 MSD，因此，链接到 *DataProvider* (数据提供者) 的 *Constraint* (约束) 中所包含的信息类型只限于 *Release Calendar* (发布日期)。

13.3 约束

13.3.1 关系类图的高阶视图

约束元数据的关系类图如图 36 所示。

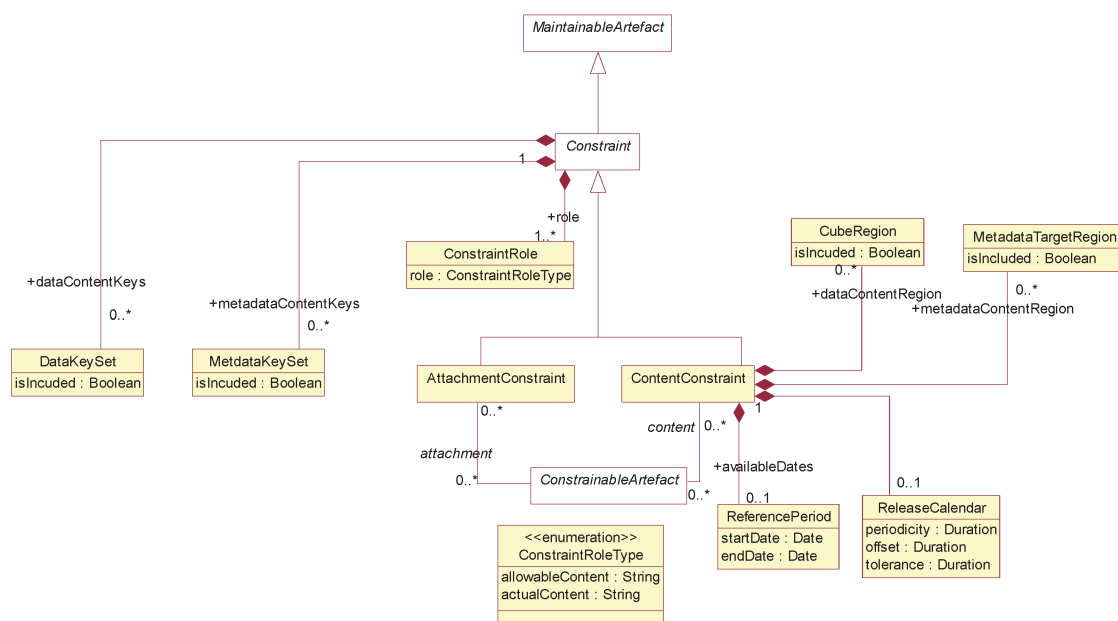


图 36 约束元数据的关系类图

13.3.2 类图说明

约束机制允许具体的约束依附于 *ConstrainableArtefact* (可约束产物)。除了 *ReferencePeriod* (引用期) 和 *ReleaseCalendar* (发布日期) 以外, 这些约束规定了在任意 *ConstrainableArtefactS* (可约束产物集) 中可能出现的值或键的总集的子集。

例如, DSD 为每个维度 (Dimension) 规定了允许代码值的列表。然而, 使用 DSD 的具体 *DataflowDefinition* (数据流定义) 可能只包含了来自于 DSD (数据结构定义) 的键范围的子集 (这种可能性的总范围有时候称为维度值的笛卡尔乘积)。除此之外, 根据 *DataflowDefinition* (数据流定义) 能够供应数据的 *DataProvider* (数据提供者) 拥有 *ProvisionAgreement* (供应协议), *DataProvider* (数据提供者) 也有可能提供约束信息, 这些信息可能会进一步约束可能性的范围, 以描述该供应者能够供应的数据。依据所包含的 *KeySet* (键集) 或 *CubeRegion* (立方体区) 来描述数据源的内容也是有用的。

ConstrainableArtefact (可约束产物) 可以拥有两种类型的 *Constraint* (约束):

- ContentConstraint* (内容约束): 仅用作一种机制来规定键的有效集合 (*DataKeySet* (数据键集)、*MetadataKeySet* (元数据键集)), 或规定了 *DataSource* (数据源) 中组件值的集合 (*CubeRegion* (立方体区域)、*MetadataTargetRegion* (元数据目标区)), 这些数据源可以是 (数据集) 或数据库 (*QueryDataSource* (查询数据源)) 等; 或规定了可以构造自 DSD 的可允许的键。对于 *ConstrainableArtefact* (可约束产物) 可以出现多个约束。例如, 可能存在为 *ConstrainableArtefact* (可约束产物) (*Role* (角色) 是 *allowableContent* (允许的内容)) 规定允许值的 *ContentConstraint* (内容约束), 此产物可以用于校验, 或用于构建一个部分代码表, 然而另一项约束可以规定数据或元数据源的实际内容 (*Role* (角色) 是 *actualContent* (实际内容))。
- AttachmentConstraint* (附加约束): 用作一种机制以定义数据完整集的切片, 或元数据可以依附在数据集或元数据集中。这些切片可以定义为键的集合 (键集 (*KeySet*)) 或组件值的集合 (立方体区 (*CubeRegion*))。某一具体的 *AttachableArtefact* (可附加的产物) 可以具有多个 *AttachmentConstraint* (附加约束)。

除了 *DataKeySet* (数据键集)、*MetadataKeySet* (元数据键集)、*CubeRegion* (立方体区)、*Metadata-*

TargetRegion(元数据目标区)之外,Constraint(约束)可以拥有 ReferencePeriod(引用期限),引用期限定义了多个日期范围(有效期(ValidityPeriod))中的某一个,这些日期范围为 ConstrainableArtefact(可约束产物)中的可得到的哪些数据或元数据规定了时间期限;以及在数据发布或报告时,引用期限还拥有 ReleaseCalendar(发布日期)。

13.3.3 关系类图

13.3.3.1 类图

键集约束类图如图 37 所示、立方体区和元数据目标区约束如图 38 所示。

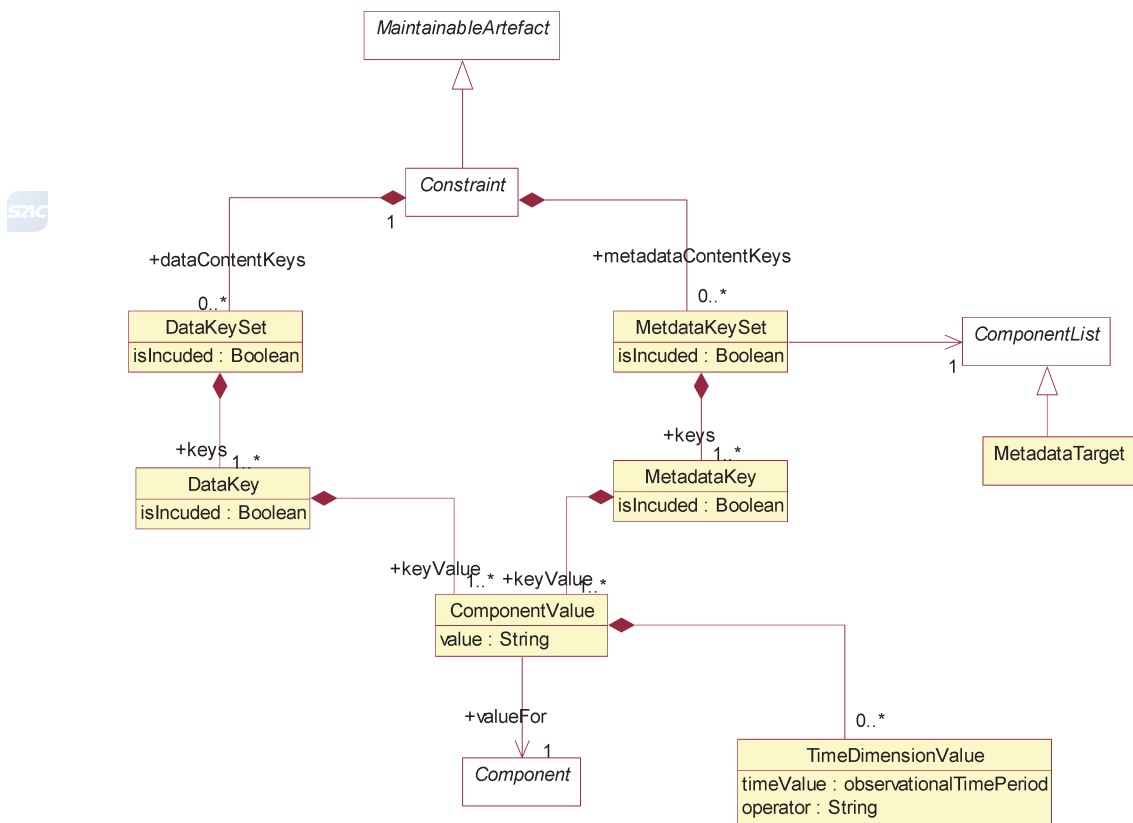


图 37 键集约束

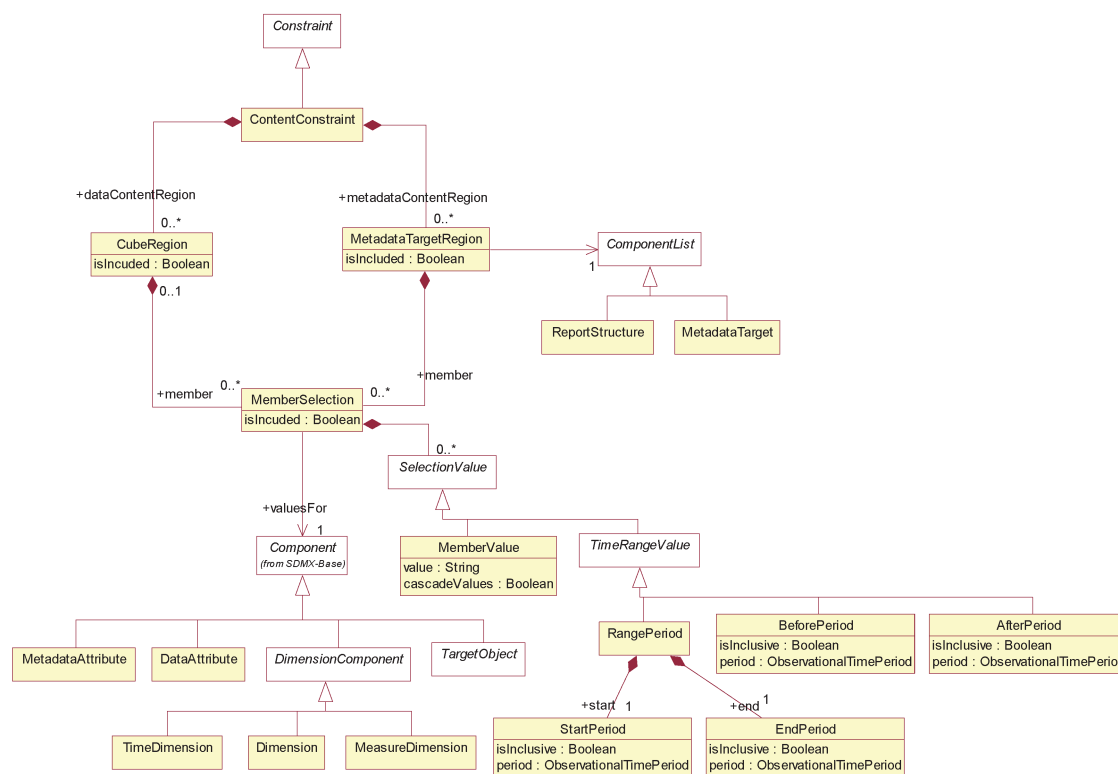


图 38 立方体区和元数据目标区约束

13.3.3.2 类图说明

Constraint (约束) 是 *MaintainableArtefact* (可维护产物)。

Constraint (约束) 可以用两种方式规定值子集：

- 以 *DataSet* (数据集) (即 *DataKeySet* (数据键集) 或 *MetadataSet* (元数据集) (即 *MetadataKeySet* (元数据键集)) 表示的键的集合。每个 *DataKey* (数据键) 或 *MetadataKey* (元数据键) 都规定了一系列的 *ComponentValues* (组件值), 其中每个组件值都引用一个组件 (例如, *Dimension* (维度)、*TargetObject* (目标对象))。当结构的 *Component* (组件) 包含于 *DataSet* (数据集) 或 *MetadataSet* (元数据集) 时, 则每个组件值是这些组件的一个值。 *MetadataKeySet* (元数据键集) 还应标识 *MetadataTarget* (元数据目标), 因此, 在 MSD 中可能存在多个元数据目标。对于 *DataKeySet* (数据键集) 来说, 由于只有一个 *DimensionDescriptor* (维度描述符) 和一个 *AttributeDescriptor* (属性描述符), 因此, 这样的标识是没有必要的;
- 作为一组 *CubeRegions* (立方体区) 或 *MetadataTaregetRegions* (元数据目标区), 其中的每一个都依据一个或多个 *MemberValues* (成员值) 来定义整体结构 (*MemberSelection* (成员选择)) 的一个“切片”, 当结构的 *Component* (组件) 出现在 *DataSet* (数据集) 或 *MetadataSet* (元数据集) 时, 这些切片可以为结构的组件而出现。

上述 a) 与 b) 的差别在于: a) 定义了一个完整的键, 而 b) 中的“切片”为每个 *Components* (组件) 定义了一个可能值的列表, 但是没有规定具体键的组合。此外, a) 中 *Component* (组件) 与 *DataKeyValue* (数据键值) 或 *MetadataKeyValue* (元数据键值) 之间的关联限定于组成键或标识符的组件, 而 b) 中则可以包含其他组件类型 (诸如属性)。 *ComponentValue.value* 和 *MemberValue.value* 中的值应与在 DSD (数据结构定义) 或 MSD (元数据结构定义) 中为该 *Com-*

ponent(组件)所声明的 Representation(表示法)相一致。

注：在所有情况下，value(值)上的“运算符”被认定为是“等价的”。此外，在 MemberValue(成员值)中，可以通过 cascadeValues(瀑布值)属性的方式来规定包含在约束中的子值(例如子代码)。

无论 DataKeySet(数据键集)、MetadataKeySet(元数据键集)、MetadataKey(元数据键)、CubeRegion(立方体区)、MetadataTargetRegion(元数据目标区)和 MemberSelection(成员选择)是否(isincluded = "真", isincluded = "假")包含在约束定义中，都可以对这些集合进行定义。例如，如果只有可能值的一个小子集没有包括在该集合中，则该小的子集可以进行定义而且不包含在该约束中，则该属性是有用的。

注：如果子构造是“包含于”而父构造是“不包含于”，则子构造包含在“不包含于”的构造列表中。

13.3.3.3 可约束产物等类及其属性说明

可约束产物等的属性如表 25 所示。

表 25 可约束产物等的属性表

类	特征/属性	说明
Constrainable Artefact 	抽象类 子类为： DataflowDefinition Metadataflow Definition ProvisionAgreement DataProvider QueryDatasource SimpleDatasource DataStructure Definition MetadataStructure Definition	具有规定约束的产物
	content	与约束了数据或元数据源中的内容的元数据的关联，而数据或元数据源链接到可约束产物
	attachment	与约束了可约束产物(元数据所依附的)的有效内容的元数据的关联
Constraint	继承自 MaintainableArtefact 抽象类 子类为： AttachmentConstraint ContentConstraint	规定了数据或元数据允许或实际内容定义的子集，数据或元数据源从定义代码表和其他有效内容的结构中衍生而来
	+availableDates	与标识了数据或元数据在数据源中时间范围的时间段的关联

表 25 (续)

类	特征/属性	说明
	+ dataContentKeys	与数据键集(即值组合)的子集的关联,该数据键集可以可约束产物所链接的结构定义衍生而来
	+ metadataContentKeys	与元数据键集(即键组合)的子集的关联,该元数据键集可以从可约束产物所链接的结构定义衍生而来
	+ dataContentRegion	关联到组件值的子集,该组件值可以从可约束产物所链接的数据结构定义衍生而来
	+ metadataContentRegion	关联到组件值的子集,该组件值可以从可约束产物所链接的元数据结构定义衍生而来
ContentConstraint	继承自 <i>Constraint</i>	根据数据或元数据源中的内容定义约束,数据或元数据源链接到约束所关联的可约束产物
	+ role	关联到约束所扮演的角色
ConstraintRole		根据其用途规定约束内容类型的方法
	allowableContent	约束所包含的组件值或键的有效子集的规定
	actualContent	根据该源中的组件值或键,约束所包含的数据或元数据源实际内容的规定
Attachment Constraint	继承自 <i>Constraint</i>	根据数据源中组件值组合以及结构定义中所关联的可约束产物来定义约束
DataKeySet		一组数据键
	isIncluded	指明了数据键集是否包含在此约束定义中
	+ keys	关联到集合中的数据键
MetadataKeySet		一组元数据键
	isIncluded	指明了元数据键集是否包含在此约束定义中
	+ keys	关联到集合中的元数据键

表 25 (续)

类	特征/属性	说明
DataKey		数据集中的键值
	isIncluded	指明了数据键是否包含在此约束定义中
	+keyValue	关联了构成此键的组件值
MetadataKey		元数据集中的键值
	isIncluded	指明了元数据键是否包含在此约束定义中
	+keyValue	关联了构成此键的组件值
ComponentValue		键的组件标识和值(例如,维度)
	value	组件的值
	+valueFor	关联到可约束产物所链接的结构中的组件(例如,维度)
TimeDimensionValue		时间维度组件的值
	timeValue	时间段的值
	operator	指出规定的值是精确的时间或时间段,或者还是范围。 一个大于(greaterThan)或大于等于(greaterThanOrEqual)的属性值,指出该值为范围的开始值(分别对应于不包含开始值或包含开始值)。 一个小于(lessThan)或小于等于(lessThanOrEqual)属性的值,指出该值为范围的结束值(分别对应于不包含结束值或包含结束值)。 在范围的一端边界没有规定的情况下,此边界视为无限(也就是,在大于等于(greaterThanOrEqual)属性规定的时间段开始之后的任意时间段)
CubeRegion		一组组件及其值,它们定义了数据结构一个子集或“切片”,而该数据结构与可约束产物链接
	isIncluded	指出立方体区是否包含在该约束定义中
	+member	与定义了值的子集的组件集的关联
MetadataTargetRegion		一组组件及其值,它们定义了元数据结构一个子集或“切片”,而该元数据结构与可约束产物链接

表 25 (续)

类	特征/属性	说明
	isIncluded	指出元数据目标区是否包含在此约束定义中
	+ member	与定义了值的子集的组件集的关联
MemberSelection		适用于该轴的某个组件的一组允许值
	isIncluded	指出成员选择类是否包含在此约束定义中
	+ valuesFor	关联到可约束产物所链接的结构中的组件,它们定义了成员值的有效表示法
MemberValue		成员选择的值集合中的一个单独值
	value	成员的一个值
	cascadeValues	指出成员选择中成员的子节点(例如子代码)
TimeRangeValue	继承自 SelectionValue 抽象类 具体类 BeforePeriod AfterPeriod RangePeriod	规定了约束选择有效的日期的时间值
BeforePeriod	继承自 TimeRangeValue	约束选择在此之前的时间段有效
	isInclusive	指出此日期是否包含在该时间段内
AfterPeriod	继承自 TimeRangeValue	约束选择在此之后的时间段有效
	isInclusive	指出日期是否包含在该时间段中
RangePeriod		日期范围的起始期和结束期
	+ start	关联到起始期
	+ end	关联到结束期
StartPeriod	继承自 TimeRangeValue	约束选择从此时间段起有效
	isInclusive	指出此日期是否包含在该时间段内
EndPeriod	继承自 TimeRangeValue	约束选择有效终止的时间段

表 25 (续)

类	特征/属性	说明
	isInclusive	指出此日期是否包含在该时间段内
ReferencePeriod		一组日期,这组日期约束了数据或元数据集中的内容
	startDate	时间段的起始日期
	endDate	时间段的结束日期
ReleaseCalendar		数据或元数据发布或报告的时间表
	periodicity	数据或元数据各次发布之间的时间段
	offset	数据首次发布与 1 月 1 日之间的间隔期
	tolerance	在此时间段之后认为数据或元数据是延误的



14 数据供应

14.1 类图

数据供应的关系和继承类图如图 39 所示。

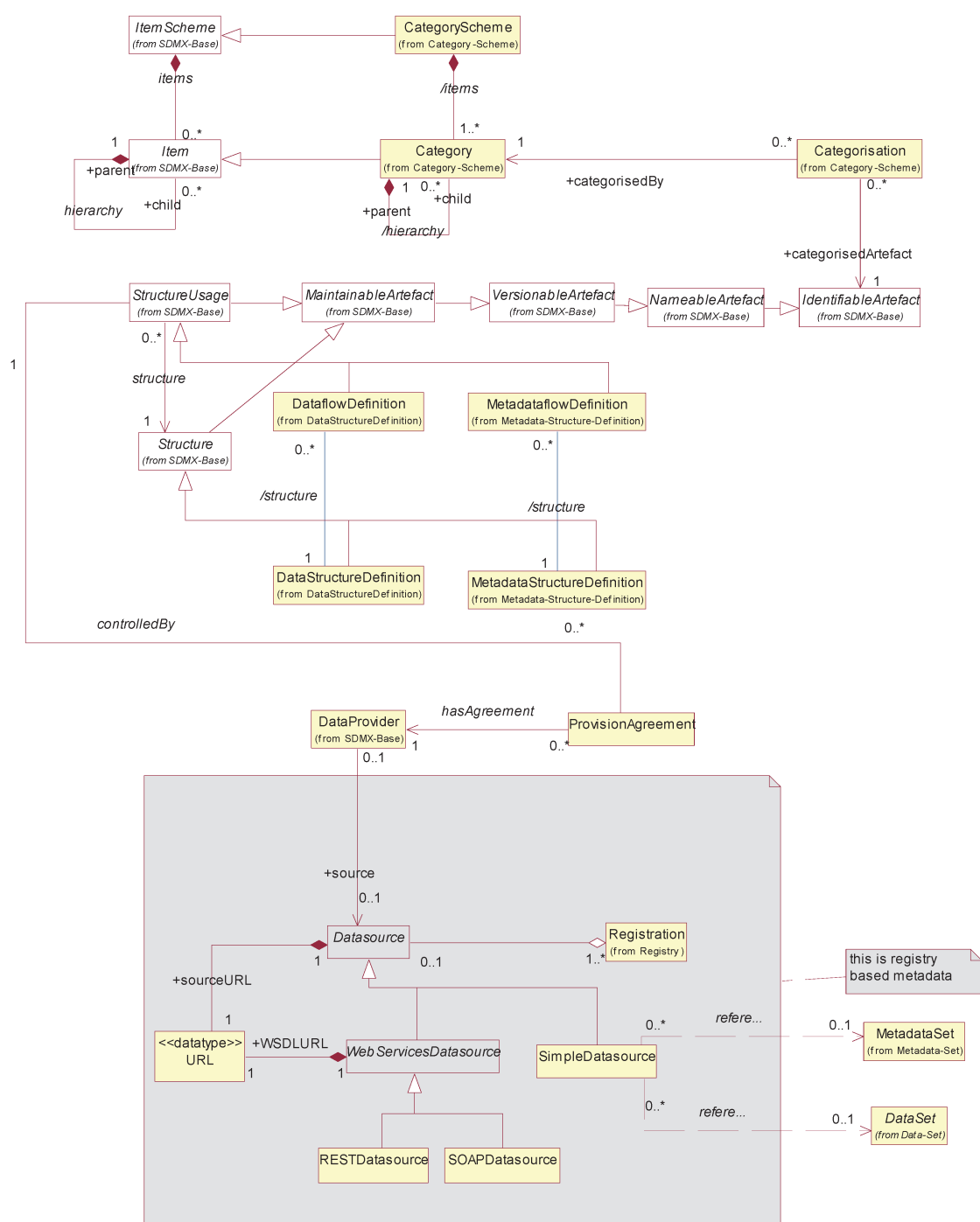


图 39 数据供应的关系和继承类图

14.2 类图说明

14.2.1 描述

本子模型将 SDMX-IM 中的许多产物链接起来，并且是 SDMX 元数据注册库的中枢，因为本子模型的所有产物应是负责数据和元数据注册的应用程序能够访问的，或是访问数据或元数据的应用程序所使用的。

尽管注册库包含了图 37 中描绘的所有元数据，但是灰色阴影区域中的类对于基于注册库的场景是

特定的。在这些场景中,数据源(实体数据和元数据集或数据库和元数据仓库)得以注册。SDMX 注册接口文档(SDMX 标准的第 5 部分)针对这些类在注册库场景中如何使用进行了详细介绍。

ProvisionAgreement(供应协议)将一些产物联接起来,对于 DataProvider(数据提供者)来说,这些产物定义了数据和元数据是如何构造和分类的(*StructureUsage*(结构用法)),并且通过数据或元数据的注册,供应协议引用了 Datasource(数据源)(可以是数据或元数据)。供应协议可以是网站(Simple-Datasource(简单数据源))上遵循 SDMX 的文件,也可以是支持 SDMX 查询并对遵循 SDMX 的文档做出响应的数据库服务(QueryDatasource(查询数据源))。

拥有 DataflowDefinition(数据流定义)和 MetadataflowDefinition(元数据流定义)具体类的 StructureUsage(结构用法)标识出了对应的 DSD(数据结构定义)或 MSD(元数据结构定义),并且通过 Categorisation(分类),可以链接到分类方案中的一个或多个 Category(分类),诸如主题域方案,通过这种方式就可以对 StructureUsage(结构用法)进行分类。这样有助于从主题域向下钻取可能相关的数据或元数据。

SimpleDatasource(简单数据源)链接到网站上的实际 DataSet(数据集)或 MetadataSet(元数据集)(这在图中表示为“引用”的依赖关系)。sourceURL(源 URL)在 DataSet(数据集)或 MetadataSet(元数据集)的注册过程中获得。其他 SimpleDatasource(简单数据源)内容的信息按照 Registration(注册)的 ContentConstraint(内容约束)存储在注册库中。

QueryDatasource(查询数据源)是表示数据源的抽象类,该数据源可以理解 SDMX-ML 查询(SOAP-Datasource(SOAP 数据源))或 RESTful 查询(RESTDatasource(REST 数据源))并能正确做出响应。这些数据源都从 Datasource(数据源)继承了 dataURL(数据 URL),而 QueryDatasource(查询数据源)则拥有另一个 URL 以定位一个 WSDL 或 WADL 文档,以描述如何访问它。其他所有支持的协议均假设使用 SimpleDatasource(简单数据源)的 URL。

图 40 是 SDMX 结构产物最终链接到数据或元数据注册的导向关系示意图。



表 26 (续)

类	特征/属性	说明
ProvisionAgreement		将数据提供者链接到相关的结构用法(例如,数据流定义或元数据流定义),而供应者为其提供数据或元数据。此协议可以通过 Constraint(约束)的方法来约束所提供的数据或元数据的范围
	+ source	关联到能够对数据或元数据查询进行处理的数据或引用元数据源
Datasource	抽象类 子类为: SimpleDatasource WebServices Datasource	可以获取数据或引用元数据的位置或服务的标识
	+ sourceURL	数据或引用元数据源(文件或 web 服务)的 URL
SimpleDatasource		可以作为文件在 URL 访问到的 SD-MX-ML 数据集
WebServices Datasource	抽象类 继承自: Datasource 子类为: RESTDatasource SOAPDatasource	可以对数据或元数据查询进行处理的数据或引用元数据源
RESTDatasource		通过 RESTful web 服务接口可以访问的数据或引用元数据源
SOAPDatasource		符合 SOAP web 服务接口的数据或引用元数据源
	+ WSDLURL	关联到 web 服务的 Web 服务定义语言(SOAP)或 Web 服务应用语言(REST)配置文件的 URL
Registration		这里仅显示了 SDMX-IM 与注册服务 API 之间的链接,并指出数据或元数据注册文档

15 过程

15.1 简介

在任何处理数据和引用元数据的系统中,系统本身就是一系列的过程,在每个过程中,数据和引用

元数据都可能会经历一系列的转化。原始数据转化到发布数据和引用元数据的路径更是如此。此处的过程模型是通用性模型,它可以通过链接到特定的可标识对象以及所使用的标识软件组件,以一种文本性方式,或以一种更为形式化的方式来捕获这些环节的关键信息。

15.2 模型-继承和关系视图

15.2.1 类图

过程和转化的继承和关系类图如图 41 所示。

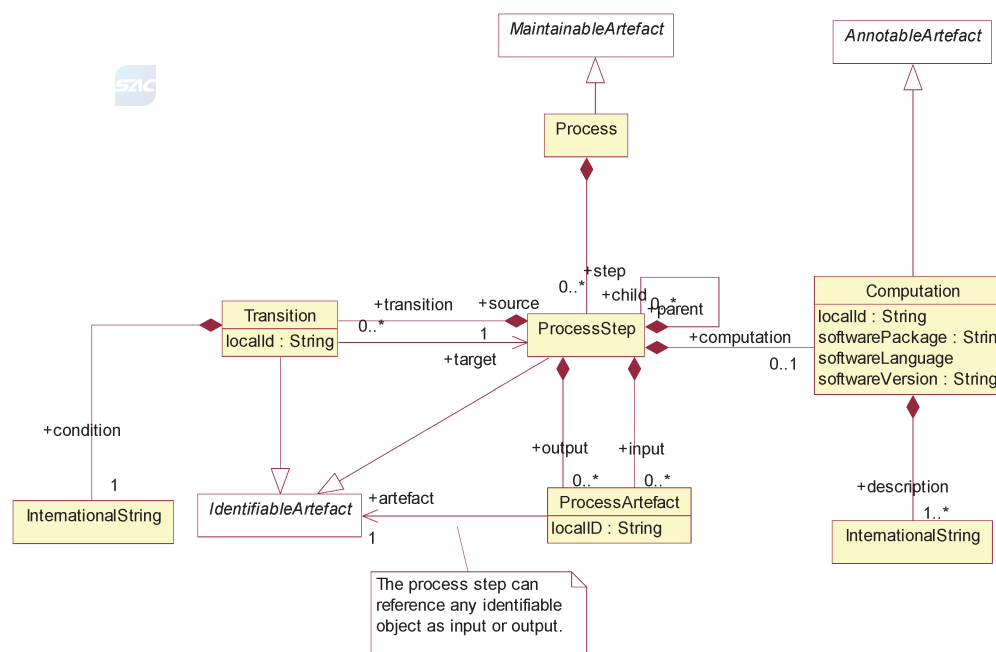


图 41 过程和转化的继承和关系类图

15.2.2 类图说明

15.2.2.1 描述

Process(过程)就是一组有层级的 ProcessStepS(过程步骤)。每个 ProcessStep(过程步骤)的输入和输出可以是零个或多个可标识物。与输入和输出的可标识物(ProcessArtefact(过程产物))的每一个关联都可以分配一个 localID(本地 ID)。

由 ProcessStep(过程步骤)所执行的计算可以有选择地由 Computation(计算)进行描述,而计算可以标识出 ProcessStep(过程步骤)所使用的软件,也可以进行多语种文本形式(+description)的描述。Transition(转化)描述了从+source ProcessStep(源过程步骤)到+target ProcessStep(目标过程步骤)的 ProcessStepS(过程步骤)的执行,这种描述是基于+condition(条件)的过程步骤结果的,且+condition(条件)可以以多语种描述。

15.2.2.2 过程和过程步骤等类及其属性说明

过程和过程步骤等类及其属性如表 27 所示。

表 27 过程等类及其属性表

类	特征/属性	说明
Process	继承自 <i>Maintainable</i>	定义或记录数据或元数据运算的操作方案,目的是验证数据或元数据根据给定的规则集合以获得新信息
	+ step	关联过程步骤
ProcessStep	继承自 <i>IdentifiableArtefact</i>	对数据或元数据执行特定的运算,目的在于根据给定的一组规则验证或衍生出新的信息
	+ input	关联到作为过程步骤输入对象的过程产物(Process Artefact)
	+ output	关联到作为过程步骤输出对象的过程产物(Process Artefact)
	+ child	关联到组合成该过程的子过程
	+ computation	关联到一个或多个计算
	+ transition	关联到一个或多个转化
Computation		以文本形式描述的过程中的计算
	localId	区分同一过程中不同的计算
	softwarePackage softwareLanguage softwareVersion	执行计算的软件信息
	+ description	描述或给出有关计算辅助信息的文本,可以是多语种的
Transition	继承自 <i>IdentifiableArtefact</i>	以文本或形式化方式表达的数据在两个特定运算(过程)之间的转换
	+ target	关联到过程步骤,该过程步骤是转化的目标
	+ condition	关联了转化的文本描述
ProcessArtefact		对象的标识,该对象是过程步骤的输入或输出
	+ artefact	关联到可标识物,该标识物是过程步骤的输入或输出

16 转换和表达式

16.1 范围

本模型包目的是追踪数据的衍生情况。它在概念上类似于数据仓库技术中的谱系,也就是,数据是

如何衍生的。

这部分模型的功能是标识和记录所执行的计算(通常为自动化的程序计算),并定义独立于语法表达式的结构,该语法可以在一定粒度水平上规定运算,以便程序可以“读取”元数据同时以任何合适的计算机语言构造出所需的表达式。

这部分模型还允许规定和记录不同数据之间的一致性规则,并采用计算式进行表达(如," $a+b=c$ "的一致性规则可以写作" $a+b-c=0$ ",并通过计算" $\text{if}((a+b-c)=0, \text{then}..., \text{else}...)$ "来进行校验)。

注:该模型在范围和内容上类似于 Object Management Group (OMG)的通用仓库元模型(CWM)规范(网址:<http://www.omg.org/cwm>)中的表达式元模型。表达式元模型在 CWM 规范的第 1 部分中的第 8.5 节中进行了描述。图 40 采用 SDMX-IM 基本类描述了 CWM 表达式元模型。

16.2 模型-继承视图

16.2.1 类图

转换类的继承类图和关系类图如图 42 所示。

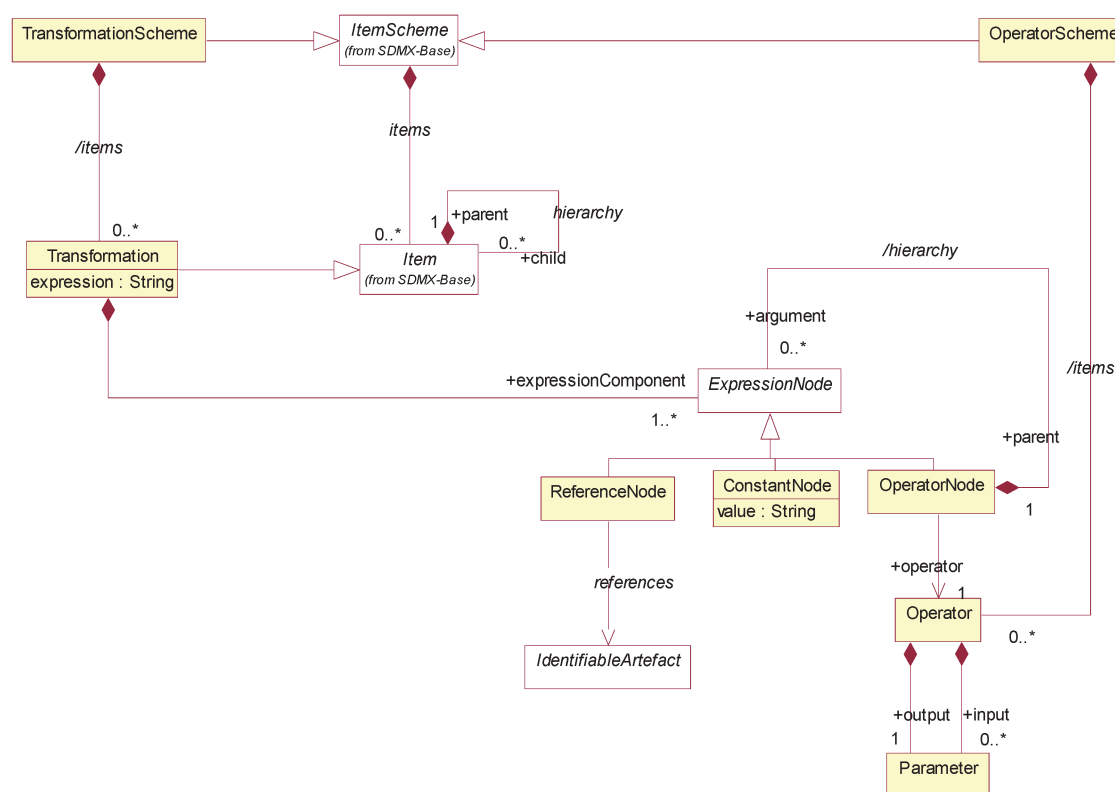


图 42 转换类的继承类图和关系类图

16.2.2 类图说明

16.2.2.1 描述

与本模型相关的 *ItemScheme*(项方案)具有以下 3 种类型:

- TransformationScheme*(转换方案)包含了一个或多个 *Transformations*(转换);
- OperatorScheme*(运算符方案)包含一个或多个 *Operators*(运算符);
- ExpressionNodeScheme*(表达式节点方案)包含一个或多个 *ExpressionNodes*(表达式节点)。

该模型是适用于表达式和转换的一个基本框架,但是它如何完整地融合到信息模型中并具有实用性还需要做大量工作。本标准的后续版本将完成这些工作。

SDMX-IM 中的表达式概念采用了树形功能视图,这使得可以用较少的表达式节点类型来表示范围宽广的表达式。表达式层级中的每个函数或数学运算符是由关联到 Operator(运算符)的 +operator 角色来表示的,它们构成了输入和输出 Parameters(参数集)。例如,算术加法运算“ $a+b$ ”可以看作是函数“ $\text{sum}(a, b)$ ”,“sum”是 Operator(运算符),“a”和“b”是它的 Parameter(参数)。参数通常是运算符的输入和输出(例如,基和指数是幂运算符的参数),当参数为特定值时,则此参数参与具体的计算(例如,在爱因斯坦公式“ $E=MC^2$ ”中,“幂”运算的参数是“C”(基)和“2”(指数))。特定函数或运算的实际语义将由具体工具的实现去解决,SDMX-IM 对此不予考虑。

表达式的 SDMX-IM 表示法的层级是通过 OperatorNode(运算符节点)关联的递归来实现。这种关联允许表达式中的子层级作为其父节点的实际参数来处理。

本模型可以用于定义数据衍生,还可以用于定义完整性校验(例如, $A+B$ 之和应等于 C)。

尽管本模型定义了与语法无关的表达式数据结构,但模型本身并没有规定一种与语法无关的表达式语法。作为一种选择,该函数也可以用文本形式来描述,描述时既可以采用函数的无结构化的解释,也可以采用诸如 BNF 的更为正式的语言。

注:BNF 是指巴克斯诺尔范式。

数据结构的工作方式如下:

实际基本数学函数(例如,加、乘、除、赋值(=)、<、>等)定义为 OperatorScheme(运算符方案)中的 Operators(运算符)。对于每个 Operator(运算符)而言,输入和输出 ParameterS(参数)均在 Parameters(参数)类中进行定义。

这些计算被定义为 TransformationScheme(转换方案)中的 Transformations(转换)。转换是一种特殊的计算并通过一个表达式来规定,这个表达式通过有次序的一个或多个 Operators(运算符)(例如,在文本形式上使用圆括号),并规定运算符参数(OperatorS'Parameters)的实际参数来获得;整个表达式的结果被赋予(=)作为转换结果的模型项(在爱因斯坦公式中为“E”)。Transformations(转换)在现有的 IdentifiableArtefacts(可标识物)基础上实施,其结果是另一个 IdentifiableArtefact(可标识物)。而计算出的 IdentifiableArtefact(可标识物)有可能随后成为其他转换的操作数。

Transformation(转换)表达式(例如,爱因斯坦积分公式“ $E=M*(C*2)$ ”)可以分解成一个 ExpressionNodes(表达式节点)的层级(本例中,表达式节点是“M”,“C”,“2”,*,*)。ExpressionNode(表达式节点)可以是 ReferenceNode(引用节点)、ConstantNode(常数节点)或 OperatorNode(运算符节点)。ReferenceNode(引用节点)引用了 identifiable model artefact(可标识模型产物)(本例中,可标识模型产物是“M”和“C”)。ConstantNode(常数节点)由一个常数值来定义(本例中,常数节点为“2”)。OperatorNode(运算符节点)引用了 OperatorScheme(运算符方案)中的 Operator(运算符)(本例中,运算符为*,*)。Transformation(转换)拥有指向其组件 ExpressionNodes(表达式节点)的关联。

ExpressionNodes(表达式节点)的层级表达了操作符在表达式中应用的顺序,并通过 OperatorNode(运算符节点)类的/hierarchy 关联获得,其中子 ExpressionNode(表达式节点)就是父 OperatorNode(运算符节点)的参数。子 ExpressionNodes(表达式节点)应以正确的顺序符合父 OperatorNode(运算符节点)所引用的 Operator(运算符)的形式参数。(子)ExpressionNode(表达式节点)可以是另一项运算(就是另一个 OperatorNode((运算符节点))的结果,也可以是一个 Constant(常数),还可以是一个 IdentifiableArtefact(可标识物(ReferenceNode(引用节点))的引用。SDMX-IM 中的所有 IdentifiableArtefacts(可标识物)都拥有唯一的统一资源名称(urn),该 urn 包含了标识它的每个对象的值。urn 的结构在注册规范中进行定义。下面是包含了 agency: code-list-id.code-id 的 urn,实际示例是“urn:sdmx:org.sdmx.infomodel.codelist.Code=TFFS:CL_AREA(1.0).1A”。

16.2.2.2 转换方案等类及其属性说明

转换方案等类及其属性如表 28 所示。

表 28 转换方案等类及其属性表

类	特征/属性	说明
TransformationScheme	继承自 <i>ItemScheme</i>	定义或记录了为衍生或验证来自其他数据的数据而要求进行转换的方案
Transformation	继承自 <i>Item</i>	单一的转换
	+expressionComponent	与表达式节点的关联
ExpressionNode	抽象类 子类 ReferenceNode ConstantNode OperatorNode	可能节点层级中的一个节点,层级节点共同定义或记录了表达式
	/hierarchy	与子表达节点的关联
ReferenceNode	继承自 <i>ExpressionNode</i>	引用了特定对象的一个特定类型的表达式节点
	references	与所引用对象的可标识物的关联
ConstantNode	继承自 <i>ExpressionNode</i>	包含常数值 of 特定类型的表达式节点
	value	常数值
OperatorNode	继承自 <i>ExpressionNode</i>	引用了运算符 of 特定类型的表达式节点
	+operator	关联到运算符,该运算符定义了运算符节点 of 数字运算符
	+arguments	关联到运算符节点的数学参数
OperatorScheme	继承自 <i>ItemScheme</i>	定义了数学运算符的方案
Operator	继承自 <i>Item</i>	运算符方案中的数字运算符
	+input	关联到运算符的输入参数
	+output	关联到运算符的输出参数
Parameter		运算符的输入或输出